Documentation-Driven Threat Analysis

Base Analysis Guide

Rebuild R7 - Candidate cumulative reconstruction

Ricostruzione progressiva della guida Base Analysis

Ogni costrutto viene ricostruito a partire dall'authority BA corrente e dalle guide precedenti, spiegato in forma generale e soltanto dopo verificato contro esempi aggiornati. Il caso DermaTriage e' evidence applicativa e worked example: non e' la fonte da cui nasce la semantica del costrutto.

RIQUADRO TEMPORANEO - WORKING SET E RIFERIMENTI - DA CANCELLARE PRIMA DELLA GUIDA DEFINITIVA

Attivi: `DDTA_DOCUMENTATION_AUTHORING_GUIDE_R7_REBUILD_R8.tex`; `DDTA_BASE_ANALYSIS_GUIDE_REBUILD_R7.tex`; `DDTA_DERMATRIAGE_PARALLEL_CASE_STUDY_R25_FR03_SPLIT_REVIEW_R1.tex`.
Piano corrente: `DDTA_R25_BASE_ANALYSIS_GUIDE_REBUILD_WORK_PLAN_R7.md`; working analysis: `DDTA_R25_DERMATRIAGE_CASE_STUDY_WORKING_ANALYSIS_R1.md`.
Riferimenti metodologici: Operational Guide R3 + BA0–BA5 come authority corrente; Core/Complete BA Candidate R1 e precedenti guide operative come research evidence per la ricostruzione didattica.

Fase corrente: C0.2 preserva `selection` e `hasPart`; il regression MR-C6 introduce ora il candidate `Boundary`/`boundaryOf` e la qualificazione `crossesBoundary`. Il boundary resta topologico e neutro rispetto a trust/risk; `Store` e binding opzionali restano nel working model.

Stato	`CANDIDATE / NON-NORMATIVE / CUMULATIVE REBUILD`
Repository baseline	`f3c481a8446917e0c083a2f416e8cd907492241e`
Predecessore cumulativo	`DDTA_BASE_ANALYSIS_GUIDE_REBUILD_R6`
Authority BA corrente	R3 Operational Guide + BA0 R1 / BA1 R1 / BA2 R3 / BA3 R1 / BA4 R1 / BA5 R1
Scopo di R7	Stessa revisione cumulativa; questo incremento aggiunge `Boundary`/`boundaryOf` e `crossesBoundary`, preservando `hasPart` e gli altri costrutti gia' ricostruiti.
Authority change	Nessuna promotion normativa. `Boundary`/`boundaryOf` e `crossesBoundary` entrano nella rebuild come candidate; `selection` resta local structure non normativa; `classify` e `storedIn` restano in review/defer.

1 Che cos'e' la Base Analysis

La *Base Analysis* e' una rappresentazione analitica condivisa del significato di progetto. Il suo compito e' rendere esplicite, in una forma comune, le informazioni che devono poter essere riutilizzate, interrogate, confrontate o proiettate verso analisi e viste successive.

Il punto di partenza resta sempre la documentazione. E' la documentazione a stabilire che cosa il progetto governa; la Base Analysis organizza quel significato senza sostituirsi alla sua authority. In questo modo consumer diversi possono lavorare sulla stessa base semantica senza dover reinterpretare ogni volta la prosa sorgente.

Idea fondamentale

La BA rende analiticamente esplicito un significato gia' governato. Non crea project truth e non diventa una fonte alternativa alla documentazione.

1.1 Che cosa resta fuori dalla BA

Una Base Analysis completa non coincide con una copia strutturata di tutto cio' che compare nei documenti. Diagrammi architetturali, DFD, modelli STRIDE e viste di progetto possono essere prodotti a valle, ma non definiscono da soli che cosa debba entrare nella BA. Allo stesso modo, la semplice presenza di un sostantivo, di un verbo, di una tecnologia o di un valore nel testo non basta a giustificare un nuovo elemento analitico.

Il criterio e' semantico: rappresentiamo cio' che deve restare distinguibile e riusabile per gli scopi dichiarati, mantenendo il livello di dettaglio effettivamente governato dal progetto.

1.2 Perche' esiste

Uno stesso significato di progetto puo' servire a consumer molto diversi. Una vista del sistema puo' aver bisogno di mostrare responsabilita' e interazioni; la traceability deve poter risalire alle fonti; il change impact deve riconoscere che cosa viene toccato da una modifica; test reasoning e threat analysis possono porre domande ancora differenti.

Se ciascun consumer ripartisse direttamente dalla prosa, aumenterebbe il rischio di interpretazioni incompatibili. La BA introduce quindi una base analitica comune: il significato viene ricostruito una volta in modo controllato e poi riutilizzato nelle diverse attivita' downstream.

1.3 Authority, minimalita' e feedback

Tre esigenze devono rimanere in equilibrio. La prima e' preservare l'authority documentale: cio' che compare nella BA deve poter essere ricondotto al significato governato. La seconda e' la minimalita': aggiungere struttura che non preserva alcuna distinzione utile rende la rappresentazione piu' pesante senza renderla piu' corretta. La terza e' il feedback: quando una vista o un'analisi incontra un problema, la difficolta' deve poter essere riportata al livello che puo' davvero risolverla.

Questi tre principi spiegano perche' la BA e' contemporaneamente una rappresentazione condivisa e un confine: abbastanza ricca da essere riusata, ma non tanto libera da trasformarsi in una seconda documentazione di progetto.

1.4 Direzione dell'authority

La governed project documentation rimane la project authority. Una Base Analysis accettata costituisce invece la shared analytical baseline: una rappresentazione comune che puo' essere consumata da viste, projection, threat method, test model e altri strumenti analitici.

La direzione dell'authority non cambia quando un consumer downstream scopre una difficolta'. Quel consumer puo' mostrare che una relazione e' ambigua, che manca un'informazione necessaria o che la BA non riesce a preservare una distinzione. Il problema torna allora alla BA come domanda di review. Se la BA dispone gia' del significato necessario, si corregge la rappresentazione; se il significato non e' governato, la domanda viene riportata alla documentazione.

Feedback governato

Una difficolta' downstream puo' provocare una review della BA e, quando necessario, una domanda documentale. Solo una modifica governata della documentazione puo' cambiare il project meaning; dopo tale modifica la BA viene rivalidata e le projection interessate vengono ricostruite.

Il feedback migliora quindi la qualita' del modello e della documentazione senza invertire la sorgente dell'authority.

1.5 Minimum sufficient BA

Una BA piu' dettagliata non e' automaticamente una BA migliore. Il livello giusto e' quello che rende esplicite le distinzioni che devono davvero sopravvivere all'analisi.

Vale la pena introdurre un significato quando serve a mantenere un'identita' riutilizzabile, a distinguere una relazione o una condizione governata, a sostenere traceability e change/revalidation oppure a rispondere a una domanda downstream materiale usando informazioni gia' presenti nella documentazione. Se nessuna di queste esigenze esiste, il dettaglio puo' restare fuori dalla rappresentazione condivisa.

Una domanda downstream puo' quindi motivare l'esplicitazione di un significato gia' governato; non puo' creare il significato che le serve.

Stopping rule iniziale

Se togliendo un dettaglio non perdiamo una distinzione governata, un'identita' da riusare, una relazione necessaria o la possibilita' di rispondere a una domanda materiale nello scope dichiarato, quel dettaglio non viene aggiunto soltanto per rendere il modello piu' completo graficamente.

`NOT SPECIFIED` e documentation gap

Quando la rappresentazione richiederebbe un fatto che la documentazione non governa, il risultato corretto puo' essere `NOT SPECIFIED` oppure una diagnosis. `NOT SPECIFIED` non segnala automaticamente un difetto: quel fatto potrebbe semplicemente non appartenere allo scope del progetto. Diventa una documentation question solo quando l'assenza impedisce di esprimere un significato che il progetto dovrebbe governare.

2 BA View / Projection Contract

Una stessa Base Analysis deve poter alimentare viste differenti senza trasformare ogni vista in una nuova fonte di significato. Per questo introduciamo un **View Contract**: uno strato esplicito che dichiara quale parte del significato disponibile viene selezionata, come viene proiettata e con quale notazione viene resa visibile.

Quando la sorgente e' la BA, il percorso puo' essere riassunto con un piccolo schema:

Governed Documentation ⟶ Accepted Base Analysis ⟶ View Contract ⟶ Rendered View

Non tutte le viste, pero', partono esclusivamente dalla BA. Una vista della gerarchia documentale puo' leggere direttamente MR, Decision e FunctionalRequirement; una vista di traceability puo' invece combinare documentazione e rappresentazione analitica. Il contract deve rendere esplicita questa scelta.

2.1 Da quale sorgente parte la view

Una `BA_ONLY` view usa soltanto semantica presente nella Base Analysis. Una `DOCUMENTATION_ONLY` view rappresenta la struttura o il contenuto governato della documentazione senza duplicarlo artificialmente nella BA. Una `DOCUMENTATION_PLUS_BA_TRACEABILITY` view mette infine in relazione gli elementi documentali con gli elementi analitici che ne rappresentano il significato.

Questa distinzione evita di inserire nella BA informazioni che servono soltanto a rendere possibile una particolare visualizzazione.

2.2 Mermaid come primo target

Mermaid viene adottato come primo target di rendering testuale da investigare. E' utile perche' mantiene i diagrammi come artifact versionabili nel repository e offre piu' famiglie di rappresentazione senza imporre un editor grafico separato.

Mermaid resta pero' una notazione di output. Se la sua sintassi rende comodo introdurre un nodo, una relazione, un participant o un subgraph, quella comodita' non basta a creare nuova semantica nella BA. Prima viene il significato; poi il View Contract decide come renderlo.

Semantic determinism e no reverse inference

A parita' di Base Analysis accettata e di projection profile, una view canonica MUST esporre lo stesso inventario semantico: stessi referent inclusi, stesse proposition, stessi role/edge meaning, stessi modifier/guard e stessi grouping semanticamente richiesti. Il renderer puo' spostare geometricamente gli oggetti gia' collegati senza violare questo requisito. Una scelta grafica non puo' aggiungere semantica ne' diventare la ragione per modificare la BA.

2.3 Che cosa deve dichiarare una view

Il contract deve identificare la view tramite `viewId` e `viewType`; deve poi indicare la sorgente con `sourceType` e `sourceBaseline` e delimitare lo `scope`. Quando la sorgente comprende la BA, specifica quali `includedReferents` e `includedPropositionsOrLocalStructures` vengono usati. Le trasformazioni sono rese esplicite nelle `projectionRules`; `coverageMode` e `omittedSemantics` spiegano quanto della sorgente viene mostrato, mentre `renderer` e `renderedArtifact` identificano la forma di output.

I campi che non hanno senso per uno specifico `sourceType` vengono marcati come non applicabili, invece di essere riempiti artificialmente.

Una struttura locale presente nella BA, inclusa una futura `operatorStructure`, non deve essere trasformata in uno stato *collapsed* del modello per semplificare il diagramma. Il projection profile puo' ometterne completamente il rendering quando non e' pertinente alla view; `coverageMode`/`omittedSemantics` rendono esplicita l'omissione senza modificare la BA sottostante.

2.4 Coverage mode

BA4 distingue due modi di dichiarare la copertura. Una view EXHAUSTIVE_FOR_DECLARED_SCOPE afferma di rappresentare tutto il significato che appartiene allo scope definito ed e' compatibile con quel tipo di vista. Una view SELECTIVE mostra deliberatamente soltanto una parte del significato disponibile.

La seconda forma e' normale e spesso desiderabile: una vista di responsabilita', per esempio, non deve necessariamente mostrare tutte le informazioni, gli stati o i vincoli presenti nella stessa BA. Cio' che viene omesso da una view selettiva non diventa per questo falso, assente o non governato.

2.5 Famiglie di viste da testare

Le prime prove dovranno coprire piu' prospettive. Alcune riguardano la struttura del progetto, come capability, responsabilita' e dipendenze. Altre mostrano aspetti dinamici o relazionali, per esempio information flow, service interaction, state/lifecycle e, quando la semantica disponibile lo consente, pipeline o processi.

Un secondo gruppo riguarda invece la documentazione: gerarchia documentale, traceability fra documentazione e BA, copertura e gap. Non fissiamo ancora il mapping di queste viste; la loro utilita' verra' verificata mentre costruiamo la BA di DermaTriage.

Il fatto che una famiglia di viste sia utile non dimostra che la BA corrente contenga gia' tutto cio' che serve a produrla.

2.6 Quando una view non e' producibile

Una view puo' mettere in evidenza un'assenza senza avere il diritto di colmarla. Se il significato richiesto non e' presente nella BA, il View Contract registra `VIEW REQUIRES MISSING BA SEMANTICS`. Se, a sua volta, la BA non puo' ricavare quel significato dalla documentazione governata, il problema diventa `BA REQUIRES MISSING GOVERNED MEANING`.

Questa distinzione permette di riportare la difficolta' al livello corretto invece di risolverla con una convenzione grafica o una supposizione del renderer.

2.7 Come una relazione entra nella view

Ogni elemento visuale che porta significato deve poter essere giustificato da una regola di projection dichiarata. Un edge Mermaid, per esempio, non e' valido perche' esiste una freccia con un nome simile nella notazione: deve essere il risultato di una trasformazione esplicita di semantica documentale o BA.

La notazione decide come mostrare il risultato della projection; non decide che cosa quel risultato significa. Per questo i mapping fra DDTA, BA e renderer verranno definiti e testati separatamente, invece di essere dedotti per somiglianza terminologica.

3 Dal testo documentale alla Base Analysis

La Base Analysis non nasce traducendo meccanicamente le frasi della documentazione in nodi e relazioni. Prima di scegliere un elemento BA dobbiamo capire quale significato il testo stabilisce realmente e quali parti di quel significato devono rimanere distinguibili nell'analisi.

La forma linguistica e la struttura analitica non coincidono necessariamente. Una frase puo' contenere piu' fatti rilevanti; piu' frasi possono contribuire allo stesso significato; una parola tecnica puo' descrivere una realizzazione senza richiedere una nuova identita' analitica.

Prima il significato, poi il costrutto

La domanda iniziale non e' quale operatore somigli al verbo che stiamo leggendo. E' quale significato di progetto resti vero indipendentemente da come la frase e' stata scritta.

Da qui si procede per gradi. Si parte dalla documentazione governata, si ricostruisce il significato che essa effettivamente stabilisce, si isolano i fatti semantici minimi che servono e si chiarisce quali identita' e quali asserzioni hanno bisogno di una forma analitica. Solo allora si materializzano BAReferent e BAProposition e, in un passaggio successivo, si sceglie il costrutto formale piu' adatto.

Nessun word-to-model mapping

La presenza di un sostantivo non crea automaticamente un BAReferent; un verbo non seleziona da solo un operatore; il confine di una frase non determina il confine di una BAProposition.

3.1 Governed semantic fact

Chiamiamo *governed semantic fact* il piu' piccolo significato governato che e' utile isolare per decidere come rappresentarlo nella BA. Non e' un nuovo tipo di elemento BA: e' un passaggio di analisi che ci permette di separare la comprensione del testo dalla scelta della sintassi analitica.

Un fatto candidato deve sempre poter essere giustificato tornando alla documentazione. Se per formularlo dobbiamo aggiungere un attore, un valore, un ordine, una causalita', una responsabilita' o una condizione non stabilita dal progetto, non stiamo piu' isolando il significato: lo stiamo completando.

DermaTriage - dominio della priorita'

La documentazione stabilisce che, quando DermaTriage produce una priorita' operativa di triage, il valore appartiene all'insieme `P1`, `P2`, `P3`, `P4`. Possiamo quindi isolare il fatto che **OperationalPriority ha come dominio ammesso P1–P4**. In questo momento non serve ancora decidere se `OperationalPriority` debba essere un BAReferent o quale operatore rappresentera' il vincolo.

DermaTriage - percorso senza immagine

Nel percorso symptom-only emergono almeno due significati distinguibili: l'assenza dell'immagine non interrompe il triage e l'urgenza viene determinata usando le informazioni sintomatologiche disponibili sullo stesso caso. Tenerli separati evita di trasformare immediatamente l'intero paragrafo in una sola asserzione sovraccarica.

3.2 Come isolare i fatti senza inventarli

La segmentazione serve a preservare differenze che potrebbero avere identita', relazioni, condizioni o provenance diverse; non serve a spezzare il testo fino a ottenere una proposition per ogni frase o per ogni verbo.

Il lavoro parte dal commitment governato e separa soltanto le distinzioni che potrebbero cambiare o essere riutilizzate indipendentemente. Le parti che perderebbero significato se isolate restano unite. Se il testo ammette piu' letture materialmente diverse, l'ambiguita' rimane visibile: non viene risolta usando conoscenza di dominio, convenzioni tecniche o dettagli dell'implementazione.

Delete test sul fatto candidato

Se elimino questa distinzione prima di costruire la BA, perdo qualcosa che la documentazione governa e che potrebbe diventare rilevante per reuse, traceability, change o una domanda downstream? Se la risposta e' no, probabilmente non serve isolarla come fatto autonomo.

3.3 Il livello documentale resta visibile

MR, Decision e FunctionalRequirement possono governare la stessa area con livelli di precisione diversi. La BA non appiattisce questa progressione. Un MR puo' stabilire un risultato e i suoi input in termini generali; una Decision puo' restringere una strategia; un FR puo' precisare un dominio, una condizione o una regola operativa.

Quando la documentazione introduce un dettaglio piu' specifico, la BA puo' aggiungere la relativa asserzione senza cancellare automaticamente il significato piu' generale che rimane valido.

4 Gli elementi fondamentali della Base Analysis

La Base Analysis distingue due famiglie fondamentali di elementi: `BAReferent` e `BAProposition`. La prima serve a dare identita' analitica a un significato di progetto; la seconda a dare identita' a un'asserzione analitica su uno o piu' significati.

BAE puo' essere usato come termine ombrello per riferirsi agli elementi della Base Analysis, ma non introduce una terza famiglia.

Due domande diverse

Quando incontriamo un significato chiediamo se deve poter essere riconosciuto e riutilizzato come identita' analitica. Quando incontriamo un'asserzione chiediamo invece se deve poter essere tracciata, revisionata o diagnosticata indipendentemente. La prima domanda porta a un BAReferent; la seconda a una BAProposition.

Questa distinzione e' semantica, non implementativa. Un tool puo' memorizzare referent e proposition in classi, tabelle o grafi differenti; cio' che deve restare stabile e' la differenza tra la cosa cui attribuiamo identita' e l'asserzione che facciamo su di essa.

4.1 BAReferent: dare identita' a un significato

Un `BAReferent` e' un'unita' di significato di progetto che deve poter essere riconosciuta e riutilizzata indipendentemente nell'analisi. Non e' limitato agli oggetti software o ai componenti runtime: puo' rappresentare una capability, un'informazione, un risultato, un comportamento, un evento, uno stato, un contratto, uno store, un boundary o un altro significato che abbia bisogno di identita' stabile.

Identity test

La domanda utile non e' "questo nome compare nel testo?", ma "questo significato deve vivere oltre l'asserzione in cui l'ho incontrato?". L'identita' separata diventa giustificata quando il significato deve essere riutilizzato in piu' proposition, qualificato o vincolato, correlato o referenziato, confrontato fra baseline, seguito attraverso change/revalidation oppure interrogato come oggetto distinto da un consumer downstream.

Se nessuna di queste esigenze esiste e la rimozione dell'identita' non fa perdere significato governato, un BAReferent separato puo' essere superfluo.

Working model di `semanticKind`

Per mantenere il modello operativo e interrogabile senza introdurre proposition di typing prive di pressione concreta, questa R7 usa `semanticKind` direttamente sul `BAReferent` come chiave controllata methodology-neutral: `semanticKind [0..*]`. Il kind non crea una nuova famiglia BA1. Provenance e review restano, per ora, alla granularita' del BAReferent; non viene introdotto un lifecycle separato per il singolo kind. La cardinalita' multipla resta disponibile ma si usa il minimo insieme sufficiente di kind realmente utile ai consumer.

Identita' di configurazione attiva

Quando la source identifica una configurazione attiva come insieme riconoscibile di proprieta', binding, componenti o versioni che qualifica il sistema o il suo environment, applicare il normale identity test. Se l'insieme deve essere riusato, confrontato fra baseline, seguito attraverso change/revalidation o interrogato per impact/security analysis, la configurazione puo' costituire un `BAReferent` autonomo. La promozione dell'insieme non promuove automaticamente le singole entry a BAReferent e non rende normative versioni o valori documentati soltanto come current realization. Un software-composition/BOM profile puo' quindi avere identita' BA anche quando package e versioni restano realization facts.

La presenza lessicale, da sola, non basta: tecnologie, valori, campi e sostantivi diventano referent solo quando serve davvero un'identita' analitica indipendente.

4.2 BAReferent: prima applicazione a DermaTriage

La documentazione consolidata di MR-01 offre gia' alcuni buoni esempi per applicare l'identity test senza congelare prematuramente il registry finale.

DermaTriage. Compare ripetutamente come soggetto di obblighi e relazioni; e' quindi un forte candidato a identita' BA riusabile.

Caso dermatologico. Collega immagine, sintomi e risultati. La necessita' di una sua identita' autonoma dipendera' soprattutto da come dovremo preservare correlation e same-case semantics nelle proposition successive.

Urgenza e priorita' operativa. Sono risultati governati distinti. L'urgenza viene prodotta e riutilizzata nei passaggi successivi; la priorita' e' ulteriormente vincolata alla P-scale. Entrambe sono candidate forti a identita' riusabile, mentre resta da decidere il livello corretto fra concetto e occurrence per singolo caso.

Il valore P1. Il fatto che sia governato non lo trasforma automaticamente in un BAReferent. Se non deve essere riutilizzato come identita' indipendente, puo' rimanere un valore locale controllato.

L'esempio non decide ancora il modello finale

Questa prima applicazione mostra come usare l'identity test. Non congela il registry DermaTriage e non decide ancora la granularita' instance-vs-concept: saranno le proposition necessarie a mostrare quali identita' devono sopravvivere autonomamente.

Differenze come actor, component, information, behavior, state, contract o store non richiedono di per se' famiglie BA diverse. Quando la distinzione e' utile ai consumer, viene preservata tramite il `semanticKind` controllato dello stesso BAReferent; ruoli, valori e proposition descrivono invece come quel referent partecipa ai fatti governati.

4.3 BAProposition: dare identita' a una asserzione

Una `BAProposition` e' un'asserzione analitica methodology-neutral su uno o piu' significati identificati. Normalmente i suoi term sono BAReferent; una firma di operatore puo' pero' ammettere esplicitamente un riferimento a un'altra BAProposition quando il nuovo fatto qualifica l'asserzione gia' identificata senza riscriverla. La sua identita' permette di collegare provenance, review, diagnosis e change disposition all'asserzione stessa, senza confonderla con i significati o con le proposition che mette in relazione.

Questa distinzione diventa importante quando un referent rimane stabile ma cambia cio' che il progetto afferma su di esso. In quel caso non dobbiamo sostituire l'identita' del referent: possiamo introdurre, modificare, supersedere o diagnosticare le proposition interessate.

A livello BA2 la forma generale corrente prevede un `semanticOperatorKey`, una o piu' participation con roleKey scoped all'operatore e, quando ammessi, polarity, modifier e strutture locali. Le firme specifiche verranno spiegate piu' avanti costrutto per costrutto.

```
BAProposition
    semanticOperatorKey
    participation [1..*]
        roleKey
        term
    polarity
    scopedModifier [0..*]
    operatorStructure [0..1, when admitted]
```

Il `term` di una participation e' normalmente un BAReferent. Un valore locale controllato e' sufficiente soltanto quando quel significato non richiede identita' indipendente. Un riferimento a BAProposition e' ammesso soltanto se la firma dell'operatore lo dichiara: deve puntare a una proposition gia' identificata e non ne modifica semanticOperatorKey, role, polarity o provenance. Non si inferiscono automaticamente nesting, inheritance o transitivita' fra proposition.

4.4 BAReferent e BAProposition lavorano insieme

Prendiamo il FunctionalRequirement che governa la priorita' operativa P1–P4. Prima di formalizzare, riconosciamo due cose diverse. Da un lato esistono significati che devono poter essere riutilizzati, come DermaTriage e OperationalPriority; dall'altro la documentazione afferma qualcosa su questi significati, per esempio che una priorita' operativa puo' essere prodotta e che il suo dominio e' limitato a P1–P4.

L'identity test suggerisce quindi almeno i referent candidati `DermaTriage` e `OperationalPriority`. Le affermazioni sul risultato e sul dominio saranno invece materializzate come BAProposition quando avremo selezionato operatori, ruoli e valori compatibili con il significato governato.

Perche' non formalizziamo troppo presto

Possiamo gia' vedere che disponibilita' della priorita' e dominio ammesso sono due asserzioni distinguibili. Non serve pero' scegliere adesso `produce`, `constrain` o un altro costrutto: prima fissiamo la grammatica generale, poi riesaminiamo ogni operatore con lo stesso contratto descrittivo.

Il punto da conservare e' semplice: il referent rappresenta il significato che deve continuare a esistere come identita' analitica; la proposition rappresenta cio' che affermiamo su quel significato. Un grafo o un renderer non devono confondere i due livelli soltanto per ottenere una forma visuale piu' comoda.

5 Granularita' e costruzione della Base Analysis

La granularita' corretta non dipende dalla lunghezza della frase ne' dal numero di elementi che un tool riesce a disegnare. Dipende da quali distinzioni devono poter essere identificate, verificate e cambiate indipendentemente.

5.1 Quando separare due proposition

Due significati meritano proposition separate quando possono divergere per provenance o stato di review, per condizione di applicabilita', per impatto di un cambiamento, perche' uno dei due diventa target di una relazione successiva oppure perche' consumer downstream diversi hanno bisogno di distinguerli. Se invece le parti hanno senso soltanto insieme, la rappresentazione deve conservarne l'unita'.

Separare significa quindi preservare indipendenza semantica, non frammentare il testo.

5.2 Procedura iniziale di costruzione

Il lavoro puo' essere condotto in sette passaggi:

1. fissare baseline, authority e scope documentale;
2. leggere il testo source-first e ricostruire il significato governato prima di pensare alla sintassi BA;
3. isolare soltanto i governed semantic fact che devono rimanere distinguibili;
4. applicare l'identity test e riconoscere le asserzioni che richiedono BAReferent e BAProposition;
5. verificare che la cardinalita' BA derivi dal significato analitico e non dalla forma dell'albero documentale; se emerge una possibile inefficienza strutturale, registrare un finding e rinviare la decisione al layer documentale;
6. selezionare operatori, ruoli, valori e strutture locali solo dopo che identita' e asserzioni sono chiare;
7. tornare alle fonti per verificare ogni elemento, preservare `NOT SPECIFIED` o diagnosis quando necessario e applicare infine stopping rule e forbidden-inference review.

Stop prima degli operatori

Prima di entrare nel catalogo dei costrutti, il lettore deve saper spiegare perche' un significato richiede un BAReferent, perche' un'asserzione richiede una BAProposition e quali parti della documentazione giustificano entrambi. Se questo non e' chiaro, scegliere un operatore e' prematuro.

6 Struttura documentale, cardinalita' BA e feedback verso monte

La documentazione DDTA organizza responsabilita', decisioni e obblighi in una struttura utile per authoring, implementazione, verifica e change management. La Base Analysis ha uno scopo diverso: rendere espliciti i significati analitici che devono essere riusati e interrogati. Per questo le due strutture non hanno una cardinalita' obbligatoriamente coincidente.

Regola di indipendenza strutturale

Gli elementi documentali sono sorgenti di significato governato, non un template da tradurre uno-a-uno in elementi BA.

```
numero di branch documentali
        !=
numero di BAReferent
        !=
numero di BAProposition
```

Piu' elementi documentali possono sostenere uno stesso significato analitico; un singolo elemento documentale puo' invece contenere piu' asserzioni che devono restare distinguibili in BA. La cardinalita' corretta emerge dai test di identita', asserzione, provenance, change e consumo downstream, non dal numero di nodi presenti nell'albero DDTA.

Traduzione meccanica vietata

Non creare un BAReferent, una BAProposition o un futuro ramo analitico soltanto perche' esiste un elemento documentale distinto. Allo stesso modo, non comprimere due significati BA distinti soltanto perche' sono scritti nello stesso requisito.

6.1 Quando la BA espone una pressione strutturale documentale

Durante la normalizzazione puo' emergere che un branch documentale contiene un significato reale e source-supported ma non possiede conseguenze downstream sufficientemente autonome da giustificare la propria esistenza come branch separato. Questo e' un risultato utile della BA, ma non autorizza la BA a riscrivere la documentazione.

Finding di review strutturale

Usare `POSSIBLE_UPSTREAM_STRUCTURAL_REDUNDANCY` come segnale di review quando il significato di un branch puo' essere preservato senza perdita dentro il branch che possiede il comportamento correlato e la separazione autonoma non mostra una utilita' downstream distinguibile.

Il finding deve registrare almeno:

- gli elementi documentali coinvolti;
- il significato governato che ciascuno porta;
- le distinzioni analitiche che devono comunque essere preservate;
- le conseguenze downstream autonome cercate;
- perche' la separazione appare necessaria oppure potenzialmente superflua;
- la review documentale da riaprire.

No authority inversion

La BA puo' diagnosticare la pressione e mantenere una cardinalita' analitica non artificiale. Non puo' dichiarare da sola `MERGE`, cancellare un elemento documentale o trasferirne l'ownership. La decisione strutturale torna alla Documentation Authoring review; se la documentazione cambia, la BA viene poi revalidata o ricostruita.

6.2 Pressure case DermaTriage: dominio P1–P4 e derivazione della priorita'

La ricostruzione DermaTriage ha mostrato il caso in forma concreta. In una revisione precedente il significato relativo alla priorita' operativa era separato in due branch:

```
DEC-MR01-02
    adozione del dominio P1-P4
    -> FR-MR01-02-01
       la priorita' appartiene a {P1,P2,P3,P4}

DEC-MR01-04
    separazione urgenza analitica / priorita' operativa
    -> FR-MR01-04-01
       regola che deriva P1-P4 da urgenza e confidence
```

La review ha chiarito che il significato "la priorita' operativa usa il dominio P1–P4" e' reale e deve essere preservato. Non era quindi corretto descrivere `DEC-MR01-02` come priva di semantica. Il problema era diverso: come branch autonomo non produceva un comportamento, una verifica, una modifica indipendente o una analisi distinta rispetto al branch che governa la derivazione della priorita'.

Esito documentale del pressure case

Il dominio P1–P4 viene mantenuto dentro `DEC-MR01-04`; `FR-MR01-04-01` continua a governare il mapping operativo. Il branch `DEC-MR01-02 / FR-MR01-02-01` puo' quindi essere consolidato senza perdere il significato del dominio.

Questo caso mostra due regole contemporaneamente:

1. una distinzione semantica puo' restare necessaria nella BA anche dopo la riduzione del numero di branch documentali;
2. il numero di distinzioni BA non dimostra quanti branch debbano esistere nella documentazione.

6.3 Ritorno alla Documentation Authoring review

La decisione sulla struttura documentale usa il **Downstream Utility Test** definito nella Documentation Authoring Guide:

Downstream Utility Test - eseguito upstream

Se elimino questo branch come branch autonomo e trasferisco il suo significato nella Decision che possiede i comportamenti correlati, perdo una distinzione necessaria per implementazione, verifica, modifica indipendente o analisi?

Se la risposta e' positiva, la separazione puo' essere giustificata. Se e' negativa, il layer documentale riesamina il branch per MERGE o REWORK. In entrambi i casi la BA non anticipa l'esito.

```
governed / working documentation
        -> source-first BA normalization
        -> structural-feedback finding
        -> Documentation Authoring review
        -> Downstream Utility Test
        -> KEEP / MERGE / REWORK documentale
        -> BA revalidation if documentation changed
```

Distinzione da preservare

"Due proposition analitiche" e "due branch documentali" sono domande diverse. La BA separa i significati quando serve all'analisi; la documentazione separa i branch quando serve al lavoro sul progetto. Le due cardinalita' possono coincidere, ma non sono obbligate a farlo.

Confine di questa revisione

La R7 mantiene il catalogo in forma cumulativa: `transfer`, `produce`, `realize`, `constrain` e `condition` sono gia' ricostruiti. I pressure case cumulativi non autorizzano a promuovere automaticamente un quinto operator: la rebuild testa invece se lo storico `decisionRule` possa diventare `operatorStructure.decisionRule` della proposition `produce`. Il caso di derivazione P-scale e il caso indipendente di qualificazione della readiness C7 forniscono ora evidence positiva sullo stesso confine semantico; fino a regression cross-corpus e checkpoint, `decisionRule` resta authority corrente e la rilocazione resta candidate non ammessa.

7 Operatore `transfer`

CURRENT / PRESERVED

7.1 Che cosa rappresenta

`transfer` rappresenta la **conveyance governata** di uno o piu' contenuti da una source identificabile verso una o piu' destination identificabili. Il costrutto non descrive genericamente un collegamento, un ordine di esecuzione o una vicinanza architetturale: il suo smallest semantic fact e' che *un content viene trasferito da A a B*.

Smallest semantic fact

Esiste un contenuto governato X la cui conveyance e' asserita da una source A verso almeno una destination B.

7.2 Quando usarlo

Usare `transfer` quando la documentazione governa sufficientemente tutti e tre i ruoli obbligatori: origine, destinazione e contenuto trasferito. Il verbo concreto della documentazione puo' essere "consegnare", "inviare", "rendere disponibile a", "passare", "trasmettere" o un'espressione equivalente; la scelta non dipende dalla parola usata ma dal significato governato.

7.3 Quando non usarlo

Non accettare una proposition `transfer` se almeno uno fra source, destination e content deve essere inventato per completare la frase. La presenza di due componenti nello stesso diagramma, di una chiamata API, di una sequenza di stage o di un dato disponibile in memoria non basta da sola a dimostrare conveyance.

Gate minimo

Se la source non permette di identificare `source`, almeno una `destination` e almeno un `content`, il risultato corretto e' `INSUFFICIENT EVIDENCE` / `NOT SPECIFIED`, non un transfer completato per plausibilita'.

7.4 Firma canonica

La firma corrente preservata da BA2 R3 e':

```
transfer
  behavior    -> BAReferent [0..1]
  source      -> BAReferent [1]
  destination -> BAReferent [1..*]
  content     -> BAReferent [1..*]
```

`polarity` appartiene alla `BAProposition` e resta obbligatoria secondo il contratto generale. Gli scoped modifier `condition` e `temporalScope` compaiono soltanto quando la source limita realmente l'asserzione.

Role	Card.	Significato
`behavior`	0..1	Identita' riusabile dello specifico comportamento di transfer, solo quando il segmento deve essere target di altre proposition, vincoli, realization, confronto o analisi autonoma.
`source`	1	Origine governata della conveyance.
`destination`	1..*	Uno o piu' destinatari governati del content.
`content`	1..*	Significato informativo/materiale la cui conveyance e' governata.

7.5 Identita' opzionale del segmento

`behavior` non viene creato per ogni scambio. Riceve identita' `BAReferent` quando il trasferimento stesso deve poter essere riusato, vincolato, realizzato, protetto, confrontato fra baseline o preso come target da piu' asserzioni.

Behavior identity test

Se elimino l'identita' dello specifico segmento di transfer, perdo la possibilita' di esprimere un significato governato che riguarda proprio il passaggio e non soltanto source, destination o content? Se no, `behavior` resta assente.

7.6 Continuita' di identita' del content

Quando un significato viene prodotto, trasferito e poi usato da un consumer senza cambiare identita' semantica, la BA riusa lo stesso `BAReferent`. La posizione "output", "content del transfer" e "input" non crea tre identita' diverse.

```
producer result   -> X
transfer content  -> X
consumer input    -> X
```

Identity continuity rule

Un nuovo BAReferent e' giustificato solo quando tra i due punti compare una trasformazione che stabilisce un significato semanticamente distinto. Cambio di variabile, serializzazione, formato fisico o collocazione non bastano da soli.

7.7 Invarianti

- ogni transfer accettato possiede esattamente una source e almeno una destination e un content;
- i roleKey sono operator-scoped: `source` significa origine della conveyance, non una classificazione universale del referent;
- destination multiple e content multipli sono ammessi solo quando la stessa asserzione governata li comprende;
- l'eventuale `behavior` identifica il segmento, non sostituisce source, destination o content;
- polarity negativa nega l'asserzione di transfer soltanto quando questa negazione e' realmente governata; assenza di evidence non equivale a transfer negativo.

7.8 Forbidden inference

Dal solo `transfer` non inferire protocollo, sincronicita', cifratura, affidabilita', successo della consegna, persistenza, ownership, storage, service consumption, transport medium, retries, acknowledgement, failure semantics o topologia intermedia. Questi significati richiedono evidence e costrutti appropriati separati.

7.9 Domande da porre alla documentazione

Le domande servono a restringere l'interpretazione; non sono un classificatore automatico.

1. La source governa realmente che un contenuto passi da un'origine a un destinatario?
2. Qual e' la `source`? E' esplicitamente identificabile senza inferenza architetturale?
3. Qual e' o quali sono le `destination`?
4. Qual e' o quali sono i `content` trasferiti?
5. Source, destination e content hanno gia' un `BAReferent` con la stessa identita' semantica? Se si', riusarlo.
6. Il content mantiene la stessa identita' fra producer, transfer e consumer oppure la source governa una trasformazione semanticamente distinta?
7. Il segmento di transfer deve essere target autonomo di property, constraint, realization, security scope, change impact o altre proposition? Se si', valutare `behavior`.
8. La proposition vale sempre oppure solo sotto una condition o un temporal scope governato?
9. La source afferma il transfer o lo nega esplicitamente? Non dedurre polarity negativa dall'assenza di trasferimento.
10. Rimane qualche ruolo obbligatorio non stabilito? Se si', fermarsi: il dato resta `NOT SPECIFIED` oppure diagnosis.

Procedura minima

Leggere il smallest governed semantic fact; identificare o riusare i BAReferent necessari; verificare source/destination/content; decidere se serve behavior; compilare una sola BAProposition minima; aggiungere scoped modifier solo se governati; infine controllare le forbidden inference.

7.10 Esempio generico minimo

Documentazione inventata ma coerente

"Il sottosistema di acquisizione MUST consegnare il `MeasurementRecord` al motore di valutazione."

Assumendo che le tre identita' abbiano superato il normale gate BA1, la proposition minima e':

```
transfer
  source      -> MeasurementAcquisition
  destination -> EvaluationEngine
  content     -> MeasurementRecord
  polarity    -> affirmative
```

Non serve un `behavior` referent se il segmento non deve essere riusato o target di altre proposition.

7.11 Esempio corrente - DermaTriage T06

Nella documentazione corrente, Stage 2 produce una `ClinicalDescription` e la rende disponibile allo Stage 3; Stage 3 usa quella stessa descrizione come input del retrieval. L'estrazione corrente e':

```
transfer
  behavior    -> ClinicalDescriptionTransfer
  source      -> ClinicalDescriptionGeneration
  destination -> HistoricalCaseRetrieval
  content     -> ClinicalDescription
  polarity    -> affirmative
```

`ClinicalDescription` e' lo stesso BAReferent gia' usato come result della generazione e come input del retrieval. `ClinicalDescriptionTransfer` riceve identita' perche' il segmento e' oggetto autonomo della review data-path / information-contract; questa scelta appartiene all'esempio, non alla definizione universale di `transfer`.

Boundary example

"Stage B usa X" non basta, da solo, a costruire `transfer` se la documentazione non stabilisce da quale source X arrivi. L'input puo' essere noto mentre il transfer resta `INSUFFICIENT EVIDENCE`.

7.12 Prompt LLM di supporto all'authoring

Il prompt e' un ausilio operativo: non cambia la semantica BA e non rende autorevole una inferenza non supportata. Deve ricevere l'estratto documentale governato e, quando disponibile, il registro dei BAReferent gia' esistenti.

```
TASK: identify governed BA transfer propositions.
Use ONLY the supplied documentation and existing BA registry.

A transfer exists only if the source supports:
  source [1], destination [1..*], content [1..*].

Rules:
1. Identify the smallest governed conveyance fact.
2. Reuse existing BAReferents when semantic identity is unchanged.
3. Propose a new BAReferent only when BA1 independent identity is needed.
4. Add behavior only if the transfer segment itself must be independently
   targeted, constrained, realized, compared, traced or analyzed.
5. Preserve the same content BAReferent across producer/transfer/consumer
   when the governed meaning is unchanged.
6. Use polarity=negative only for an explicitly negated transfer assertion.
7. Add condition/temporalScope only when explicitly governed.
8. If source, destination or content is missing, return INSUFFICIENT_EVIDENCE.
9. Do not infer protocol, network topology, persistence, success, ownership,
   encryption, service consumption, transport medium or failure semantics.
10. Do not invent operators, roles, states or metadata.

Return EXACTLY the authoring wrapper and BA output shape shown next.
```

Human review remains mandatory

L'LLM puo' proporre referent e proposition; l'accettazione richiede comunque verifica contro source, identity criterion, canonical naming e provenance. Una risposta sintatticamente completa non sana un gap documentale.

7.13 Output esatto atteso dall'estrazione assistita

Il wrapper seguente e' un formato di authoring/review, non un nuovo costrutto BA. La parte di output BA usa soltanto la shape ammessa.

```
TRANSFER_ASSESSMENT
  decision = ACCEPT_TRANSFER | NOT_A_TRANSFER | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  notSpecified = [<only unresolved material items>]

BA_OUTPUT
  BAReferent reuse/candidates:
    <canonical names only as needed>

  BAProposition <candidate-id>
    semanticOperatorKey = transfer
    behavior    -> <BAReferent>      # omit line if not justified
    source      -> <BAReferent>
    destination -> <BAReferent>      # repeat for 1..*
    content     -> <BAReferent>      # repeat for 1..*
    polarity    = affirmative | negative
    scopedModifier = <condition|temporalScope>  # omit if absent
```

Se `decision != ACCEPT_TRANSFER`, `BA_OUTPUT` non contiene una proposition transfer accettabile. I nomi proposti restano candidate finche' non superano canonical naming, provenance e review.

7.14 Discriminazioni con altri costrutti

-

7.15 Rappresentazione Mermaid di transfer

Per `transfer`, la projection rende direttamente leggibili i role dell'operatore. `source` e `destination` sono gli endpoint; `content` occupa la posizione centrale ed e' mostrato con una forma documento. L'intero percorso direzionale `source -> content -> destination` rappresenta **una sola** `BAProposition transfer`: le due frecce sono una decomposizione grafica del percorso, non due proposition.

Quando il role opzionale `behavior` e' presente, viene visualizzato come annotazione distinta del transfer: nodo giallo collegato al content con una linea tratteggiata e non direzionale. Questa linea e' una convenzione di projection del role `behavior`; non rappresenta una seconda relazione BA.

Parte di transfer	Rappresentazione Mermaid
`source`	Nodo endpoint all'origine del percorso.
`content`	Nodo centrale con forma documento e sfondo bianco.
`destination`	Nodo endpoint raggiunto dal percorso direzionale.
`behavior`	Nodo opzionale con sfondo giallo; linea tratteggiata non direzionale verso il content.
Proposition id / polarity	Metadati della stessa `BAProposition`; non diventano un nodo del data path.

```
flowchart LR
  S["ClinicalDescriptionGeneration<br/>source"]
  C@{ shape: doc, label: "ClinicalDescription<br/>content" }
  D["HistoricalCaseRetrieval<br/>destination"]
  B["ClinicalDescriptionTransfer<br/>behavior"]

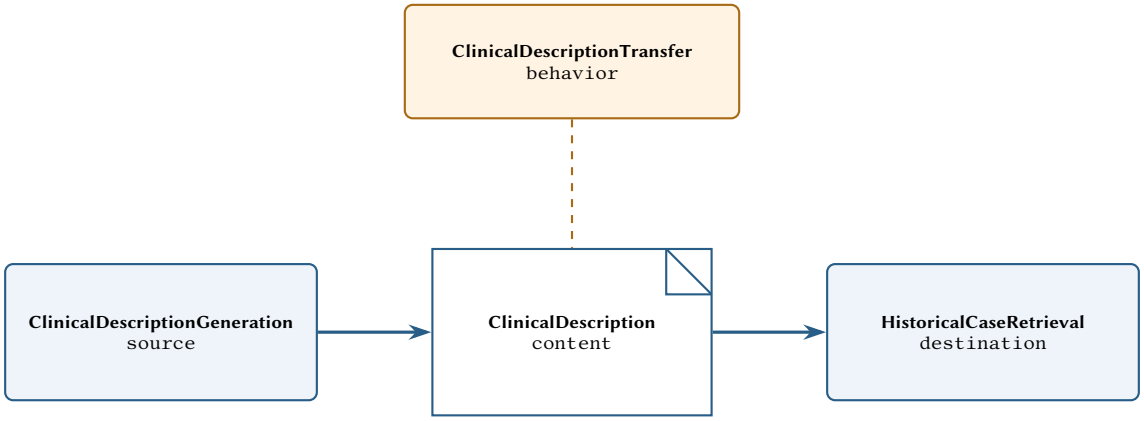
  S --> C --> D
  B --- C

  classDef endpoint fill:#EEF4F9,stroke:#275D86,color:#17324D
  classDef content fill:#FFFFFF,stroke:#275D86,color:#17324D
  classDef behavior fill:#FFF4E3,stroke:#A96A16,color:#17324D
  class S,D endpoint
  class C content
  class B behavior
  linkStyle 2 stroke:#A96A16,stroke-width:1.5px,stroke-dasharray:5 5
```

Il codice usa il caso T06 come esempio concreto. Se `behavior` fosse assente, il nodo giallo e la linea tratteggiata verrebbero omessi; il percorso `source -> content -> destination` resterebbe invariato.

7.16 Esempio grafico corrispondente

Il diagramma seguente applica la stessa convenzione al caso T06: il documento bianco e' il `content`; il percorso con frecce rende la direzione del `transfer`; il nodo giallo identifica il `behavior` opzionale senza trasformarlo in uno stadio del flusso.



BAP-T06-01 `transfer` `polarity = affirmative`

Lettura del grafico

Il percorso completo da `source` a `destination`, passando per il documento `content`, e' la projection di una sola proposition `transfer`. La linea tratteggiata verso il nodo giallo esprime soltanto la presenza del role opzionale `behavior`; non e' una BAProposition aggiuntiva e non introduce una seconda direzione di flusso.

8 Operatore `produce`

CURRENT / PRESERVED

8.1 Che cosa rappresenta

`produce` rappresenta l'asserzione governata che un actor o una capability **rende disponibile uno o piu' risultati/output**, eventualmente usando input esplicitamente governati. Il costrutto descrive il fatto produttivo minimo; non descrive automaticamente come il risultato viene calcolato, dove viene memorizzato o a chi viene successivamente trasferito.

Smallest semantic fact

Esiste un actor/capability `A` che rende disponibile almeno un risultato governato `R`. Eventuali input `I` appartengono alla stessa proposition solo quando la documentazione governa esplicitamente che vengano usati per quella produzione.

8.2 Quando usarlo

Usare `produce` quando la documentazione governa che un actor/capability produce, genera, calcola, restituisce, emette o rende disponibile un risultato identificabile. Il verbo concreto non e' decisivo: deve essere governato il significato "A rende disponibile R". Gli input sono opzionali e vengono inclusi soltanto quando la source li collega alla produzione del risultato.

8.3 Quando non usarlo

Non accettare una proposition `produce` quando il risultato deve essere inventato, quando e' nota soltanto l'esistenza di un dato senza un actor che lo renda disponibile, oppure quando un input viene aggiunto solo perche' compare altrove nell'architettura. La presenza di una sequenza di stage, di un componente o di una dipendenza temporale non dimostra da sola una produzione.

Gate minimo

Se la source non permette di identificare esattamente un `actor` e almeno un `result`, il risultato corretto e' `INSUFFICIENT EVIDENCE` / `NOT SPECIFIED`. L'assenza di input espliciti, invece, non invalida `produce`: `input` ha cardinalita' `0..*`.

8.4 Firma canonica

La firma completa e':

```
produce
  actor    -> BAReferent [1]
  input    -> BAReferent [0..*]
  result   -> BAReferent [1..*]
  polarity = affirmative | negative [1]
```

Gli scoped modifier `condition` e `temporalScope` compaiono soltanto quando la source limita realmente l'asserzione.

Role	Card.	Significato
`actor`	1	Actor/capability a cui la documentazione attribuisce il fatto di rendere disponibile il risultato.
`input`	0..*	Significati che la documentazione dichiara usati dalla stessa produzione; non vengono ricostruiti da dipendenze plausibili.
`result`	1..*	Uno o piu' risultati/output governati resi disponibili dall'actor.

8.5 Continuita' di identita' di input e result

Un `result` non riceve una nuova identita' soltanto perche' viene successivamente trasferito o usato come `input` da un altro comportamento. Quando il significato governato resta lo stesso, la BA riusa lo stesso `BAReferent`.

```
producer result   -> X
transfer content  -> X
consumer input    -> X
```

Allo stesso modo, un `input` di `produce` e' il medesimo significato gia' identificato altrove quando non esiste una trasformazione semanticamente distinta.

8.6 Invarianti

- ogni `produce` accettato possiede esattamente un `actor` e almeno un `result`;
- `input` e' opzionale: la proposition resta valida anche quando la source non governa input espliciti;
- input multipli o result multipli appartengono alla stessa proposition solo quando la stessa asserzione governata li lega allo stesso fatto produttivo;
- i roleKey sono operator-scoped: essere `result` in una proposition non assegna al referent un tipo universale "output";
- polarity negativa richiede una negazione governata della produzione; assenza di evidence non equivale a `produce` negativo.

8.7 Forbidden inference

Dal solo `produce` non inferire algoritmo, modello, trasformazioni intermedie, destination, transfer, protocollo, serializzazione, persistence, storage, ownership, creazione ex novo dell'item, successo runtime, retry, failure semantics o completezza dello schema del risultato. Non aggiungere inoltre un `input` soltanto perche' un dato e' disponibile nel sistema o compare in uno stadio precedente.

8.8 Domande da porre alla documentazione

Le domande servono a restringere l'interpretazione; non sono un classificatore automatico.

1. La source governa realmente che qualcosa renda disponibile un risultato/output?
2. Qual e' esattamente l'`actor`? E' attribuito dalla documentazione senza inferenza architetturale?
3. Qual e' o quali sono i `result` prodotti?
4. La source governa uno o piu' `input` usati per quella produzione? Se non li governa, lasciarli assenti.
5. Actor, input e result hanno gia' un `BAReferent` con la stessa identita' semantica? Se si', riusarlo.
6. Piu' input o piu' result appartengono davvero allo stesso fatto produttivo oppure la documentazione governa produzioni distinte?
7. Un result mantiene la stessa identita' quando viene trasferito o riusato downstream? Se si', non creare un nuovo referent per il cambio di posizione.
8. La proposition vale sempre oppure solo sotto una condition o un temporal scope governato?
9. La source afferma la produzione o la nega esplicitamente? Non dedurre polarity negativa dall'assenza del risultato.
10. Rimane non stabilito l'actor o ogni result? Se si', fermarsi: il dato resta `NOT SPECIFIED` oppure diagnosis.

Procedura minima

Leggere il smallest governed semantic fact; identificare o riusare l'actor e almeno un result; aggiungere soltanto gli input esplicitamente legati alla produzione; compilare una sola BAProposition minima; aggiungere scoped modifier solo se governati; infine controllare le forbidden inference e la continuita' di identita' downstream.

8.9 Esempio generico minimo

Documentazione inventata ma coerente

"La capability `RiskAssessmentComputation` MUST usare il `MeasurementRecord` per rendere disponibili un `RiskAssessment` e una `AssessmentConfidence`."

Assumendo che le identita' abbiano superato il normale gate BA1, la proposition e':

```
produce
  actor  -> RiskAssessmentComputation
  input  -> MeasurementRecord
  result -> RiskAssessment
  result -> AssessmentConfidence
  polarity -> affirmative
```

I due result restano nella stessa proposition perche' la frase governa un unico fatto produttivo che li rende disponibili insieme.

8.10 Esempio corrente - DermaTriage Stage 3

Nella documentazione corrente, lo Stage 3 usa la `ClinicalDescription` prodotta dallo Stage 2 per il retrieval e rende disponibile il contesto dei casi storici recuperati. La proposition accettata corrente e':

```
BAP-FR03-03-01
semanticOperatorKey = produce
  actor  -> HistoricalCaseRetrieval
  input  -> ClinicalDescription
  result -> RetrievedHistoricalCaseContext
polarity = affirmative
```

`ClinicalDescription` e' lo stesso `BAReferent` gia' usato come `content` del transfer T06; non viene duplicato come "Stage3Input". Il dettaglio ChromaDB / embedding appartiene ad altre proposition gia' separate e non viene incorporato nella shape di `produce`.

Boundary example

"Il componente B dispone di X" non basta, da solo, a costruire `produce`: puo' descrivere disponibilita' o stato senza governare chi produce X. Se actor o result non sono stabiliti, l'esito resta `INSUFFICIENT EVIDENCE`.

8.11 Prompt LLM di supporto all'authoring

Il prompt e' un ausilio operativo: non cambia la semantica BA e non rende autorevole una inferenza non supportata. Deve ricevere l'estratto documentale governato e, quando disponibile, il registro dei BAReferent gia' esistenti.

```
TASK: identify governed BA produce propositions.
Use ONLY the supplied documentation and existing BA registry.

A produce proposition exists only if the source supports:
  actor [1], result [1..*].
Input is optional: input [0..*].

Rules:
1. Identify the smallest governed production fact: actor makes result available.
2. Reuse existing BAReferents when semantic identity is unchanged.
3. Propose a new BAReferent only when BA1 independent identity is needed.
4. Add input only when the source explicitly governs its use in this production.
5. Keep multiple inputs/results in one proposition only when the same governed
   production fact binds them to the same actor.
6. Preserve result identity when it later appears as transfer content or input.
7. Use polarity=negative only for an explicitly negated production assertion.
8. Add condition/temporalScope only when explicitly governed.
9. If actor or every result is missing, return INSUFFICIENT_EVIDENCE.
10. Do not infer algorithm, implementation, destination, transfer, persistence,
    storage, creation, ownership, serialization or failure semantics.
11. Do not invent operators, roles, states or metadata.

Return EXACTLY the authoring wrapper and BA output shape shown next.
```

Human review remains mandatory

L'LLM puo' proporre referent e proposition; l'accettazione richiede comunque verifica contro source, identity criterion, canonical naming e provenance. Una risposta sintatticamente completa non sana un gap documentale e non rende implicito un input non governato.

8.12 Output esatto atteso dall'estrazione assistita

Il wrapper seguente e' un formato di authoring/review, non un nuovo costrutto BA. La parte di output BA usa soltanto la shape ammessa.

```
PRODUCE_ASSESSMENT
  decision = ACCEPT_PRODUCE | NOT_A_PRODUCE | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  notSpecified = [<only unresolved material items>]

BA_OUTPUT
  BAReferent reuse/candidates:
    <canonical names only as needed>

  BAProposition <candidate-id>
    semanticOperatorKey = produce
    actor  -> <BAReferent>
    input  -> <BAReferent>          # repeat for 0..*; omit if absent
    result -> <BAReferent>          # repeat for 1..*
    polarity = affirmative | negative
    scopedModifier = <condition|temporalScope>  # omit if absent
```

Se `decision != ACCEPT_PRODUCE`, `BA_OUTPUT` non contiene una proposition `produce` accettabile. I nomi proposti restano candidate finche' non superano canonical naming, provenance e review. L'assenza di `input` non e' un errore quando la source governa actor e result ma non esplicita gli input.

8.13 Discriminazioni con altri costrutti

-

8.14 Rappresentazione Mermaid di produce

Per `produce`, la projection mette l'`actor` al centro del fatto produttivo. Gli eventuali `input` entrano nell'actor; i `result` escono dall'actor. L'intero insieme di frecce associato allo stesso proposition id rappresenta **una sola** `BAProposition produce`: le singole frecce visualizzano le participation della proposition e non creano proposition aggiuntive.

Nel worked example DermaTriage, `ClinicalDescription` e `RetrievedHistoricalCaseContext` sono mostrati con lo stesso simbolo documento bianco gia' usato per il contenuto informativo nella projection di `transfer`. Il role resta pero' quello della proposition corrente: rispettivamente `input` e `result`. La forma grafica non rinomina i role BA.

Parte di produce	Rappresentazione Mermaid
`input`	Nodo opzionale a sinistra; nel worked example informativo usa il simbolo documento bianco. Freccia direzionale verso l'actor. Ripetibile per `0..*`.
`actor`	Nodo centrale che identifica l'actor/capability della production.
`result`	Nodo a destra; nel worked example informativo usa il simbolo documento bianco. Freccia in uscita dall'actor. Ripetibile per `1..*`.
Proposition id / polarity	Metadati dell'intero pattern; non diventano nodi del flusso.

```
flowchart LR
  I@{ shape: doc, label: "ClinicalDescription<br/>input" }
  A["HistoricalCaseRetrieval<br/>actor"]
  R@{ shape: doc, label: "RetrievedHistoricalCaseContext<br/>result" }

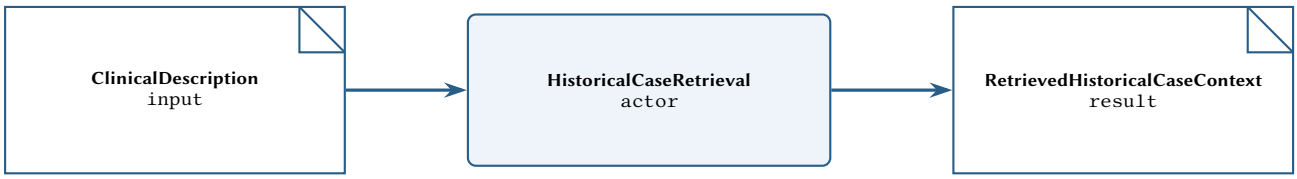
  I --> A --> R

  classDef actor fill:#EEF4F9,stroke:#275D86,color:#17324D
  classDef content fill:#FFFFFF,stroke:#275D86,color:#17324D
  class A actor
  class I,R content
```

Se `input` fosse assente, il nodo a sinistra e la relativa freccia verrebbero omessi. Con piu' input o result, i nodi vengono ripetuti e convergono/divergono dall'unico actor mantenendo lo stesso proposition id.

8.15 Esempio grafico corrispondente

Il diagramma seguente applica la convenzione alla proposition DermaTriage `BAP-FR03-03-01`. Il nodo centrale e' l'`actor`; l'input entra nell'actor e il result ne esce. I due referent informativi mantengono il simbolo documento bianco usato nella projection precedente. Il grafico non aggiunge destination, storage o dettagli di implementazione che `produce` non governa.



BAP-FR03-03-01 `produce` `polarity = affirmative`

Lettura del grafico

L'intero pattern `input -> actor -> result` e' la projection di una sola proposition `produce`. Se la stessa proposition avesse piu' input o piu' result, comparirebbero piu' rami verso o dall'actor senza cambiare il numero di BAProposition rappresentate.

9 Operatore `realize`

CURRENT / PRESERVED

9.1 Che cosa rappresenta

`realize` rappresenta l'asserzione governata che uno o piu' significati concreti **realizzano o materializzano un significato piu' astratto** nella baseline considerata. Il costrutto preserva la distinzione tra *che cosa* il progetto richiede o governa a livello astratto e *quale elemento concreto* ne costituisce la realizzazione corrente.

Smallest semantic fact

Esiste un significato astratto A e almeno una realizzazione concreta R tale che la documentazione governa che R realizzi/materializzi A. I due significati restano distinti BAReferent: la relation non li fonde e non trasforma automaticamente l'abstract in una classe software.

9.2 Quando usarlo

Usare `realize` quando la documentazione distingue un significato astratto — per esempio capability, contract, behavior o strategia richiesta — da uno o piu' elementi concreti scelti per materializzarlo nella baseline. La relazione deve essere governata: la semplice presenza di una tecnologia dentro un componente non basta.

9.3 Quando non usarlo

Non usare `realize` per significare che un componente usa una libreria, legge dati da un database, dipende da un servizio, produce un output o trasferisce informazione. Questi fatti possono richiedere altre proposition e non diventano realization solo perche' il concreto compare nell'implementazione.

Gate minimo

Se la source identifica soltanto l'elemento concreto ma non il significato astratto che esso realizza, non inventare retroattivamente l'abstract per completare la BA. Registrare `INSUFFICIENT EVIDENCE / NOT SPECIFIED` e, quando materialmente utile, riportare il gap alla review documentale.

9.4 Firma canonica

La firma completa e':

```
realize
  abstract     -> BAReferent [1]
  realization  -> BAReferent [1..*]
  polarity     = affirmative | negative [1]
```

Gli scoped modifier `condition` e `temporalScope` compaiono soltanto quando la source limita realmente l'asserzione.

Role	Card.	Significato
`abstract`	1	Significato astratto che la baseline governa indipendentemente dalla realizzazione concreta scelta.
`realization`	1..*	Uno o piu' BAReferent concreti che la documentazione identifica come materializzazione dell'abstract.

Cardinalita' multipla non significa alternativa

La cardinalita' `realization [1..*]` permette a una proposition di riferirsi a piu' realization governate. Da questa cardinalita' non si deduce che esse siano alternative, intercambiabili, ridondanti o usate congiuntamente: tali distinzioni richiedono evidence separata.

9.5 Continuita' di identita' tra abstract e realization

`abstract` e `realization` sono significati distinti e mantengono identita' indipendente. La relation serve proprio a preservare contemporaneamente il significato stabile e la scelta concreta della baseline.

```
abstract capability  -> ClinicalDescriptionGeneration
realization concrete -> Qwen2-VL-7B-Instruct
```

La realization non sostituisce il BAReferent astratto. Se altri documenti qualificano, vincolano o riusano `ClinicalDescriptionGeneration`, tali proposition continuano a riferirsi all'abstract e non vengono automaticamente trasferite alla tecnologia concreta.

Funzione logica, continuita' nel lifecycle ed equivalenza relativa

Funzione logica e realization. Quando un componente tecnico ha un ruolo funzionale distinguibile, la BA preserva, quando source-supported, sia l'identita' della funzione logica sia l'identita' della realizzazione concreta e le collega con `realize`. L'ordine dell'analisi non determina l'identita': se la stessa entita' compare prima nel setup e poi nel sistema operativo, o viceversa, lo stesso BAReferent viene riusato finche' il significato resta invariato. Installazione, configurazione, runtime, adattamento o altre fasi del lifecycle non giustificano da sole copie phase-specific del referent.

Equivalenza relativa. Due componenti concreti non sono equivalenti in assoluto. Una sostituzione puo' mantenere stabile lo stesso abstract soltanto nella misura in cui la nuova realization realizza la stessa funzione logica e preserva i contract e i constraint governati applicabili. `realize` da sola non implica sostituibilita' drop-in: se cambiano input, output, vincoli o boundary funzionale, il change impact deve riaprire i significati coinvolti invece di nascondere il cambiamento come semplice sostituzione tecnologica.

9.6 Invarianti

- ogni `realize` accettato possiede esattamente un `abstract` e almeno una `realization`;
- abstract e realization restano BAReferent distinti e riusabili;
- piu' realization appartengono alla stessa proposition soltanto quando la stessa asserzione governata le lega allo stesso abstract;
- la relation deve esprimere materializzazione del significato astratto, non mera presenza tecnica o collaborazione;
- polarity negativa richiede una negazione governata della realization; assenza di evidence non equivale a `realize` negativo.

9.7 Forbidden inference

Dal solo `realize` non inferire subclassing, ereditarieta' software, equivalenza semantica completa, sostituibilita' drop-in, compatibilita' di interfaccia, deployment topology, ownership, dipendenza, consumo di servizio, flusso dati, produzione, transfer, algoritmo interno, modello di sicurezza o completezza delle realization possibili. La projection puo' ricordare una gerarchia abstract/concrete, ma non importa automaticamente la semantica UML delle classi.

9.8 Domande da porre alla documentazione

Le domande cercano la distinzione abstract/concrete prima della sintassi BA.

1. Qual e' il significato astratto che la documentazione governa indipendentemente dalla tecnologia o strategia corrente?
2. Quale elemento concreto la source dichiara che realizza, implementa o materializza quel significato?
3. Se rimuovo il nome della tecnologia concreta, l'abstract resta un significato comprensibile e governato? Se no, verificare che non stiamo inventando un livello astratto.
4. Il concreto realizza realmente l'abstract oppure lo usa, lo invoca, ne dipende, gli fornisce input o ne riceve output?
5. Abstract e realization hanno gia' BAReferent con la stessa identita' semantica? Se si', riusarli.
6. Se compaiono piu' realization, la documentazione le lega davvero allo stesso abstract nella stessa asserzione? Non inferire alternative o composizione.
7. La source governa condition o temporal scope della realization?
8. La source afferma o nega esplicitamente la realization? Non dedurre polarity negativa dall'assenza di una tecnologia.
9. La tecnologia e' nominata ma la funzione astratta resta implicita? Fermarsi e registrare un possibile gap documentale invece di completarlo per plausibilita'.

Procedura minima

Isolare prima il significato astratto; verificare poi che la source identifichi esplicitamente almeno una realizzazione concreta; riusare le identita' BA esistenti; compilare una sola proposition minima; infine riesaminare che nessuna relazione di uso, dipendenza, produzione o trasferimento sia stata compressa impropriamente dentro `realize`.

9.9 Esempio generico minimo

Documentazione inventata ma coerente

"La capability `DocumentClassification` MUST essere realizzata nella baseline corrente dalla strategia `TransformerClassificationStrategy`."

Assumendo che le identita' abbiano superato il normale gate BA1, la proposition e':

```
realize
  abstract     -> DocumentClassification
  realization  -> TransformerClassificationStrategy
  polarity     -> affirmative
```

Il fatto preservato e' la scelta concreta rispetto a un significato astratto gia' distinguibile; non vengono inferiti modello specifico, formato degli input o contratto degli output.

9.10 Esempio corrente - DermaTriage Stage 2

La documentazione corrente distingue la capability `ClinicalDescriptionGeneration` dalla realizzazione concreta usata nello Stage 2. L'analisi corrente rappresenta quindi:

```
BAP-FR03-02-02
semanticOperatorKey = realize
  abstract     -> ClinicalDescriptionGeneration
  realization  -> Qwen2-VL-7B-Instruct
polarity = affirmative
```

Questa proposition non sostituisce la proposition `produce` che governa la produzione di `ClinicalDescription`. `realize` risponde a "quale elemento concreto materializza la capability?", mentre `produce` preserva il fatto che la capability rende disponibile il risultato.

Boundary example

"Il componente `A` usa `TechnologyB`" non basta a costruire `realize`. Senza un abstract governato e una relazione di materializzazione esplicita, l'uso della tecnologia resta un fatto diverso oppure un gap da riesaminare nella documentazione.

9.11 Prompt LLM di supporto all'authoring

Il prompt non deve creare un abstract soltanto per rendere sostituibile o piu' elegante una tecnologia gia' nominata. Un gap documentale resta un gap.

```
TASK: identify governed BA realize propositions.
Use ONLY the supplied documentation and existing BA registry.

A realize proposition exists only if the source supports:
  abstract [1], realization [1..*].

Rules:
1. Identify the smallest governed abstract/concrete materialization fact.
2. Reuse existing BAReferents when semantic identity is unchanged.
3. Propose a new BAReferent only when BA1 independent identity is needed.
4. Accept realization only when the source governs that the concrete referent
   realizes/materializes the abstract referent.
5. Do not convert mere use, dependency, invocation, production or data flow
   into realization.
6. Keep multiple realizations in one proposition only when the same governed
   assertion binds them to the same abstract.
7. Do not infer alternatives, replaceability, composition or completeness.
8. Use polarity=negative only for an explicitly negated realization assertion.
9. Add condition/temporalScope only when explicitly governed.
10. If abstract or every realization is missing, return INSUFFICIENT_EVIDENCE.
11. If a concrete technology is named but its abstract function is not governed,
    report the documentation gap; do not invent the abstraction.
12. Do not invent operators, roles, states or metadata.

Return EXACTLY the authoring wrapper and BA output shape shown next.
```

Human review remains mandatory

La distinzione fra "realizza" e "usa" e' semanticamente materiale. L'LLM puo' proporla, ma l'accettazione richiede verifica contro la source e contro le altre proposition gia' estratte, soprattutto quando la tecnologia partecipa anche a produzione, servizio o transfer.

9.12 Output esatto atteso dall'estrazione assistita

Il wrapper seguente e' un formato di authoring/review, non un nuovo costrutto BA. La parte di output BA usa soltanto la shape ammessa.

```
REALIZE_ASSESSMENT
  decision = ACCEPT_REALIZE | NOT_A_REALIZE | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  documentationGap = <abstract function missing | none>
  notSpecified = [<only unresolved material items>]

BA_OUTPUT
  BAReferent reuse/candidates:
    <canonical names only as needed>

  BAProposition <candidate-id>
    semanticOperatorKey = realize
    abstract    -> <BAReferent>
    realization -> <BAReferent>      # repeat for 1..*
    polarity = affirmative | negative
    scopedModifier = <condition|temporalScope>  # omit if absent
```

Se `decision != ACCEPT_REALIZE`, `BA_OUTPUT` non contiene una proposition `realize` accettabile. `documentationGap` appartiene al wrapper di review e non alla BA: serve soltanto a rendere visibile quando una tecnologia concreta e' documentata senza che sia governata la capability astratta che ci si aspetterebbe di poter sostituire o riesaminare.

9.13 Discriminazioni con altri costrutti

-

9.14 Rappresentazione Mermaid di realize

La projection usa una disposizione verticale simile alla relazione abstract/concrete: l'`abstract` e' sopra, la `realization` concreta sotto e una linea tratteggiata con triangolo vuoto punta verso l'abstract. La convenzione rende immediata la lettura "il concreto realizza l'astratto" senza trasformare la BA in un modello di classi.

Parte di realize	Rappresentazione Mermaid
`abstract`	Nodo superiore, marcato graficamente come significato astratto.
`realization`	Nodo inferiore; ripetibile per `1..*`.
Relazione	Linea tratteggiata con triangolo vuoto rivolto verso l'abstract e label `realize`.
Proposition id / polarity	Metadati dell'intero pattern; non diventano nodi della gerarchia.

```
classDiagram
direction BT

class ClinicalDescriptionGeneration {
  <<abstract>>
}

class Qwen2VL7BInstruct {
  <<realization>>
}

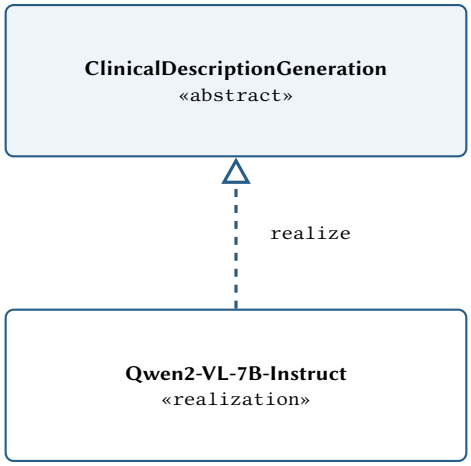
Qwen2VL7BInstruct ..|> ClinicalDescriptionGeneration : realize
```

Somiglianza grafica non equivale a subclassing

Il triangolo e la disposizione verticale sono una convenzione di projection per rendere visibile la relation abstract/concrete. Non autorizzano inferenze di ereditarieta', type hierarchy, interface conformance o object-oriented design.

9.15 Esempio grafico corrispondente

Il diagramma applica la convenzione alla proposition DermaTriage `BAP-FR03-02-02`. `ClinicalDescriptionGenera` resta il significato astratto; `Qwen2-VL-7B-Instruct` e' la realizzazione concreta della baseline. Il grafico non rappresenta input, output o transfer: questi appartengono ad altre proposition.



BAP-FR03-02-02 `realize` `polarity = affirmative`

Lettura del grafico

La freccia si legge dal concreto verso l'astratto: `Qwen2-VL-7B-Instruct` realizza `ClinicalDescriptionGeneration`. Se una stessa proposition contenesse piu' realization governate, piu' rami tratteggiati convergerebbero sullo stesso abstract mantenendo un unico proposition id; il grafico non dichiarerebbe da solo se tali realization siano alternative o congiunte.

10 Operatore `constrain`

CURRENT / PRESERVED

10.1 Che cosa rappresenta

`constrain` rappresenta una **restrizione governata, riusabile e interrogabile** applicata a un significato identificato. Il costrutto serve quando il progetto non si limita a nominare un dato, un comportamento o uno stato, ma ne governa anche un contratto, un dominio ammesso, una proprieta' richiesta o un'altra limitazione stabile che deve poter essere recuperata e analizzata separatamente.

Smallest semantic fact

Esiste un target governato `T` e almeno una restriction `C` tale che la documentazione stabilisce che `T` sia soggetto a `C`. La restriction puo' essere un significato governato con identita' propria oppure una struttura locale controllata; in entrambi i casi il target resta esplicito.

10.2 Quando usarlo

Usare `constrain` quando la restriction deve restare distinguibile e interrogabile: per esempio un data/content contract, un allowed domain, una proprieta' obbligatoria, un requisito di confidenzialita' applicato a un delivery o un medium imposto a uno specifico comportamento di trasferimento.

10.3 Quando non usarlo

Non usare `constrain` per descrivere quale valore possiede una singola occorrenza runtime, quale ramo di una regola decisionale viene scelto, quale elemento viene selezionato, chi produce o trasferisce un dato, oppure quale meccanismo esegue materialmente il controllo. La restriction qualifica il target: non sostituisce il comportamento che produce, trasporta, seleziona o verifica.

Gate minimo

Prima di introdurre `constrain`, identificare il target esatto. Se non e' chiaro se la restriction appartenga al contenuto trasferito, al behavior di transfer, a un contratto riusabile o a un altro significato governato, non propagare il vincolo per convenienza: mantenere il gap esplicito e tornare alla documentazione quando necessario.

10.4 Firma canonica

La firma top-level resta unica:

```
constrain
  constraintTarget -> BAReferent [1]
  constraintValue  -> controlled value/structure [1..*]
  polarity         = affirmative | negative [1]
```

Gli scoped modifier `condition` e `temporalScope` compaiono soltanto quando la source limita realmente la restriction.

Role	Card.	Significato
`constraintTarget`	1	BAReferent il cui significato e' ristretto dalla proposition.
`constraintValue`	1..*	Restriction governata: puo' essere un BAReferent con identita' propria oppure una forma locale controllata.

10.5 Due modi di esprimere la restriction

A. Constraint nominato / riusabile. Quando il contratto o la restriction possiede identita' analitica indipendente:

```
constraintValue -> <BAReferent>
```

Esempio: `ClinicalDescriptionContentContract`. Il nome non e' un'etichetta grafica: e' un BAReferent riusabile e change-addressable.

La stessa forma copre un configuration profile con identita' BA indipendente:

```
DermaTriageEnvironment
  constrained by -> HardwareConfigurationProfile
```

`HardwareConfigurationProfile` resta un normale BAReferent usato come `constraintValue`: non introduce un ruolo `constraintSpecification`, un nuovo operatore o una terza shape di `constrain`. Il profilo puo' essere aperto a un livello successivo mediante ulteriori proposition che ne rendono interrogabile la struttura governata.

B. Constraint locale. Quando non serve una nuova identita' BA, la restriction resta inline. Puo' essere un dominio locale controllato, oppure una struttura di proprieta':

```
constraintValue
  property   -> <controlled semantic key> [1]
  presence   -> required [0..1]
  vocabulary -> [<controlled typed local value | BAReferent> 1..*] [0..1]
```

Un allowed domain semplice puo' essere espresso direttamente come `constraintValue` locale, senza creare un BAReferent artificiale soltanto per ospitare l'elenco.

10.6 Continuita' di identita' e livello del constraint

Il `constraintTarget` mantiene la propria identita' indipendentemente dalla restriction. Se `constraintValue` e' un BAReferent nominato, anche il constraint mantiene identita' separata e puo' a sua volta essere target di altre proposition.

```
ClinicalDescription
  constrained by -> ClinicalDescriptionContentContract

ClinicalDescriptionContentContract
  constrained by -> structured local restrictions
```

Questa catena permette di distinguere *a quale dato si applica il contratto* da *come il contratto e' composto*. Una modifica della struttura interna del contratto non richiede di rinominare il dato o di duplicare tutte le proposition che vi fanno riferimento.

10.7 Invarianti

- ogni proposition accettata possiede esattamente un `constraintTarget` e almeno un `constraintValue`;
- un BAReferent usato come constraint nominato richiede normale giustificazione BA1 di identita' indipendente;
- un configuration profile accettato come BAReferent puo' essere `constraintValue` quando la source governa che il target debba rispettare quella configurazione;
- l'identita' del profilo non rende automaticamente normative le sue entry e non promuove automaticamente property, package, versioni o valori locali a BAReferent;
- un profilo software/BOM puo' restare analiticamente riusabile per change, vulnerability e supply-chain impact anche quando le singole versioni sono current realization;
- un configuration profile puo' a sua volta essere `constraintTarget` di proposition piu' fini quando la struttura interna e' source-supported;
- `presence -> required` afferma la presenza della proprieta'; la sola `vocabulary` non la implica;
- piu' constraintValue restano nella stessa proposition soltanto se la stessa asserzione governata li applica allo stesso target e scope;
- una restriction sul content non si propaga automaticamente al behavior di transfer, e una restriction sul behavior non modifica automaticamente lo schema del content.

10.8 Forbidden inference

Dal solo `constrain` non inferire il valore runtime osservato, validazione riuscita, enforcement mechanism, *apply/load* della configurazione, algoritmo, ordine di valutazione, selection, mapping decisionale, produzione, transfer, ownership, responsibility, encryption effettivamente applicata o conformita' di una specifica occorrenza. `Target constrain Profile` afferma il vincolo governato, non che una procedura applichi materialmente il profilo. Un allowed domain non dice quale valore verra' scelto; un data contract non dice da solo chi invia o riceve il dato.

10.9 Domande da porre alla documentazione

Le domande servono prima a localizzare la restriction e soltanto dopo a scegliere la sua forma.

1. Qual e' il significato esatto che viene ristretto? Il target e' un dato/content, un behavior di transfer, uno stato, un contratto o altro BAReferent?
2. La source governa realmente una restriction stabile e interrogabile oppure descrive soltanto un dettaglio locale senza utilita' analitica autonoma?
3. La restriction possiede un nome/identita' riusabile nel progetto? Se si', esiste o serve un BAReferent dedicato?
4. Se la restriction resta locale, e' un dominio diretto oppure una proprieta' strutturata?
5. Per una proprieta' strutturata, qual e' il `property` key governato?
6. La source richiede che la proprieta' sia presente? Usare `presence -> required` solo in questo caso.
7. La source governa un insieme chiuso di valori ammessi? Inserirlo in `vocabulary` senza dedurre automaticamente la presenza.
8. Il vincolo riguarda il dato stesso oppure il modo in cui quel dato viene trasferito? Non spostare il target tra content e transfer behavior.
9. Piu' restriction appartengono davvero alla stessa asserzione e allo stesso scope oppure devono restare proposition separate?
10. La restriction vale sempre oppure soltanto sotto condition/temporal scope governati?
11. La source afferma o nega esplicitamente il constraint? Non dedurre polarity negativa dall'assenza di un valore o di una proprieta'.
12. Se property, domain, scope o target non sono governati, lasciare `NOT SPECIFIED` / diagnosis invece di completarli per plausibilita'.

Procedura minima

Identificare il target; determinare se la restriction richiede identita' propria oppure resta locale; compilare soltanto i dettagli effettivamente governati; verificare la separazione content/transfer behavior; infine riesaminare che la proposition non stia simulando una regola decisionale, una selection o un runtime fact.

10.10 Esempio generico minimo

Documentazione inventata ma coerente

"Ogni `TelemetryRecord` trasferito tra i componenti MUST rispettare il `TelemetryRecordContract`. Il contratto richiede rappresentazione JSON e la presenza dei campi `eventId` e `timestamp`."

Assumendo identita' BA gia' giustificate:

```
constrain
  constraintTarget -> TelemetryRecord
  constraintValue  -> TelemetryRecordContract
  polarity = affirmative

constrain
  constraintTarget -> TelemetryRecordContract
  constraintValue
    property   -> representation
    presence   -> required
    vocabulary -> [JSON]
  constraintValue
    property -> eventId
    presence -> required
  constraintValue
    property -> timestamp
    presence -> required
  polarity = affirmative
```

10.11 Esempio corrente - DermaTriage Stage 2

La documentazione corrente stabilisce che `ClinicalDescription` abbia un contratto di contenuto riusabile e che tale contratto sia testuale, composto da cinque parti richieste.

```
BAP-FR03-02-03  constrain
  constraintTarget -> ClinicalDescription
  constraintValue  -> ClinicalDescriptionContentContract

BAP-FR03-02-04  constrain
  constraintTarget -> ClinicalDescriptionContentContract
  constraintValue -> representation required in [TEXTUAL]
  constraintValue -> partCount required in [5]
  constraintValue -> shape required
  constraintValue -> borders required
  constraintValue -> color required
  constraintValue -> texture required
  constraintValue -> suspiciousFeatures required
```

Il primo fatto identifica *quale contratto governa il dato*; il secondo rende interrogabile *come quel contratto e' composto*. I due livelli non vengono fusi.

10.12 Prompt LLM di supporto all'authoring

Il prompt deve preservare sia il target corretto sia il livello del constraint. In particolare non deve spostare una restriction dal dato al transfer behavior o viceversa soltanto per rendere il diagramma piu' comodo.

```
TASK: identify governed BA constrain propositions.
Use ONLY the supplied documentation and existing BA registry.

A constrain proposition exists only if the source supports:
  constraintTarget [1], constraintValue [1..*].

Rules:
1. Identify the smallest governed reusable/queryable restriction fact.
2. Identify the exact target before extracting the restriction.
3. Reuse existing BAReferents when semantic identity is unchanged.
4. Use a named constraint BAReferent only when independent BA1 identity is
   justified; otherwise keep the restriction local.
5. When the source identifies an active configuration/profile as a stable set
   that must be reused, compared across baselines or queried for change/security
   impact, apply BA1 to the set itself; do not keep it local merely because its
   entries are concrete.
6. Promoting a configuration/profile does not promote its entries. Preserve
   package/version/BOM or current-realization facts without inventing normativity.
7. For local structured constraints, use only governed property, presence and
   vocabulary information. Do not invent schema fields.
8. presence=required only when the source explicitly requires presence.
9. vocabulary does not imply presence.
10. Preserve the distinction between content constraints and transfer-behavior
    constraints. Do not move a restriction between them without source support.
11. Do not turn runtime values, validation results, decision mappings, selection,
    production, transfer or configuration-application behavior into constrain.
12. Keep multiple constraintValue entries together only when they share the same
    governed target and assertion scope.
13. Use polarity=negative only for an explicitly negated restriction assertion.
14. Add condition/temporalScope only when explicitly governed.
15. If target or every restriction value is missing, return INSUFFICIENT_EVIDENCE.
16. Do not invent operators, roles, states, BAReferents or metadata.

Return EXACTLY the authoring wrapper and BA output shape shown next.
```

Human review remains mandatory

L'LLM puo' riconoscere la forma del constraint, ma la review umana deve verificare soprattutto identity promotion, scope e placement: lo stesso vocabolo tecnico puo' qualificare il content, il behavior di transfer o un contratto nominato con effetti analitici diversi.

10.13 Output esatto atteso dall'estrazione assistita

Il wrapper distingue la decisione di authoring dalla BA. `constraintForm` serve alla review e non introduce un nuovo field BA.

```
CONSTRAIN_ASSESSMENT
  decision = ACCEPT_CONSTRAIN | NOT_A_CONSTRAINT | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  constraintForm = NAMED_REFERENT | LOCAL_DIRECT | LOCAL_STRUCTURED
  targetKind = CONTENT | TRANSFER_BEHAVIOR | OTHER
  documentationGap = <material unresolved meaning | none>
  notSpecified = [<only unresolved material items>]

BA_OUTPUT
  BAReferent reuse/candidates:
    <canonical names only as needed>

  BAProposition <candidate-id>
    semanticOperatorKey = constrain
    constraintTarget -> <BAReferent>
    constraintValue  -> <BAReferent | controlled local value/domain>
      # OR repeat local structured values as governed:
      property   -> <controlled semantic key>
      presence   -> required            # omit if absent
      vocabulary -> [<controlled values>] # omit if absent
    polarity = affirmative | negative
    scopedModifier = <condition|temporalScope>  # omit if absent
```

Se `decision != ACCEPT_CONSTRAIN`, `BA_OUTPUT` non contiene una proposition `constrain` accettabile. `constraintForm` e `targetKind` sono soltanto controlli di authoring per evitare che la scelta della rappresentazione alteri il significato governato.

Un configuration profile riusabile usa `constraintForm = NAMED_REFERENT` e compare nel BA output come normale `constraintValue -> <BAReferent>`. Non introdurre campi ad hoc come `constraintSpecification`; l'eventuale decomposizione del profilo avviene con proposition BA ulteriori e source-supported.

10.14 Discriminazioni con altri costrutti

-

10.15 Rappresentazione Mermaid di constrain

La projection mantiene visibile il `constraintTarget` e colloca la restriction **vicino al significato che qualifica**. Un constraint nominato usa un nodo dedicato; una restriction locale strutturata puo' essere resa come nota/card collegata al target. La linea tratteggiata etichettata `constrain` visualizza una BAProposition distinta e non modifica le frecce di data flow.

Target del constraint	Placement nella view
Content / dato trasferito	Constraint accanto al nodo documento/content. Source e destination continuano a leggere lo stesso BAReferent informativo.
Transfer behavior	Constraint accanto al BAReferent del behavior/segmento di transfer, non al content.
Constraint nominato	Nodo riusabile collegato al target con linea tratteggiata `constrain`. Un configuration profile usa questa stessa projection.
Constraint locale strutturato	Nota/card vicina al target, con le sole property/presence/vocabulary governate.

Per un configuration profile, una view meno dettagliata puo' fermarsi al BAReferent del profilo e alla proposition `target constrain profile`; una view successiva puo' aprire il profilo mostrando constraint interni source-supported. Il cambio di livello e' una scelta del View Contract: non crea una nuova semantica e non autorizza a inventare detail assenti dalla BA.

```
flowchart LR
  S["ClinicalDescriptionGeneration<br/>source"]
  C@{ shape: doc, label: "ClinicalDescription<br/>content" }
  D["HistoricalCaseRetrieval<br/>destination"]
  K["ClinicalDescriptionContentContract<br/>named constraint"]
  F["TEXTUAL; 5 parts; required fields<br/>structured constraint"]

  S --> C --> D
  C -.->|constrain| K
  K -.->|constrain| F

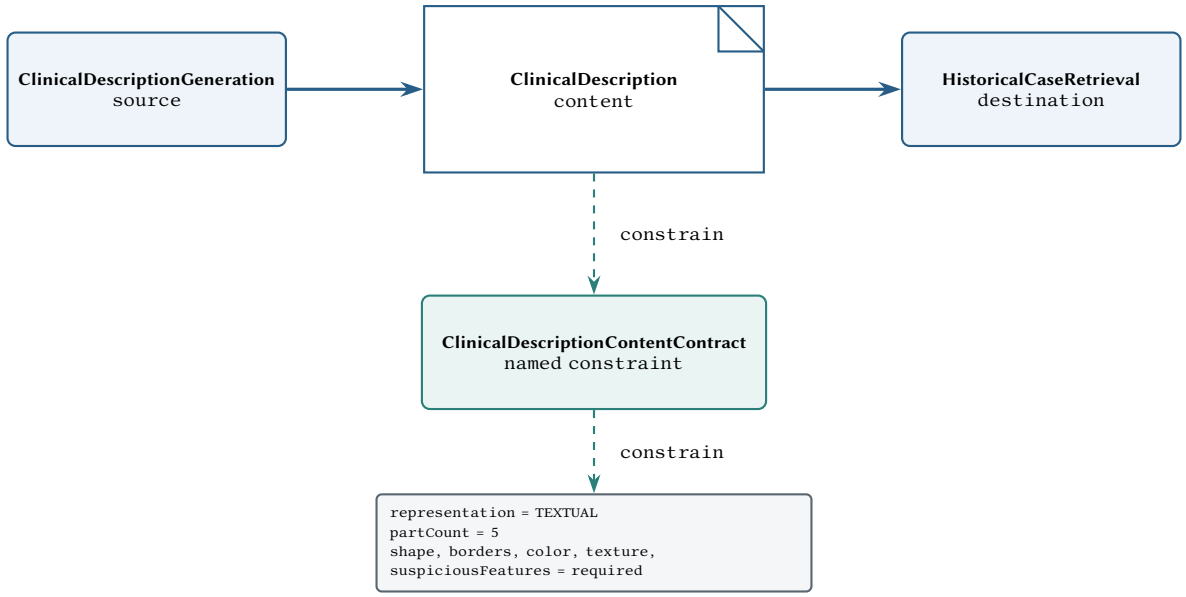
  classDef endpoint fill:#EEF4F9,stroke:#275D86,color:#17324D
  classDef content fill:#FFFFFF,stroke:#275D86,color:#17324D
  classDef contract fill:#EAF5F3,stroke:#2A7F78,color:#17324D
  classDef detail fill:#F4F6F8,stroke:#5B6770,color:#17324D
  class S,D endpoint
  class C content
  class K contract
  class F detail
```

Placement non equivale a propagazione

Mostrare un data contract vicino a un transfer non significa che il transfer stesso sia il `constraintTarget`. Il target resta il content quando la restriction descrive forma o dominio del dato; diventa il transfer behavior soltanto quando la source governa una restriction sul delivery, per esempio medium o confidentiality del segmento.

10.16 Esempio grafico corrispondente

Il diagramma combina senza fonderle tre proposition: il `transfer` di `ClinicalDescription`, il constraint che associa il dato al `ClinicalDescriptionContentContract` e il constraint che descrive il contratto. Il data path resta `source -> content -> destination`; le linee tratteggiate appartengono a proposition `constrain` separate.



Lettura del grafico

Chi produce e chi riceve condividono lo stesso `ClinicalDescription`; il contratto rende visibile che cosa quel dato deve rispettare. Se una restriction riguardasse invece il delivery — per esempio il medium o la confidentiality del trasferimento — il nodo di constraint verrebbe collegato al BAReferent del transfer behavior e non a `ClinicalDescription`.

11 Scoped modifier `condition`

CURRENT / PRESERVED

11.1 Che cosa rappresenta

`condition` e' uno **scoped modifier BA2** che limita localmente l'applicabilita' di una BAProposition. Il fatto espresso dall'operatore e dai suoi participant non cambia identita': la condition stabilisce soltanto *quando quella specifica asserzione si applica.* Non e' un nuovo top-level operator e non e' una decision rule abbreviata.

Smallest semantic fact

Esiste una BAProposition governata `P` e una condizione locale `C` tale che `P` e' asserita soltanto quando `C` vale nello scope documentato. Eliminando `condition` si perderebbe il confine di applicabilita' della proposition, non il significato del suo operatore.

11.2 Quando usarlo

Usare `condition` quando la source governa un guard/applicability scope per una singola proposition e quel significato non richiede identita', provenance, participant o riuso autonomi. La condizione deve essere source-grounded; non puo' essere dedotta soltanto perche' un comportamento sembra plausibile in uno scenario.

11.3 Quando non usarlo

Non usare `condition` per codificare un mapping IF/THEN/ELSE che determina quale risultato viene prodotto, per una dependency/prerequisite, per un constraint riusabile o per nascondere un significato che deve diventare BAReferent/BAProposition autonomo.

Gate minimo

La domanda non e' "c'e' un if nella frase?", ma "la documentazione limita quando questa stessa proposition vale?". Se la condition seleziona risultati, introduce participant propri o deve essere riusata indipendentemente, fermarsi e riesaminare il costrutto.

11.4 Forma canonica nella BAProposition

La grammatica BA2 preservata ammette `scopedModifier [0..*]` e, nel lower bound corrente, soltanto i kind `condition` e `temporalScope`. R7 rende esplicita `condition` con la seguente forma di authoring:

```
BAProposition
  semanticOperatorKey = <operator>
  participation ...
  polarity = affirmative | negative

  scopedModifier
    condition -> <source-grounded local applicability statement>
```

La parte destra di `condition` e' contenuto locale controllato della proposition, non un BAReferent implicito. La serializzazione deve preservare la formulazione canonica scelta durante la review senza inventare property, boolean o stati non governati.

Elemento	Card.	Significato
`scopedModifier`	0..*	Modifier locali ammessi dalla BAProposition; il lower bound corrente contiene `condition` e `temporalScope`.
`condition`	per instance 1	Guard/applicability statement governato per la proposition che lo contiene.

Nessun DSL logico generale

La forma locale non introduce automaticamente AND, OR, truth table, property model o evaluation order. Se la source governa una costruzione decisionale che seleziona un risultato, il candidato da riesaminare e' `decisionRule`, non una condition sempre piu' complessa.

11.5 Locality / identity boundary

Una condition resta embedded soltanto finche' modifica localmente una proposition e non richiede identita' analitica indipendente. BA2 R2/R3 preserva il promotion test: se il significato viene riusato, separatamente revisionato, diventa participant-bearing o deve essere target di altre asserzioni, si applicano i normali criteri BA1 invece di nasconderlo nel modifier.

```
LOCAL
  produce(...)
  condition -> LesionImage unavailable

PROMOTION PRESSURE
  NoImageCondition must be reused / constrained / traced independently
  -> evaluate BAReferent / BAProposition identity under BA1
```

11.6 Invarianti

- la condition qualifica l'intera BAProposition, non un singolo edge scelto dal renderer;
- l'operatore, i roleKey e i BAReferent della proposition mantengono la stessa identita';
- una condition locale non crea automaticamente un nuovo BAReferent;
- l'assenza della condition opposta non autorizza a inventare un ramo ELSE;
- l'ordine testuale di piu' facts o view non implica evaluation order;
- la condition deve essere governata dalla source allo stesso livello documentale che giustifica la proposition o da evidence tracciabile compatibile con tale scope.

11.7 Forbidden inference

Da `condition` non inferire detection logic, boolean property, stato runtime persistente, branch complementare, fallback, precedence, sequencing, retry/failure semantics, dependency, constraint riusabile o mapping di risultato. "Image unavailable" non autorizza da solo a inventare `LesionImage.available = FALSE`.

11.8 Domande da porre alla documentazione

1. Quale BAProposition e' limitata dalla condizione? Il modifier deve avere un owner propositionale preciso.
2. La source sta dicendo *quando la proposition vale* oppure *come una condizione determina quale risultato viene scelto*?
3. La condition e' esplicita/governata oppure la stiamo deducendo da un percorso ritenuto probabile?
4. Il significato puo' restare locale senza BAReferent autonomo?
5. E' riusato, separatamente revisionato, participant-bearing o target di altre proposition? Se si', applicare l'identity test BA1.
6. La formulazione richiederebbe di inventare property, boolean, enum o stato? Se si', non completarli per plausibilita'.
7. Esiste davvero un ramo opposto governato? Una condition non implica automaticamente `ELSE` o il complemento logico.
8. La source governa una dependency/prerequisite? In tal caso verificare `dependOn`, non `condition`.
9. La source governa una restriction riusabile? In tal caso verificare `constrain`.
10. La condition e' temporale? Se descrive before/after/during/until, verificare `temporalScope` invece di condition.

Procedura minima

Estrarre prima la BAProposition senza modifier; verificare che il fatto sia semanticamente completo; isolare poi il guard governato; applicare locality/identity test; aggiungere `condition` soltanto se cambia l'applicabilita' della stessa proposition. Infine controllare che non stia simulando decisionRule, dependOn o constrain.

11.9 Esempio generico minimo

Documentazione inventata ma coerente

"Quando la modalita' di manutenzione e' attiva, `SystemMonitor` produce un `MaintenanceAlert` a partire da `SensorStatus`."

Assumendo i BAReferent gia' giustificati:

```
produce
  actor  -> SystemMonitor
  input  -> SensorStatus
  result -> MaintenanceAlert
  polarity = affirmative
  scopedModifier
    condition -> MaintenanceMode active
```

La condition dice quando questa produzione si applica. Non afferma che `MaintenanceMode` sia un boolean, non crea un ramo per la modalita' inattiva e non descrive come viene rilevato lo stato.

11.10 Esempio corrente - DermaTriage symptom-only

La documentazione R25 stabilisce: "Quando l'immagine della lesione non e' disponibile, DermaTriage determina l'urgenza del caso utilizzando le informazioni sintomatologiche disponibili associate allo stesso caso."

```
produce
  actor  -> DermaTriage
  input  -> SymptomInformation
  result -> CaseUrgency
  polarity = affirmative
  scopedModifier
    condition -> LesionImage unavailable
```

Non serve materializzare `NoImageAvailableCondition` come BAReferent finche' il significato resta locale. Il percorso image-based con immagine disponibile deve essere supportato da proposition/documentazione proprie; non viene inventato come `ELSE` di questa proposition.

11.11 Prompt LLM di supporto all'authoring

```
TASK: identify governed proposition-local condition modifiers.
Use ONLY the supplied documentation, accepted BA and current BA authority.

A condition modifier exists only when the source limits when an otherwise
complete BAProposition applies.

Rules:
1. Identify the complete BAProposition first: operator, roles and referents.
2. Extract condition only if the source explicitly governs the applicability guard.
3. The condition qualifies the whole proposition, not an arbitrary visual edge.
4. Keep the condition local when it has no independent identity/reuse.
5. Do not create a BAReferent only to name a one-use guard.
6. If the condition is reused, separately reviewed, participant-bearing or targeted
   by other propositions, stop and request BA1 identity review.
7. Do not invent boolean properties, enums, states or opposite conditions.
8. Do not infer ELSE, fallback or complementary behavior from one condition.
9. Do not use condition for IF/THEN/ELSE result construction; review decisionRule.
10. Do not use condition for prerequisite/dependency; review dependOn.
11. Do not use condition for reusable restrictions; review constrain.
12. Do not infer temporal order; temporal applicability belongs to temporalScope.
13. If the guard is not source-grounded, return INSUFFICIENT_EVIDENCE.
14. Do not invent operators, roles, BAReferents or metadata.

Return EXACTLY the authoring wrapper and BA output shape shown next.
```

Human review remains mandatory

La review umana deve verificare soprattutto il confine fra applicabilita' locale e semantica autonoma. La forma linguistica "quando/se" e' soltanto un indizio: non decide automaticamente condition, decisionRule o dependOn.

11.12 Output esatto atteso dall'estrazione assistita

```
CONDITION_ASSESSMENT
  decision = ACCEPT_CONDITION | NOT_A_CONDITION | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  ownerProposition = <candidate-id / identified governed fact>
  identityPressure = NONE | REVIEW_BA1
  notSpecified = [<only unresolved material items>]

BA_OUTPUT
  BAReferent reuse/candidates:
    <canonical names only as needed>

  BAProposition <candidate-id>
    semanticOperatorKey = <existing operator>
    <operator-scoped roles and terms>
    polarity = affirmative | negative
    scopedModifier
      condition -> <source-grounded local applicability statement>
```

`ownerProposition` e `identityPressure` appartengono al wrapper di review, non alla BA.

11.13 Discriminazioni con i costrutti vicini

Confine	Discriminatore
condition / decisionRule	condition cambia quando la proposition si applica; decisionRule governa come condizioni/input determinano un risultato.
condition / constrain	condition e' locale all'asserzione; constrain e' una restriction riusabile/interrogabile sul proprio target.
condition / dependOn	condition e' guard di applicabilita'; dependOn afferma prerequisite/dependency e supporta impact propagation.
condition / BAReferent	il modifier resta locale; se il significato richiede identita'/riuso autonomi, applicare BA1.
condition / temporalScope	condition governa un guard non temporale; before/after/during/until appartengono al temporal scope.

11.14 Rappresentazione Mermaid di condition

La `condition` qualifica l'intera BAProposition. La projection canonica la rende come un **tag di condition** colorato, posto sopra il ramo governato. Il tag non e' un BAReferent, non e' un input dell'operatore e non e' un nodo decisionale: visualizza soltanto lo `scopedModifier.condition` della proposition owner o del blocco di vista selezionato per quel ramo.

Semantica BA	Regola di projection
`condition` modifier	Un tag ellittico colorato con label canonica `<condition>`.
Applicabilita'	Il tag viene posto in testa al ramo governato; la sua posizione rende visibile l'applicabilita' locale senza introdurre dataflow o nuovi role BA.
Owner block	Nelle viste composte il ramo puo' essere rappresentato con *un solo blocco owner* per lato, quando l'obiettivo e' mostrare la separazione di condition e non l'espansione completa di tutte le proposition sottostanti.
Due condition distinte	Producono due tag distinti; nessun ramo viene inventato come complemento dell'altro.
Decision rule	`decisionRule` usa un tag ellittico analogo per forma al tag di `condition`, ma appartiene alla famiglia cromatica blu di `produce`; la shape comune non implica equivalenza semantica. `condition` resta ambra e non introduce automaticamente una forma `IF/ELSE`.

```
flowchart LR
  C1([Condition A])
  B1["Owner block A"]

  C2([Condition B])
  B2["Owner block B"]

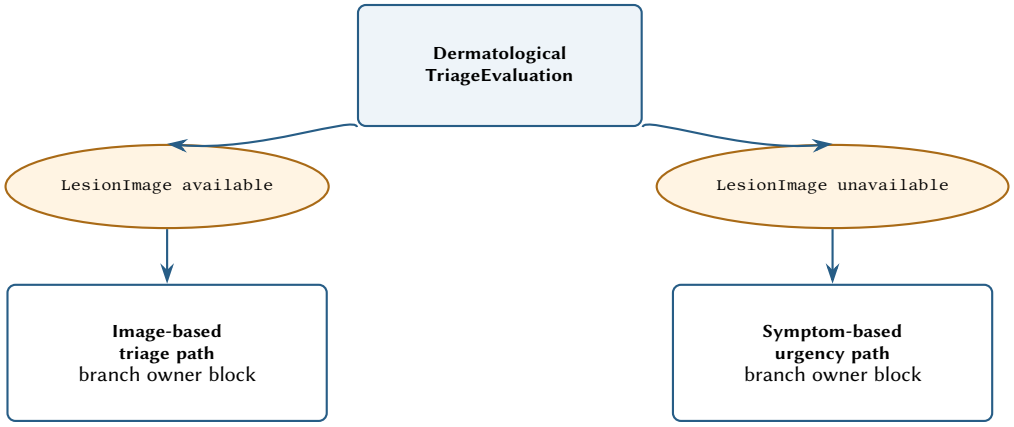
  C1 --> B1
  C2 --> B2
```

Semantic determinism della projection

Con la stessa BA accettata e lo stesso projection profile devono restare invariati: numero dei tag di condition, shape ellittica, label canonica della condition e ramo owner. La posizione geometrica dei nodi puo' variare: in questa fase non richiediamo layout determinism. Se la vista usa un solo blocco owner per ramo, anche quella scelta deve essere esplicita e coerente con lo scope della view.

11.15 Esempio grafico corrispondente

Il diagramma mostra sullo stesso foglio due rami con `condition` distinte. I tag colorati `LesionImage available` e `LesionImage unavailable` stanno sopra i rispettivi rami. Per mantenere la vista leggibile, ogni lato della condition e' rappresentato con **un solo blocco owner** invece di espandere tutto cio' che sarebbe ulteriormente derivabile dalla documentazione.



`condition = LesionImage available` `condition = LesionImage unavailable`

Lettura del grafico

Il tag colorato indica soltanto la condition del ramo. Non e' un nodo decisionale e non implica automaticamente un ramo complementare. In questa vista ciascun lato e' compresso in un solo blocco owner: la projection evidenzia il fatto che due condition diverse governano due rami diversi, ma non espande automaticamente l'intera struttura interna dei rami.

12 Operator structure `decisionRule`

CANDIDATE / NOT ADMITTED

12.1 Che cosa rappresenta

`decisionRule` e' qui una **candidate operator structure** locale a una BAProposition `produce`. Rappresenta il caso in cui la source non governa soltanto che un actor produce un result da uno o piu' input, ma governa anche *come condizioni o valori degli input selezionano il valore del result.* La candidate non rinomina il costrutto: riesamina soltanto la collocazione dello storico `decisionRule`. Actor, input e result restano quelli dell'owner `produce`; la struttura decisionale viene annidata. La regression corrente comprende due famiglie DermaTriage positive indipendenti, derivazione della priorita' P-scale e qualificazione della readiness C7, e aggiunge ora i pressure case C7-P02 sugli endpoint: comparison stretta `lt`, validita' rispetto a un dominio governato e occorrenza di un fatto governato.

Smallest semantic fact

Esiste una proposition `produce` gia' completa e una regola governata `R` che determina quale valore del suo result viene selezionato a partire dagli input/condizioni documentati. Eliminando `decisionRule` si conserva il fatto che il result viene prodotto, ma si perde il mapping operativo governato che seleziona il valore del result.

12.2 Quando usarla

Usare la candidate quando la documentazione esplicita un mapping del tipo "in queste condizioni, il result assume questo valore" e actor/input/result coincidono semanticamente con quelli di una production gia' identificata. La regola deve essere governata: non basta che un mapping sia plausibile o implementato nel codice.

12.3 Quando non usarla

Non usarla per dire soltanto quando la proposition si applica: quello resta `scopedModifier.condition`. Non usarla per dichiarare soltanto il dominio ammesso del result: quello resta `constrain`. Non usarla per dependency, sequencing o fallback non governati.

Delta rispetto all'authority corrente

R3 mantiene `decisionRule` come top-level operator con propri role `actor/input/result`. R7 non cambia tale authority: testa se lo stesso `decisionRule` possa essere rilocato come `operatorStructure` dell'owner `produce`, evitando la duplicazione di actor/input/result. La candidate richiede regression e promotion esplicita.

12.4 Forma canonica candidate dentro `produce`

La forma di rebuild mantiene la proposition `produce` come owner del fatto produttivo e aggiunge al massimo una struttura locale di selezione del risultato:

```
BAProposition
  semanticOperatorKey = produce
  participation
    actor  -> <BAReferent> [1]
    input  -> <BAReferent> [0..*]
    result -> <BAReferent> [1..*]
  polarity = affirmative | negative
  scopedModifier [0..*]
    condition -> <local applicability guard>
  operatorStructure [0..1]
    decisionRule
      branch [1..*]
        when -> <source-grounded selection condition> [1]
        resultAssignment [1..*]
          target -> <result BAReferent>
          value  -> <controlled local value | BAReferent>
```

Elemento	Card.	Significato
`operatorStructure`	0..1	Struttura locale dell'owner proposition; non crea da sola una seconda BAProposition.
`decisionRule`	per owner 1	Mapping governato che seleziona il valore del result.
`branch`	1..*	Branch esplicitamente governati dalla source.
`when`	per branch 1	Condizione di selezione source-grounded; puo' preservare una residual condition esplicita senza inventare un ELSE globale.
`resultAssignment`	1..*	Assegnazione governata a un result gia' partecipante all'owner `produce`.

Comparison key sintetici nella candidate R7

La candidate normalizza le due relazioni gia' presenti nell'authority storica: `equals -> eq` e `notEquals -> ne`. Questa rinomina non estende la semantica. Il pressure test priority DermaTriage richiede `confidence > 0.85`, quindi `gt`; il test C7-P02 su `/analyze` richiede realmente `response time < 30s`, quindi aggiunge `lt` come seconda comparison candidate source-driven. I simboli `=`, `!=`, `>` e `<` sono rendering leggibili, non semantic key alternativi. `le`, `ge`, range e aritmetica restano non introdotti finche' nuovi casi governati non li richiedono.

12.5 Locality / identity boundary

Nella candidate, il `decisionRule` annidato condivide l'identita' della BAProposition `produce` che qualifica. Non viene creato un secondo fact con gli stessi actor/input/result soltanto per ospitare il mapping. Questo e' precisamente il delta da verificare contro l'attuale `decisionRule` top-level dell'authority.

```
OWNER FACT
  produce(actor, input(s), result)

LOCAL STRUCTURE
  operatorStructure.decisionRule
    -> how the governed result value is selected
```

Se la regola deve avere provenance, review/change lifecycle, riuso o targetability realmente indipendenti dall'owner `produce`, la rilocazione locale non va forzata: il caso riapre l'identity/BA2 design e puo' falsificare questa candidate.

12.6 Invarianti

- actor, input e result sono quelli della proposition `produce`; la struttura non li duplica;
- `condition` e `decisionRule` possono coesistere: la prima governa quando la production si applica, la seconda quale valore del result viene selezionato una volta applicabile;
- ogni branch deve essere source-grounded; un branch mancante non viene inventato come ELSE;
- una residual condition come "gli altri casi HIGH" resta limitata allo scope governato e non diventa un default globale;
- una residual condition non viene riscritta automaticamente come complemento booleano o negazione del branch precedente: "other applicable HIGH cases" non autorizza `NOT(gt)` ne' un comparison key `le`;
- missing, invalid o out-of-domain restano non specificati quando la source non stabilisce che appartengano al residuale;
- `gt` e `lt` sono comparison stretti: il valore esattamente uguale alla soglia non soddisfa il comparison; cio' non definisce da solo un branch residuale;
- l'ordine dei branch non implica precedence a meno che la source non governi un ordine necessario;
- il mapping non modifica il dominio ammesso del result se tale dominio e' governato separatamente da `constrain`.

12.7 Forbidden inference

Non inferire completezza del mapping, default, precedence, short-circuit, gestione degli input mancanti, coercion dei tipi, range non documentati, error/failure handling, complementi booleani o comparison key ulteriori. Non trasformare valori locali come `P1` in BAReferent se non superano l'identity test BA1. L'eventuale *output branching* resta una pressure distinta: la candidate corrente seleziona valori di un result gia' governato e non crea output autonomi senza identity/downstream evidence separata.

12.8 Domande da porre alla documentazione

1. Esiste gia' un fatto `produce` completo con actor, input e result governati?
2. La source sta dicendo *quando la production vale* o *quale valore del result viene selezionato*?
3. Le condizioni del mapping sono esplicite e governate, oppure servirebbe inventare property o soglie?
4. I branch producono valori dello stesso result della proposition owner? Se sembrano output diversi, hanno davvero identita' e relazioni downstream autonome oppure sono valori dello stesso result?
5. Il dominio dei valori e' governato separatamente e richiede anche un `constrain`?
6. Esistono branch residuali espliciti ("altri casi", "otherwise")? Qual e' il loro scope esatto?
7. La source governa il predicato complementare del residuale oppure soltanto il residuale stesso? Se governa soltanto il residuale, non normalizzarlo in `NOT(...)` o in un comparison inverso.
8. I branch sono semanticamente disgiunti oppure servirebbe precedence/evaluation order non documentato?
9. Le comparison richieste sono rappresentabili con i key ammessi dalla candidate corrente (`eq`, `ne`, `gt`, `lt`)? Se no, registrare pressure invece di inventare operatori di confronto.
10. Il branch richiede la conformita' a un dominio gia' governato o l'occorrenza di un altro fatto governato? Se si', verificare se il significato puo' essere riusato senza duplicazione; in assenza di sintassi ammessa, registrare `GOVERNED_CRITERION_REUSE_PRESSURE`.
11. Il comportamento sulla soglia esatta, su input missing/invalid o fuori dominio e' governato? Se non lo e', lasciarlo `NOT SPECIFIED` e non usarlo per chiudere un branch residuale.
12. La regola deve essere riusata, revisionata o tracciata indipendentemente dall'owner `produce`? Se si', fermare la rilocazione locale e riaprire identity/BA2 review.
13. La source governa davvero un mapping di risultato oppure soltanto una dependency, restriction o classification?

Procedura minima

Estrarre prima `produce` senza rule; stabilizzare actor/input/result; separare eventuale `condition` di applicabilita'; isolare poi soltanto i branch che selezionano il result; preservare valori, comparison e residual wording governati; riconoscere separatamente eventuali criteri che riusano domini o fatti gia' governati senza convertirli in booleani o branch duplicati; testare soglia esatta e casi missing/invalid senza completarli per intuizione; infine confrontare la collocazione candidate annidata con l'attuale `decisionRule` top-level per assicurare che nessuna identita' o provenance necessaria venga persa.

12.9 Esempio generico minimo

Documentazione inventata ma coerente

"`QueueClassifier` produce `QueueClass` da `RequestClassification`. Se `classKind` e' `EXPRESS`, il risultato e' `FAST`; se `classKind` e' `STANDARD`, il risultato e' `NORMAL`."

```
produce
  actor  -> QueueClassifier
  input  -> RequestClassification
  result -> QueueClass
  polarity = affirmative
  operatorStructure
    decisionRule
      branch
        when -> RequestClassification.classKind eq EXPRESS
        resultAssignment -> QueueClass = FAST
      branch
        when -> RequestClassification.classKind eq STANDARD
        resultAssignment -> QueueClass = NORMAL
```

L'assenza di altri branch non autorizza a inventare un default.

12.10 Regression case A - DermaTriage priority

`FR-MR01-04-01` governa quattro selezioni e ha gia' superato il pressure test sui boundary case usati in review: soglia esatta, confidence missing/invalid, valori MEDIUM/LOW con confidence alta e scope image-based. La seguente resta una *candidate authoring view*, non BA gia' accettata:

```
produce
  actor  -> OperationalPriorityDerivation
  input  -> CaseUrgency
  input  -> TriageConfidence
  result -> OperationalTriagePriority
  scopedModifier
    condition -> LesionImage available
  operatorStructure
    decisionRule
      branch
        when -> CaseUrgency eq HIGH AND TriageConfidence gt 0.85
        resultAssignment -> OperationalTriagePriority = P1
      branch
        when -> other applicable cases within CaseUrgency eq HIGH
        resultAssignment -> OperationalTriagePriority = P2
      branch
        when -> CaseUrgency eq MEDIUM
        resultAssignment -> OperationalTriagePriority = P3
      branch
        when -> CaseUrgency eq LOW
        resultAssignment -> OperationalTriagePriority = P4
```

Il secondo branch preserva un residuale governato: non equivale a `TriageConfidence le 0.85`, non copre automaticamente missing/invalid e non e' un `ELSE` globale. `gt` e' l'unico comparison key nuovo richiesto da questo caso.

12.11 Regression case B - DermaTriage readiness C7

FR-C7-01-01 governa una qualificazione distinta dalla derivazione P-scale: la verifica usa il risultato osservato del health check per rendere disponibile il result di readiness. Le identity analitiche corrispondenti sono preservate nel case study come `ServiceHealthCheckResult` e `ReadinessOutcome`. Il fatto `produce` e' accettato nel case study; la struttura annidata resta candidate metodologica.

```
produce
  actor  -> ServiceReadinessVerification
  input  -> ServiceHealthCheckResult
  result -> ReadinessOutcome
  polarity = affirmative
  operatorStructure
    decisionRule
      branch
        when -> HTTP status eq 200 AND efficientnet eq true
                AND chromadb eq true AND b4_connection eq true
        resultAssignment -> ReadinessOutcome = PASS
```

Perche' non `condition`

I criteri non stabiliscono *quando* la proposition `produce` si applica: stabiliscono *quale valore* viene assegnato al suo result. La qualification deve restare applicabile anche quando la pass condition non e' soddisfatta. Una `condition -> criteria satisfied` renderebbe invece applicabile l'intera proposition soltanto nel caso positivo.

Branch complementare e output branching

Il caso governa il branch positivo `ReadinessOutcome = PASS`; non autorizza a inventare `ELSE -> FAIL`. Se documentazione successiva governera' un esito non-PASS, potra' essere aggiunto un branch source-grounded. Se emergessero invece due risultati con identita' e relazioni downstream autonome, riesaminare il confine result/value e registrare output-branching pressure senza promuoverlo per simmetria.

Valore di regression

Priority e readiness usano domini diversi ma preservano lo stesso discriminatore: `condition` governa applicability della proposition; `decisionRule` governa result selection all'interno di una `produce` gia' completa. Il secondo caso riduce il rischio che la rilocazione sia un adattamento specifico alla sola P-scale, ma non sostituisce la regression cross-corpus richiesta per l'admission.

12.12 Pressure case C - DermaTriage endpoint verification C7-P02

La review source-first di `C7-P02` separa `/analyze` e `/diagnose`. Entrambi mantengono il pattern `produce observation -> verification outcome`; la pressione nuova non riguarda la collocazione locale della `decisionRule`, ma la forma dei predicati usati nel branch positivo.

C7-P02 /analyze - comparison + governed-domain reuse

Il criterio documentato combina `response time < 30s` con "valid P-scale value". Il primo caso giustifica `lt` come comparison candidate source-driven; il secondo NON autorizza a duplicare `P1 | P2 | P3 | P4` come quattro equality branch.

```
produce
  actor  -> DirectAnalysisEndpointVerification
  input  -> DirectAnalysisEndpointObservation
  result -> DirectAnalysisEndpointVerificationOutcome
  operatorStructure
    decisionRule
      branch
        when -> responseTime lt 30s AND
                <valid OperationalTriagePriority in governed P1-P4 domain>
        resultAssignment -> DirectAnalysisEndpointVerificationOutcome = PASS
```

La seconda riga tra parentesi angolari e' **pressure notation**, non sintassi BA ammessa. Il dominio deve restare posseduto dal suo owner; la verification lo usa come criterio senza riscriverlo.

C7-P02 /diagnose - governed-fact occurrence reuse

Il criterio documentato combina HTTP `200` con l'avvenuto write-back del risultato in B4. `eq 200` e' rappresentabile; "result written to B4" non viene normalizzato come `b4Write=true` e non crea un secondo `transfer`.

```
produce
  actor  -> B4IntegratedDiagnosisEndpointVerification
  input  -> B4IntegratedDiagnosisEndpointObservation
  result -> B4IntegratedDiagnosisEndpointVerificationOutcome
  operatorStructure
    decisionRule
      branch
        when -> HTTP status eq 200 AND
                <governed B4 result-write occurrence criterion>
        resultAssignment -> B4IntegratedDiagnosisEndpointVerificationOutcome = PASS
```

Disposition metodologica della pressure

`lt` entra nella candidate comparison vocabulary perche' richiesto da una source reale. Il riuso di un dominio governato e il riuso dell'occorrenza di un governed fact restano invece `GOVERNED_CRITERION_REUSE_PRESSURE`: non viene introdotto un nuovo operator, un boolean DSL o una sintassi di reference prima di ulteriori casi/regression. Questi endpoint sono pressure cases coerenti con la rilocazione locale, non una promotion aggiuntiva di `decisionRule`.

12.13 Prompt LLM di supporto all'authoring

```
TASK: identify governed result-selection logic for an existing produce proposition.
Use ONLY supplied documentation, accepted BA and current BA authority.
This R7 nesting of decisionRule is CANDIDATE and does not replace current top-level
    ↪ decisionRule authority.

Rules:
1. Identify the complete produce proposition first: actor, input(s), result(s),
    ↪ polarity.
2. Separate proposition applicability from result selection.
3. Put applicability in scopedModifier.condition, never in operatorStructure.
    ↪ decisionRule.
4. Identify nested decisionRule only if the source governs how conditions/input
   values select a value of an existing produce result.
5. Reuse the produce actor/input/result. Do not duplicate them as a second fact.
6. Extract only source-governed branches and assignments.
7. Do not invent ELSE/default/completeness/precedence/evaluation order.
8. Preserve explicit residual wording such as "other HIGH cases" in its exact scope.
    ↪
   Do not rewrite it as NOT(previous condition) or as an inferred inverse
    ↪ comparison.
9. In this R7 candidate use eq/ne as normalized current semantics; gt and lt are
   candidate strict comparisons justified by governed DermaTriage cases.
   Any other comparison returns VOCABULARY_PRESSURE.
10. Do not infer handling for threshold equality, missing, invalid or out-of-domain
    inputs unless the source explicitly governs it.
11. Do not turn allowed-domain semantics into duplicated result-selection branches.
    If a pass criterion requires validity in an existing governed domain, return
    GOVERNED_CRITERION_REUSE_PRESSURE and preserve the criterion wording.
12. If a pass criterion requires that another governed fact occurred, do not invent
    a boolean or duplicate that fact; return GOVERNED_CRITERION_REUSE_PRESSURE.
13. If the rule needs independent identity/reuse/provenance/change lifecycle,
    return IDENTITY_PRESSURE and stop local nesting.
14. Do not invent BAReferents, properties, typed values, boolean DSL nodes or
    ↪ metadata.

Return EXACTLY the review wrapper and candidate BA shape shown next.
```

Human review remains mandatory

La review umana deve verificare tre confini distinti: `condition` contro result selection, residuale contro complemento inferito e local structure contro independent proposition identity. La presenza linguistica di "if", "otherwise" o di un simbolo di confronto non decide da sola nessuno di questi confini.

12.14 Output esatto atteso dall'estrazione assistita

```
RESULT_SELECTION_ASSESSMENT
  decision = ACCEPT_CANDIDATE | NOT_RESULT_SELECTION | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  ownerProduce = <candidate-id / identified governed fact>
  authorityStatus = CANDIDATE_RELOCATION_OF_DECISION_RULE
  comparisonStatus = CURRENT_EQ_NE | CANDIDATE_GT | CANDIDATE_LT |
    ↪ VOCABULARY_PRESSURE
  criterionReuseStatus = NONE | GOVERNED_DOMAIN_REUSE_PRESSURE |
    ↪ GOVERNED_FACT_REUSE_PRESSURE
  residualStatus = NONE | SOURCE_GROUNDED_RESIDUAL | AMBIGUOUS_SCOPE
  identityPressure = NONE | REVIEW_BA1_BA2
  notSpecified = [<only unresolved material items>]

BA_OUTPUT_CANDIDATE
  BAProposition <owner-id>
    semanticOperatorKey = produce
    <produce roles and terms>
    polarity = affirmative | negative
    scopedModifier [only if independently source-grounded]
      condition -> <applicability guard>
    operatorStructure
      decisionRule
        <source-grounded branches and result assignments>
```

I campi di assessment appartengono al wrapper di review, non alla BA. `CURRENT_EQ_NE` indica la normalizzazione delle semantiche storiche; `CANDIDATE_GT` e `CANDIDATE_LT` segnalano le sole comparison strette richieste dai casi governati correnti. `GOVERNED_DOMAIN_REUSE_PRESSURE` e `GOVERNED_FACT_REUSE_PRESSURE` non sono operatori o role BA: impediscono di duplicare un constraint/fact o di inventare un boolean mentre la forma di riuso e' ancora aperta. `SOURCE_GROUNDED_RESIDUAL` non autorizza a materializzare un complemento booleano.

12.15 Discriminazioni con i costrutti vicini

Confine	Discriminatore
decisionRule / condition	la rule seleziona il valore del result; condition cambia quando l'intera proposition si applica.
decisionRule / constrain	la rule governa la selezione runtime/operativa; constrain governa un dominio o restriction senza dire come un valore viene scelto.
decisionRule / produce	produce resta il fatto owner; la rule aggiunge dettaglio di selezione senza creare un secondo actor/input/result.
nested decisionRule / top-level decisionRule	e' lo stesso costrutto canonico: l'authority corrente lo rappresenta come operator top-level; la candidate R7 ne testa soltanto la rilocazione come `operatorStructure` dell'owner `produce`.
decisionRule / dependOn	la rule seleziona un risultato; dependOn dichiara prerequisite/dependency e non simula branch IF/THEN.

12.16 Rappresentazione Mermaid della operator structure

Quando una view renderizza `decisionRule`, usa un **tag ellittico locale della famiglia cromatica di `produce`**. La shape e' analoga a quella di `condition`, ma il colore separa la semantica: ambra = applicability; blu = result selection. Il tag non e' un BAReferent, non e' un secondo actor e non crea dataflow.

Semantica BA	Regola di projection candidate
`produce`	Pattern blu `input(s) -> actor -> result(s)`.
`condition`	Tag ellittico ambra: quando l'owner si applica.
`decisionRule`	Tag ellittico blu: come i branch selezionano il valore del result.
Branch / assignment	Se espansi, elementi locali blu; non BAReferent e non dataflow.
Visibility	Tag e branch possono essere omessi dal projection profile senza cambiare la BA.

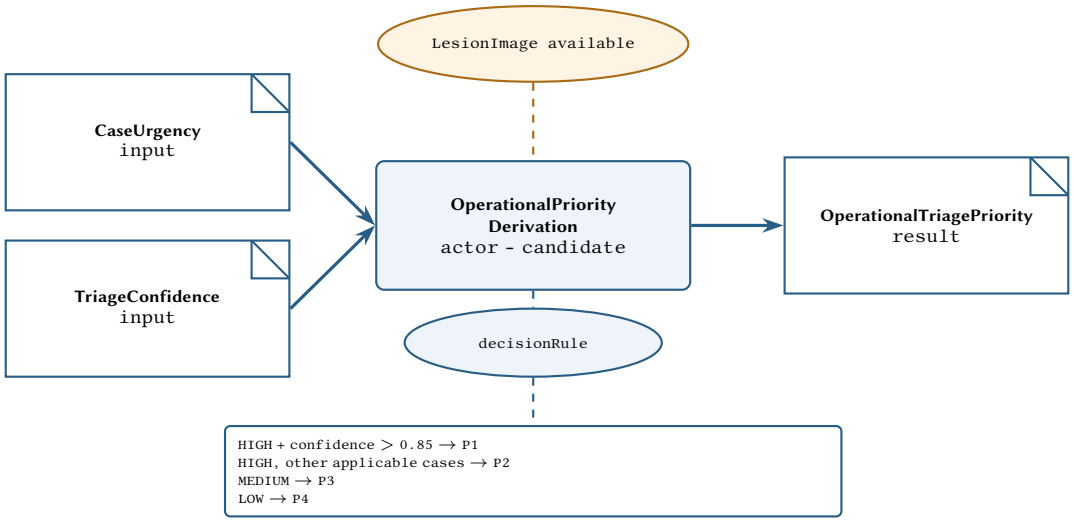
```
flowchart LR
  I["Input"] --> A["Actor"] --> R["Result"]
  C([condition]) -.- A
  DR([decisionRule]) -.- A
  DR -.- B1["branch 1: Result = value 1"]
  DR -.- B2["branch 2: Result = value 2"]
  classDef c fill:#FFF4E3,stroke:#A96A16
  classDef p fill:#EEF4F9,stroke:#275D86
  class C c
  class DR,B1,B2 p
```

Semantic determinism e omissione controllata

A parita' di BA e projection profile, la decisione di includere o omettere `operatorStructure` deve essere deterministica. Se inclusa, branch/label/assignment derivano dalla BA; se omessa, la BA resta invariata e la view dichiara l'omissione tramite il proprio coverage contract. Nessun modello "collapsed" viene creato. Un criterion-reuse ancora in pressure non viene reso graficamente come se esistesse gia' una relation BA formalizzata: la view puo' annotare la pressure, ma non deve fabbricare un edge semantico.

12.17 Esempio grafico corrispondente

La figura usa il `produce` del regression case priority. `condition` e `decisionRule` condividono la forma ellittica ma non il colore: ambra per applicability, blu per result selection. I branch restano dettaglio locale e non BAReferent.



Lettura della candidate view

Blu = produzione e result selection locale; ambra = applicability. Il tag blu `decisionRule` e il box dei branch sono projection della stessa struttura locale, non dataflow. Il branch "HIGH, other applicable cases" resta un residuale source-grounded e non un complemento `le 0.85`. La figura corregge la projection senza promuovere la candidate.

13 Semantic kind `store`

CURRENT / PRESERVED

13.1 Che cosa rappresenta

`Store` e' la lettura umana del semantic kind canonico `store`. Non introduce una terza famiglia BA1 e non e' un nuovo operatore BA2: `semanticKind=store` descrive direttamente la natura di un `BAReferent` nel working model corrente. La source governa una risorsa nella quale informazioni o artifact possono rimanere disponibili oltre la singola operazione che li ha collocati o consultati, senza che la BA debba conoscere la tecnologia concreta usata per conservarli o raggiungerli.

Smallest semantic fact

Esiste una risorsa governata con identita' analitica indipendente che funge da luogo/fonte/destinazione di contenuto mantenuto disponibile oltre l'operazione corrente. Eliminando il semantic kind `store` si perde la distinzione tra tale risorsa e un normale process endpoint, service endpoint o contenuto in transito.

13.2 Quando usarlo

Usare `store` quando la documentazione governa una risorsa identificabile in cui dati, informazioni, record, versioni o artifact vengono conservati, mantenuti, archiviati, salvati, cached o resi disponibili per uso successivo. Le parole sono soltanto indizi: il semantic kind deriva dal significato source-supported e dall'identity test BA1.

13.3 Quando non usarlo

Non assegnare `semanticKind=store` a un elemento soltanto perche' e' un file path, una tabella, un bucket, un database product, un repository nominale o un endpoint API. Un endpoint puo' esporre informazione calcolata o transiente; un path puo' essere soltanto una configurazione. La tecnologia corrente non decide la categoria BA.

Incremento R7 sotto test - binding opzionali

Il semantic kind `store` resta la distinzione preservata. Questa R7 candidate testa due semplificazioni complementari: il kind e' registrato direttamente sul `BAReferent` e l'authoring shape opzionale `binding(locator,direction)` arricchisce lo stesso Store. Non vengono creati `StoreBinding`, nuovi BAReferent o nuovi operatori. Il binding profile e la responsabilita' residua di `classify` restano sotto regression prima di una futura promotion metodologica.

13.4 Forma canonica BA e profilo authoring

Nel working model R7 il semantic kind e' parte della descrizione analitica del `BAReferent`; non viene creata una `BAProposition` `classify` soltanto per assegnare `store`. Il profilo `Store` presenta la stessa identita' insieme agli eventuali binding concreti senza creare una nuova identity family.

```
BA CORE / STORE PROFILE
BAReferent <store-id>
  canonicalName = <project canonical name>
  semanticKind  -> store

  binding [0..*]
    locator   -> <opaque source-supported literal> [1]
    direction -> IN | OUT [0..2, unique]
```

Elemento	Card.	Significato
Store referent	1	`BAReferent` che supera l'identity test e conserva il semantic kind canonico `store`.
`semanticKind`	1 per uno Store accettato	Chiave controllata methodology-neutral del `BAReferent`. Non crea una terza famiglia BA1 e non richiede una proposition di typing separata nel working model.
`binding`	0..*	Enrichment opzionale dello stesso Store. Uno Store senza binding concreto resta pienamente valido.
`locator`	1 per binding	Literal opaco source-supported: path, endpoint, URI, table/bucket identifier o altra forma concreta. Non crea una tassonomia tecnologica.
`direction`	0..2 uniche	Direzione relativa allo Store: `IN` = contenuto entra nello Store; `OUT` = contenuto esce dallo Store. Omettere se la direzione non e' governata.

Un solo costrutto, enrichment progressivo

`semanticKind` e `binding` descrivono lo stesso BAReferent: non introducono una proposition di typing ne' un secondo costrutto. MR, Decision e FR possono riusare la stessa identita' Store e aggiungere binding mancanti o direzioni ulteriormente governate. Provenance/review restano, per ora, alla granularita' del BAReferent e l'enrichment non autorizza duplicati.

13.5 Identity continuity e invarianti

- il canonical semantic-kind key resta `store`; il display `Store` non crea un alias normativo;
- `semanticKind=store` e' registrato sul BAReferent; non materializzare `classify` soltanto per il typing ordinario;
- provenance e review del kind seguono, in questo working model, il BAReferent nel suo insieme; un lifecycle field-level verra' riaperto solo se un pressure case reale lo richiede;
- uno Store e' sempre un `BAReferent` riusabile: un dettaglio di binding nuovo non crea una nuova identita' se il significato della risorsa e' invariato;
- `binding` e' opzionale: assenza di locator non nega l'esistenza dello Store;
- ogni locator resta opaco alla BA; filesystem, SMB, API, database e object storage non diventano sottotipi BA;
- `direction` e' sempre relativa allo Store: `IN` significa ingresso nello Store, `OUT` uscita dallo Store;
- un binding puo' avere soltanto locator quando la source governa la realizzazione/access location ma non la direzione;
- lo stesso locator puo' supportare sia `IN` sia `OUT`; locator diversi possono supportare direzioni diverse;
- `transfer` verso uno Store e' la rappresentazione corrente sufficiente di un comportamento governato di save/write; `transfer` dallo Store e' la rappresentazione corrente sufficiente di load/read;
- se la source governa soltanto che un contenuto e' mantenuto in una risorsa, non inventare un `transfer`: riconoscere lo Store e preservare soltanto i binding effettivamente supportati;
- enrichment documentale: campo mancante + nuova evidence compatibile → completamento; evidence compatibile aggiuntiva → enrichment; evidence materialmente diversa → review, non overwrite silenzioso.

13.6 Forbidden inference

Da `store` non inferire permanenza illimitata, retention minima/massima, ownership, versioning, deletion policy, replica, consistency, encryption, localita', protocollo, access control o lifecycle. Un file temporaneo puo' essere Store se la source governa la sua funzione di mantenere contenuto disponibile per un uso successivo. Eventuale retention temporale e' semantica separata e non viene inserita nella definizione minima.

No synthetic persistence relation

Questa R7 non ammette `storedIn`. Finche' `Store + transfer` preserva i fatti governati necessari, non si aggiunge una relazione at-rest per rendere il grafo piu' completo. Un counterexample futuro puo' riaprire il tema.

13.7 Domande da porre alla documentazione

1. La source governa una risorsa nella quale informazioni o artifact possono rimanere disponibili oltre la singola operazione corrente?
2. Tale risorsa deve avere identita' analitica indipendente e poter essere riusata come endpoint o target in piu' proposition/viste?
3. Sto riconoscendo la risorsa logica oppure sto promuovendo automaticamente un path, endpoint, table o prodotto tecnologico?
4. Se compare un endpoint API, la source governa realmente una risorsa mantenuta dietro quell'accesso oppure soltanto una risposta calcolata/transiente/service-only?
5. Esiste gia' lo stesso Store nel registry BA e deve essere riusato?
6. La source governa un locator concreto? Se si', conservarlo come binding opaco senza dedurne tecnologia ulteriore.
7. La direzione rispetto allo Store e' esplicita? `IN`, `OUT`, entrambe, oppure `NOT SPECIFIED`?
8. Un livello documentale downstream aggiunge un binding o una direzione allo stesso Store? Se si', arricchire l'identita' esistente invece di creare un nuovo costrutto.
9. Un FR aggiunge soltanto campi dello Store oppure governa anche un comportamento operativo coerente? Il field completion puo' essere evidence di downstream utility, ma non giustifica da solo un FR.
10. La source governa davvero un movimento di content da/verso lo Store? Solo allora costruire la corrispondente proposition `transfer`.
11. Esistono binding incompatibili tra fonti/livelli? In tal caso registrare review/conflict invece di scegliere il piu' concreto.
12. Retention, expiry, replacement o deletion sono governati? Se si', preservarli come semantica separata; non incorporarli implicitamente in `store`.

Procedura minima

Ricostruire prima il significato della risorsa; applicare BA1; assegnare al BAReferent `semanticKind=store`; riusare la stessa identita' cross-level; raccogliere locator source-supported; assegnare `IN/OUT` soltanto quando la direzione e' governata; costruire eventuali `transfer` separatamente; infine verificare che MR/Decision/FR abbiano aggiunto significato senza duplicare Store o trasformare il completamento di un campo nell'unica ragione di esistenza di un branch.

13.8 Esempio generico minimo

Documentazione inventata ma coerente

"Il sistema conserva i `ReportRecord` nel `ReportStore`. La registrazione avviene tramite `POST /reports`; il recupero tramite `GET /reports/{id}`."

Assumendo che l'identita' superi BA1, l'authoring profile e':

```
Store ReportStore
  binding
    locator   -> "POST /reports"
    direction -> IN
  binding
    locator   -> "GET /reports/{id}"
    direction -> OUT
```

La BA materializza direttamente `ReportStore` come BAReferent con `semanticKind=store`. I transfer di save/read vengono aggiunti solo se source, content e destination/source necessari sono governati.

13.9 Pressure case DermaTriage RC-001

La source corrente preservata stabilisce che le versioni del classificatore sono mantenute nel path `models/versions/`. Il tracking usa `db/model_versions.json`; il modello attivo usa `models/efficientnet_b4.pth`. Per il primo elemento la working extraction puo' riconoscere:

```
Store ClassifierVersionStore
  binding
    locator -> "models/versions/"
    # direction omitted: access direction not yet established by this fact alone
```

Cosa resta da decidere nel rerun RC-001

La sezione Store non chiude automaticamente se `ClassifierVersionRegistry` e `ActiveClassifierModelLocation` siano Store distinti ne' quale transfer realizzi il restore. Tali identita' vengono riesaminate source-first dopo C0.1 e, per il version choice, dopo `selection`. Il meccanismo copy/move/overwrite/load/reference-switch resta `NOT SPECIFIED`.

13.10 Prompt LLM di supporto all'authoring

```
TASK: identify governed BAReferents that qualify for semanticKind store,
and preserve optional source-supported Store bindings.
Use ONLY supplied documentation, existing BA registry and current BA authority.

Rules:
1. Apply the normal BA1 identity test before creating a new BAReferent.
2. Accept store only when the source supports an independently identifiable
   resource in which information/artifacts can remain available beyond the
   current operation.
3. Reuse an existing Store BAReferent when semantic identity is unchanged.
4. Record semanticKind = store directly on that BAReferent. Do not create
   a classify proposition solely to perform ordinary semantic typing.
5. Do not infer Store from a filesystem path, database/table/bucket,
   repository word or API endpoint alone.
6. Preserve a concrete access/storage locator only when source-supported.
7. Keep locator opaque; do not invent a technology taxonomy.
8. direction is relative to the Store: IN enters Store; OUT leaves Store.
9. Add IN/OUT only when direction is governed. Locator-only is valid.
10. MR/Decision/FR may enrich the SAME Store with additional supported fields.
   Do not create a duplicate Store merely because a lower level is more concrete.
11. A lower-level field completion is not, by itself, evidence that a new FR
    or Decision must exist; preserve the independent behavioral/governance gate.
12. Do not infer transfer from Store existence or locator presence.
13. When movement is governed, use current transfer semantics; do not invent
    read, write or storedIn operators.
14. Do not infer retention duration, permanence, deletion, versioning,
    ownership, consistency, encryption, protocol or lifecycle.
15. Conflicting bindings trigger REVIEW; never silently overwrite one source.
16. If retained-availability meaning is missing, return NOT_A_STORE or
    INSUFFICIENT_EVIDENCE rather than completing the source.

Return EXACTLY the review wrapper and authoring output shape shown next.
```

Human review remains mandatory

La review deve verificare identita' Store, semantic kind, locator e direzione, ma nel working model non apre un lifecycle separato per ciascun campo. Un output completo dell'LLM non sana una source che governa soltanto un endpoint o un path senza significato di storage.

13.11 Output esatto atteso dall'estrazione assistita

Il wrapper integra semantic kind ed enrichment opzionale nello stesso BAReferent. Il blocco mostrato sotto non e' una nuova famiglia BA e non genera una proposition di typing separata.

```
STORE_ASSESSMENT
  decision = ACCEPT_STORE | NOT_A_STORE | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  identity = REUSE | NEW_CANDIDATE
  bindingStatus = NONE | LOCATOR_ONLY | DIRECTED | MIXED | REVIEW_CONFLICT
  notSpecified = [<only unresolved material items>]

BA_OUTPUT
  BAReferent <canonical store referent name>
    semanticKind -> store

    binding [0..*]
      locator   -> <opaque literal>
      direction -> IN | OUT  # repeat 0..2 only if source-supported
```

13.12 Discriminazioni con i costrutti vicini

Confine	Discriminatore
Store / Information	lo Store e' la risorsa che mantiene disponibilita'; il content/artifact conservato mantiene la propria identita'.
Store / transfer	Store identifica un endpoint semantico riusabile; transfer governa una conveyance specifica. Store non crea da solo movimento.
Store / realize	un locator embedded preserva un binding opaco; usare `realize` quando una realization necessita propria identita' BA e relazione interrogabile con l'abstract.
Store / consumeService	usare un'API/service capability non dimostra che il service sia Store; l'accesso service e la risorsa conservata restano separabili.
Store / observe	read/query/retrieve possono richiedere review `observe`; questa sezione non risolve C0.3 e usa transfer quando la source governa conveyance.
Store / selection	Store puo' fornire una source population; `selection` governa quale item/subset viene scelto, non dove e' conservato.
Store / storedIn	`storedIn` resta deferito. Va riaperto solo se un fatto at-rest governato non e' preservabile con Store, transfer e documentazione senza inventare operazioni.
semanticKind / classify	`semanticKind` descrive la natura analitica del BAReferent. Questa R7 non richiede `classify` per il typing ordinario; `classify` resta in review per eventuali classification assertion indipendenti dimostrate da futuri pressure case.
Binding / BA fact	un binding completa la descrizione dello stesso Store; non e' automaticamente una nuova `BAProposition` ne' un motivo sufficiente per creare un FR.

13.13 Rappresentazione Mermaid di Store

Il semantic kind `store` pilota la forma del nodo: un cilindro rende immediatamente riconoscibile la risorsa mantenuta. La shape e' projection semantics derivata dal semantic kind; non crea un nuovo tipo BA. Il role assunto dallo Store in una proposition determina invece posizione/direzione del grafo e non modifica la shape.

Semantica	Regola di projection
`semanticKind=store`	Nodo cilindrico con nome canonico del `BAReferent`.
Store destination	Se lo Store partecipa come `destination`: source → content → cilindro; lettura umana write/save.
Store source	Se lo Store partecipa come `source`: cilindro → content → destination; lettura umana read/load.
Direction `IN`	Compatibile con accesso verso lo Store; non genera da solo una proposition transfer.
Direction `OUT`	Compatibile con accesso dallo Store; non genera da solo una proposition transfer.
Locator	Se la view lo include, compare come annotazione grigia/tratteggiata del cilindro; non come nuovo BA node.

```
flowchart LR
  P["ReportGeneration<br/>source"]
  C@{ shape: doc, label: "ReportRecord<br/>content" }
  S[("ReportStore<br/>store / destination")]
  B["IN: POST /reports"]

  P --> C --> S
  B -.- S

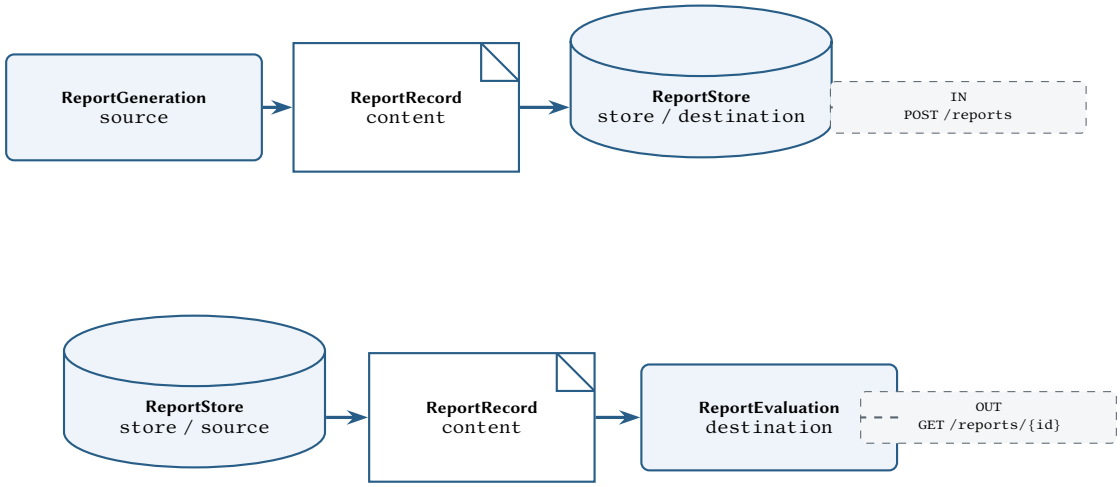
  classDef endpoint fill:#EEF4F9,stroke:#275D86,color:#17324D
  classDef content fill:#FFFFFF,stroke:#275D86,color:#17324D
  classDef store fill:#EEF4F9,stroke:#275D86,color:#17324D
  classDef binding fill:#F4F6F8,stroke:#5B6770,color:#17324D
  class P endpoint
  class C content
  class S store
  class B binding
```

No reverse inference dalla projection

Il cilindro deriva da `semanticKind=store`. La presenza di un cilindro o di una label IN/OUT in una vista non autorizza a creare Store, binding o transfer che non siano gia' source-supported nella BA/authoring evidence.

13.14 Esempio grafico corrispondente

Le due righe mostrano lo stesso semantic kind in due proposition transfer indipendenti. IN/OUT sono direzioni dei binding relative allo Store; le frecce derivano invece dai role source/destination di ciascun transfer.



Lettura del grafico

Il cilindro segnala il semantic kind `store`; non significa database. Le due occorrenze grafiche di `ReportStore` sono la stessa identita' riusata in due proposition distinte. IN/OUT descrivono i locator source-supported; i transfer restano i fatti BA che governano la conveyance. Una view puo' omettere le annotazioni di binding mantenendo invariati Store e transfer.

14 Reusable local structure `selection`

REUSABLE LOCAL / NON-NORMATIVE

14.1 Che cosa rappresenta

`selection` preserva il significato governato secondo cui, all'interno di una proposition `produce`, uno o piu' candidate disponibili da una candidate source governata vengono scelti come membro, elemento o subset del result. La candidate source puo' coincidere con un input dell'owner oppure essere una source semanticamente distinta, come uno Store che mantiene disponibili i candidate. Non e' un nuovo `semanticOperatorKey`: il fatto owner resta `produce`; `selection` esplicita soltanto come il result e' determinato a partire da candidate gia' disponibili.

Smallest semantic fact

Data una source population o candidate source governata e un result governato, esiste una distinzione materialmente utile tra i candidate disponibili e quelli che diventano il selected result. Rimuovendo `selection`, la BA conserva eventualmente che un result viene prodotto ma perde *quali candidate possono entrarvi e secondo quale base governata.*

14.2 Quando usarla

Usare la struttura quando la source governa membership/subset, bounded choice, recent-N, top-K, best/argmax-like choice o altra scelta di candidate e questa semantica non e' ricostruibile dal solo `produce`. La struttura puo' descrivere filter-only selection, ranking-based selection o bounded selection senza assumere che tali forme coincidano.

14.3 Quando non usarla

Non usare `selection` per una semplice condition di applicabilita', per un allowed domain statico, per un transfer, per una query/read, per un output gia' determinato senza candidate population o per completare un algoritmo non documentato. La presenza lessicale di "select", "top", "recent", "best" o "filter" e' soltanto un indizio.

Stato nella Rebuild R7

La firma viene formalizzata qui come reusable local / second-level structure di `produce`. Il top-level inventory BA2 resta invariato e nessuna promotion normativa e' implicata da questa integrazione nella guida candidate.

14.4 Forma canonica locale

La struttura vive dentro l'owner `produce`; non duplica actor, input o result.

```
BAProposition <owner-id>
  semanticOperatorKey = produce
  actor  -> BAReferent [1]
  input  -> BAReferent [0..*]
  result -> BAReferent [1..*]

  operatorStructure
    selection
      candidateSource
        -> BAReferent [1..*]
      selectedResult
        -> owner.result [1]
      membershipCriteria
        -> <controlled local criterion | BAReferent> [0..*]
      selectionBasis
        -> <controlled local basis | BAReferent> [0..*]
      bound
        -> <positive integer | governed parameter> [0..1]
```

Elemento	Card.	Significato
`candidateSource`	1..*	Uno o piu' BAReferent che identificano la population/-source governata dei candidate. Possono coincidere con input dell'owner, ma uno Store puo' essere candidate source senza essere duplicato come `produce.input`: accesso e conveyance restano fatti separati. Non creare collection artificiali per completare la firma.
`selectedResult`	1	Uno dei result dell'owner `produce` la cui membership/choice viene spiegata dalla struttura.
`membershipCriteria`	0..*	Criteri di eleggibilita'/membership. Possono restare local structured values; diventano BAReferent solo se BA1 richiede identita' autonoma.
`selectionBasis`	0..*	Base usata per scegliere tra candidate eleggibili: recency, similarity, score, best/maximum o altra semantica source-supported. Non implica output ordering.
`bound`	0..1	Limite superiore governato alla cardinalita' selezionata, espresso come valore positivo o parametro governato. Non implica cardinalita' esatta o underfill behavior.

Minimum sufficient local structure

I campi opzionali si materializzano soltanto quando aggiungono significato governato. Se la source afferma una selection ma lascia ignoto il criterio esatto, la struttura puo' preservare candidate source e selected result lasciando gli slot mancanti non specificati invece di inventarli.

14.5 Invarianti

- `produce` resta l'owner del fatto: actor/input/result non vengono duplicati dentro `selection`;
- `candidateSource` deve riferire una source/population realmente governata; puo' coincidere con un input dell'owner oppure essere una source distinta, incluso uno Store; non duplicare lo Store come `produce.input` soltanto per soddisfare la firma e non promuovere una "population" sintetica;
- `selectedResult` e' un result dell'owner e non una seconda produzione;
- `membershipCriteria` governa chi puo' appartenere al selected result; `selectionBasis` governa come scegliere tra candidate eleggibili;
- filter implementation e selected-membership semantics non sono la stessa cosa: la struttura preserva il risultato della scelta, non impone un algoritmo di filtering;
- `bound=N` significa limite governato della selection; non prova che N elementi esistano sempre ne' che il result debba contenerne esattamente N;
- recent-N e top-K non implicano automaticamente ordine del result; recency/similarity possono essere basi di scelta senza essere un output-order contract;
- best/argmax-like choice non implica una full ranking materializzata o riusabile di tutti i candidate;
- uno Store puo' essere `candidateSource`; cio' non crea da solo read/query, retrieval, transfer o una relazione at-rest;
- un data/content contract puo' restringere candidate, request o selected result; il contract non seleziona membership e `selection` non definisce schema/field admissibility.

14.6 Forbidden inference

Da `selection` non inferire tie-break, stabilita' dell'ordinamento, deduplication, underfill, retry, pagination, overlap, reuse cross-cycle, sampling randomness, exhaustive search, persistence, access path, query protocol, successful retrieval o transfer. Non inferire inoltre che un field assente da un contract sia proibito: l'eventuale closure del data contract appartiene a PR-07 e deve essere governata separatamente.

No output-order field per convenienza

Questa firma non introduce `outputOrdering`. Se la source governa l'ordine materiale del result, conservarlo nel minimo owning construct/contract appropriato invece di confonderlo con la base usata per scegliere membership.

14.7 Domande da porre alla documentazione

1. Esiste gia' un fatto `produce` source-supported che rende disponibile il selected result?
2. Quale BAReferent identifica la candidate source/population da cui si sceglie? Coincide realmente con un input dell'owner oppure e' una source distinta, per esempio uno Store?
3. Il result e' davvero un elemento/subset scelto dalla candidate source oppure e' un output calcolato con altra semantica?
4. La source governa criteri di eleggibilita'/membership? Quali, senza tradurli in un allowed domain statico?
5. La source governa una base di scelta tra candidate eleggibili: recency, similarity, score, maximum/-minimum, priority o altro?
6. Esiste un bound governato? E' un massimo/selection cap oppure la source governa realmente una cardinalita' esatta?
7. "Most recent", "top", "best" o "highest" selezionano membership o governano anche l'ordine del result? Non assumere il secondo fatto.
8. Meno candidate del bound, tie, duplicati, overlap, reset o reuse sono governati oppure restano `NOT SPECIFIED`?
9. La candidate source e' uno Store? Se si', riconoscere lo Store separatamente senza inventare access/retrieval.
10. Esiste un request/content/result contract? Se si', il contract governa forma/ammissibilita' del content; non sostituisce la selection.
11. La source descrive una decision rule su valori alternativi dello stesso result invece di membership/-subset? In tal caso riesaminare `decisionRule`.
12. Togliendo `selection`, quale significato governato non sarebbe piu' ricostruibile dal solo `produce`?

Procedura minima

Identificare prima l'owner `produce`; riusare i suoi result e gli input realmente governati; scegliere la candidate source senza creare collection sintetiche e senza forzarla dentro `produce.input` quando e' una source distinta; estrarre separatamente membership criteria, selection basis e bound; lasciare espliciti i gap; verificare infine i confini con Store, contract, retrieval, transfer, `condition`, `constrain` e `decisionRule`.

14.8 Esempio generico minimo

Documentazione inventata ma coerente

"Il sistema costruisce `SelectedReports` scegliendo al massimo 10 `ReportRecord` pertinenti e privilegiando i piu' recenti."

```
produce
  actor  -> ReportSelectionCapability
  input  -> ReportRecord
  result -> SelectedReports

  operatorStructure
    selection
      candidateSource     -> ReportRecord
      selectedResult      -> SelectedReports
      membershipCriteria  -> pertinent
      selectionBasis      -> mostRecent
      bound               -> 10
```

L'esempio non afferma che l'output sia ordinato per data, che 10 elementi esistano sempre o che i record siano letti da uno Store.

14.9 Positive control corrente - DermaTriage FR-06

FR-06 governa la costruzione dell'evidence set del ciclo prompt da una finestra limitata delle correzioni cliniche pertinenti piu' recenti. La local structure gia' sperimentata nel case study puo' essere normalizzata cosi':

```
produce
  actor  -> DermaTriage
  input  -> ClinicalReviewResult
  result -> PromptEvolutionEvidenceSet

  operatorStructure
    selection
      candidateSource     -> ClinicalReviewResult
      selectedResult      -> PromptEvolutionEvidenceSet
      membershipCriteria  -> correction
      membershipCriteria  -> pertinence
      selectionBasis      -> mostRecent
      bound               -> 20
```

"Most recent" governa la membership selezionata; non prova output ordering. Underfill, deduplication, reuse, overlap e scope globale/per-capability restano gap documentali.

14.10 Positive control - top-K similarity

La pressure evidence DermaTriage per il retrieval storico separa candidate source, riferimento di query, similarity basis, bound e selected result. Il query input resta input dell'owner ma non e' candidate source.

```
produce
  actor  -> HistoricalCaseSimilarityRetrieval
  input  -> ClinicalDescriptionEmbedding
  input  -> HistoricalCaseVectorIndex
  result -> RetrievedHistoricalCases

  operatorStructure
    selection
      candidateSource -> HistoricalCaseVectorIndex
      selectedResult  -> RetrievedHistoricalCases
      selectionBasis  -> mostSimilarTo(ClinicalDescriptionEmbedding,
                                       metric=CosineSimilarity)
      bound            -> 5
```

La forma preserva top-5 membership senza inferire che `RetrievedHistoricalCases` debba essere materialmente ordinato per similarity o che il comportamento underfill sia definito.

14.11 Positive control - best checkpoint

La forma best/argmax-like richiede ancora meno struttura:

```
produce
  actor  -> BaselineTraining
  input  -> TrainingCheckpoints
  result -> BestBaselineCheckpoint

  operatorStructure
    selection
      candidateSource -> TrainingCheckpoints
      selectedResult  -> BestBaselineCheckpoint
      selectionBasis  -> maximum ValidationMacroF1
```

Il fatto che il best sia scelto secondo Validation Macro F1 non implica una full ranking persistita, un top-K, un bound aggiuntivo o uno Store dei checkpoint.

Tre forme, una sola local structure

Recent-N, top-K e best condividono candidate source + selected result ma non richiedono gli stessi campi opzionali. La reusable structure evita di creare operatori distinti `recentN`, `topK`, `rank` o `argmax`.

14.12 Composizione con Store e data/content contract

I costrutti descrivono assi distinti e possono comporsi senza fondersi:

```
Store CandidateStore

constrain
  constraintTarget -> SelectionRequest
  constraintValue  -> SelectionRequestContract

produce
  actor  -> RetrievalCapability
  input  -> SelectionRequest
  result -> SelectedItems

  operatorStructure
    selection
      candidateSource -> CandidateStore
      selectedResult  -> SelectedItems
      <criteria/basis/bound only if governed>

constrain
  constraintTarget -> SelectedItems
  constraintValue  -> SelectedItemsContract
```

Lo Store dice dove i candidate restano disponibili ed entra nella local structure come `candidateSource`; non deve essere duplicato come `produce.input` soltanto per rendere possibile la selection. `selection` dice quali candidate diventano result; il contract governa forma/ammissibilita' del request o del content; eventuali accessi e `transfer` governano separatamente la conveyance. Se un field come `age` non compare in un contract, questa sezione non conclude che sia forbidden: la closure del field set e' tema PR-07 separato.

14.13 Pressure case RC-001 - version choice

Il checkpoint RC-001 richiede una previous acceptable classifier version/state ma non governa ancora come scegliere tra piu' versioni accettabili. Con lo Store gia' riconosciuto:

```
produce
  actor  -> ??
  input  -> ModelVersionTrackingArtifact
  result -> PreviousAcceptableVersionOrState

  operatorStructure
    selection
      candidateSource    -> ClassifierVersionStore
      selectedResult     -> PreviousAcceptableVersionOrState
      membershipCriteria -> acceptable
      selectionBasis     -> ModelVersionTrackingArtifact
```

`ClassifierVersionStore` e' la candidate source ma non viene duplicato come `produce.input`; il content reso disponibile al processo e la relativa conveyance vengono modellati separatamente. Non vengono inventati immediate predecessor, latest-known-good, version ordering, query protocol, copy/move/overwrite/load o reference-switch. C0.3 retrieval/access decide quando il retrieval richiede un solo transfer di risposta e quando una request governata richiede una coppia request/response.

14.14 Prompt LLM di supporto all'authoring

```
TASK: identify governed local selection structure for an existing produce fact.
Use ONLY supplied documentation, current BA registry and current BA authority.

A selection exists only when the source supports a material distinction between
candidate(s) available to produce and the member/item/subset that becomes one
of its results.

Rules:
1. Identify/reuse the owner produce proposition first.
2. Reuse owner result roles and only the owner inputs that are independently
   ↪ governed.
3. candidateSource must point to one or more governed BAReferents that identify
   the candidate population/source. It MAY be an owner input or a distinct Store.
4. If candidateSource is a distinct Store, do NOT add that Store as owner.input
   merely to satisfy the selection signature; access/conveyance is reviewed
   ↪ separately.
5. Do not invent a collection BAReferent merely to satisfy candidateSource.
6. selectedResult must point to exactly one owner result.
7. Extract membershipCriteria only for source-supported eligibility/membership.
8. Extract selectionBasis only for source-supported choice among candidates.
9. Extract bound only when a governed maximum/selection cap exists.
10. bound does not imply exact cardinality, underfill, tie-break or retry behavior.
11. recent-N/top-K/best do not imply output ordering or full ranking.
12. Store may be candidateSource, but selection does not imply read/query,
    retrieval, transfer, persistence or storedIn.
13. A contract may constrain request/candidate/result shape; it does not select
    membership. Do not infer closed-field semantics from omission.
14. Do not convert condition, constrain, decisionRule or runtime validation into
    selection merely because the prose uses select/filter/best/top/recent.
15. Preserve NOT SPECIFIED for tie, deduplication, reuse, overlap, ordering,
    pagination, sampling and other unsupported details.
16. If removing selection loses no governed membership/choice meaning beyond
    produce, return NOT_SELECTION.

Return EXACTLY the review wrapper and BA local-structure output shown next.
```

Human review remains mandatory

La review deve verificare soprattutto owner produce, candidate source, distinzione membership/-basis e negative inference. Un output sintatticamente completo non sana una source che governa soltanto un result o una query senza selected-membership semantics.

14.15 Output esatto atteso dall'estrazione assistita

```
SELECTION_ASSESSMENT
  decision = ACCEPT_LOCAL_SELECTION | NOT_SELECTION | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  ownerProduce = <existing or candidate produce-id>
  candidateSource = [<governed candidate source BAReferent 1..*>]
  selectedResult = <owner result>
  membershipStatus = NONE | GROUNDED | PARTIAL | NOT_SPECIFIED
  basisStatus      = NONE | GROUNDED | PARTIAL | NOT_SPECIFIED
  boundStatus      = NONE | GROUNDED | NOT_SPECIFIED
  notSpecified = [<only unresolved material items>]

BA_LOCAL_STRUCTURE_R7
  BAProposition <owner-id>
    semanticOperatorKey = produce
    <existing produce roles>
    operatorStructure
      selection
        candidateSource -> <BAReferent> [1..*]
        selectedResult  -> <owner.result> [1]
        membershipCriteria -> <...> [only if grounded]
        selectionBasis     -> <...> [only if grounded]
        bound              -> <...> [only if grounded]
```

I campi di assessment appartengono al wrapper di review e non alla BA.

14.16 Discriminazioni con costrutti vicini

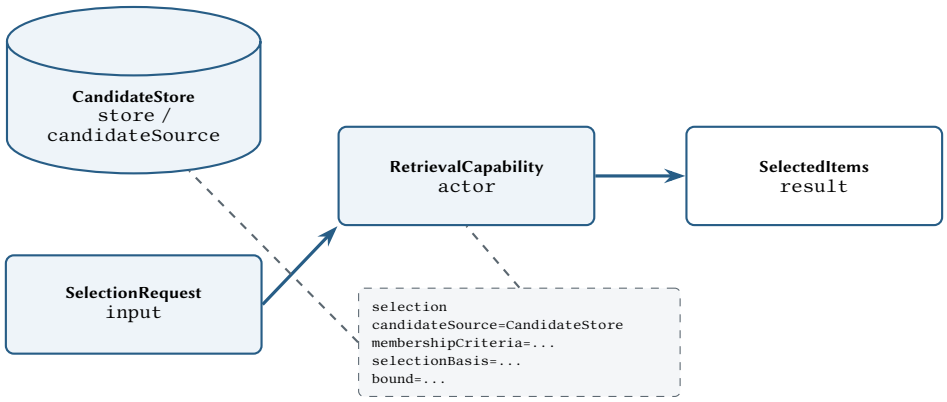
Confine	Discriminatore
selection / produce	`produce` rende disponibile il result; `selection` spiega quali candidate diventano quel result.
selection / decisionRule	`selection` sceglie membership/item/subset da candidate source; `decisionRule` governa branch/assignment di un result.
selection / constrain	`constrain` restringe un dominio/meaning; non decide quale runtime candidate entra nel selected result.
selection / Store	Store mantiene candidate disponibili; selection sceglie tra essi. Nessuna read/query e' implicata.
selection / observe-retrieval	access/query puo' rendere dati disponibili; non stabilisce da solo la selected membership.
selection / transfer	selection non implica conveyance; il selected result puo' essere trasferito soltanto con un fatto separato.
selection / contract	contract governa shape/ammissibilita' del content; selection governa membership/choice.
selection / ranking-filtering	ranking/filtering possono contribuire a basis/criteria; non sono automaticamente operatori BA indipendenti.

14.17 Projection della local structure

Come per `decisionRule`, una view che mostra `selection` usa un'annotazione grigia tratteggiata associata all'owner `produce`. La struttura non e' un BAReferent, non crea un nuovo dataflow edge e puo' essere omessa da projection che non richiedono il dettaglio di membership.

```
flowchart LR
  S[("CandidateStore\nstore / candidateSource")]
  Q["SelectionRequest\ninput"]
  A["RetrievalCapability\nactor"]
  R["SelectedItems\nresult"]
  SEL["selection\ncandidateSource=CandidateStore\ncriteria ...\nbasis ...\nbound
    ↪ ..."]

  Q --> A --> R
  SEL -.- A
  SEL -.- S
```



Lettura del grafico

Il cilindro deriva da `semanticKind=store` ed e' referenziato come `candidateSource` dalla local structure; non compare una freccia blu Store → actor soltanto per soddisfare `selection`. Le frecce blu appartengono all'owner `produce`; un eventuale accesso allo Store richiede semantica separata di retrieval/transfer. Il box grigio qualifica soltanto la membership del result. La figura non implica query/read, transfer, contract closure o output ordering.

15 Operatore hasPart

REBUILD CONTRACT

15.1 Che cosa rappresenta

hasPart rappresenta l'asserzione governata che un BAReferent indicato come part costituisce un elemento della composizione del BAReferent indicato come whole nel contesto considerato. Il costrutto rende esplicita una struttura whole/part senza ridurla a uso, dipendenza, collocazione fisica o ownership.

Smallest semantic fact

Esiste un significato W e un significato distinto P tali che la documentazione governa che P appartenga alla composizione di W. W assume il role whole; P assume il role part. La relation non fonde le due identita'.

15.2 Quando usarlo

Usare hasPart quando la documentazione governa realmente che un significato sia parte, componente o elemento costitutivo della composizione di un altro significato e i due significati devono restare distinguibili nell'analisi. Le parole usate dalla prosa non sono decisive: deve essere supportato il rapporto di composizione.

15.3 Quando non usarlo

Non usare hasPart per significare che W usa, richiede, produce, realizza, contiene fisicamente, ospita, memorizza, acquisisce da, dipende da o e' vincolato da P. Questi fatti possono coesistere con una composizione, ma richiedono evidence e proposition proprie.

Gate minimo

Se la source non permette di identificare esattamente un whole, un part e il fatto che il secondo appartenga alla composizione del primo, l'esito e' INSUFFICIENT EVIDENCE / NOT SPECIFIED. Una lista di elementi usati dal sistema o collocati nello stesso ambiente non basta da sola.

15.4 Firma canonica

La firma corrente del candidate di rebuild e' intenzionalmente binaria:

```
hasPart
  whole     -> BAReferent [1]
  part      -> BAReferent [1]
  polarity  = affirmative | negative [1]
```

Gli scoped modifier `condition` e `temporalScope` compaiono soltanto quando la source limita realmente l'asserzione.

Role	Card.	Significato
`whole`	1	BAReferent la cui composizione comprende la parte governata.
`part`	1	BAReferent che la documentazione governa come parte diretta del whole nella proposition considerata.

Una coppia whole/part per proposition

Ogni `BAProposition hasPart` rappresenta una sola coppia diretta `whole`/`part`. Se lo stesso whole possiede tre parti governate, la BA conserva tre proposition distinguibili. Questo permette provenance, review, condition e change impact indipendenti.

15.5 Lettura inversa

`partOf` non viene introdotto come secondo operatore. "`P` e' parte di `W`" e' la lettura linguistica inversa della stessa proposition `W hasPart P`; non crea una seconda identita' semantica.

15.6 Continuita' di identita' di whole e part

`whole` e `part` sono role operator-scoped. Un referent che e' `part` in una proposition puo' essere `whole` in un'altra senza cambiare identita'. Allo stesso modo, la stessa parte puo' comparire in piu' contesti di composizione senza diventare automaticamente un nuovo BAReferent.

```
A hasPart B
B hasPart D
```

B resta un solo BAReferent: e' `part` rispetto ad A e `whole` rispetto a D.

15.7 Invarianti

- ogni `hasPart` accettato possiede esattamente un `whole` e un `part`;
- whole e part restano identita' distinte e riusabili;
- il nesting di piu' proposition non crea closure transitiva;
- polarity negativa richiede una negazione governata della composizione; assenza di evidence non equivale a `hasPart` negativo;
- nessun constraint viene propagato automaticamente dal whole alla part o dalla part al whole.

15.8 Forbidden inference

Dal solo `hasPart` non inferire ownership del lifecycle, contenimento fisico, co-location, dipendenza, uso, requirement, realization, produzione, transfer, storage, installazione, acquisizione, host, equivalenza di owner o propagazione di constraint.

Nessuna transitivita' automatica

Da `A hasPart B` e `B hasPart D` non inferire una terza proposition `A hasPart D`. Una view puo' mostrare D geometricamente dentro A per effetto del nesting ricorsivo, ma quella ancestry visuale non crea una relation BA aggiuntiva.

15.9 Domande da porre alla documentazione

Le domande cercano il significato di composizione prima della forma grafica o della sintassi.

1. Qual e' esattamente il `whole`? La sua identita' e' gia' governata o richiede BA1 review?
2. Qual e' esattamente il `part`? Deve sopravvivere come identita' analitica indipendente?
3. La source governa che il part appartenga alla composizione del whole, oppure governa soltanto uso, disponibilita', dipendenza, location o requisito?
4. Whole e part hanno gia' BAReferent con la stessa identita' semantica? Se si', riusarli.
5. La relation vale sempre oppure soltanto sotto una condition o un temporal scope governato?
6. La source afferma o nega esplicitamente la composizione? Non dedurre polarity negativa dall'assenza della parte.
7. Il part e' anche whole di altre parti? Se si', conservarne la stessa identita' e lasciare alla projection il nesting ricorsivo.
8. Lo stesso part appartiene direttamente a piu' whole? Non duplicare il BAReferent: sara' la projection a gestire occurrence multiple.
9. Un constraint oggi applicato al whole deve davvero essere retargeted alla part? `hasPart` da solo non autorizza questo spostamento.

Procedura minima

Isolare il smallest governed composition fact; applicare l'identity test a whole e part; compilare una sola proposition binaria; aggiungere modifier soltanto se governati; infine verificare forbidden inference, non-transitivita' e assenza di retargeting automatico di constraint o altre proposition.

15.10 Esempio generico minimo

Documentazione inventata ma coerente

"L'ambiente applicativo e' composto da un sottosistema di calcolo e da un sottosistema di configurazione."

Assumendo che le identita' abbiano superato BA1, la documentazione supporta due proposition distinte:

```
ApplicationEnvironment hasPart ComputeSubsystem
ApplicationEnvironment hasPart ConfigurationSubsystem
```

Non si deduce che i due subsystem siano fisicamente collocati nello stesso host, che condividano lifecycle o che ereditino i constraint dell'ambiente.

15.11 Regression case corrente - DermaTriage MR-C6

La documentazione DDTA corrente governa che `DermaTriageEnvironment` sia composto da risorse di calcolo, software, modelli, dati e configurazioni. Il regression test ha mostrato che la BA corrente preserva la produzione dell'ambiente ma non la sua composizione interna.

Per `FR-C6-01-01`, il candidate minimale e':

```
BAReferent candidate:
  ComputeAndStorageResources

BAProposition candidate BAP-FRC6-01-01-05
  semanticOperatorKey = hasPart
  whole -> DermaTriageEnvironment
  part  -> ComputeAndStorageResources
  polarity = affirmative
```

Boundary del regression case

Le proposition correnti `minimumRAM = 16 GB` e `minimumStorage = 50 GB` restano constraint di `DermaTriageEnvironment` finche' una review separata non dimostra che debbano essere re-targeted. L'introduzione di `hasPart` rende visibile la composizione; non riscrive automaticamente proposition gia' supportate.

15.12 Prompt LLM di supporto all'authoring

Il prompt e' un ausilio operativo. Non usa il nesting desiderato del diagramma come evidence e non crea parti soltanto per ottenere una topologia piu' leggibile.

```
TASK: identify governed BA hasPart propositions.
Use ONLY the supplied documentation and existing BA registry.

A hasPart proposition exists only if the source supports:
  whole [1], part [1].

Rules:
1. Identify the smallest governed direct composition fact.
2. Reuse existing BAReferents when semantic identity is unchanged.
3. Propose a new BAReferent only when BA1 independent identity is needed.
4. Accept hasPart only when the source governs that part belongs to
   the composition of whole.
5. Do not convert use, requirement, dependency, location, hosting,
   realization, production, transfer, storage or acquisition into hasPart.
6. Emit one whole/part pair per BAProposition.
7. Do not infer transitive hasPart propositions.
8. Do not retarget existing constraints merely because a part exists.
9. If the same part belongs to multiple wholes, reuse the same BAReferent.
10. Use polarity=negative only for an explicitly negated composition fact.
11. Add condition/temporalScope only when explicitly governed.
12. If whole, part or composition evidence is missing, return
    INSUFFICIENT_EVIDENCE.
13. Do not invent operators, roles, states or metadata.

Return EXACTLY the authoring wrapper and BA output shape shown next.
```

Human review remains mandatory

La distinzione fra composizione e semplice presenza tecnica e' semanticamente materiale. L'LLM puo' proporre whole e part, ma l'accettazione richiede verifica contro source, identity criterion e proposition gia' estratte.

15.13 Output esatto atteso dall'estrazione assistita

Il wrapper seguente e' un formato di authoring/review, non un nuovo costrutto BA. La parte BA_OUTPUT usa soltanto la shape ammessa.

```
HASPART_ASSESSMENT
  decision = ACCEPT_HAS_PART | NOT_A_HAS_PART | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  notSpecified = [<only unresolved material items>]

BA_OUTPUT
  BAReferent reuse/candidates:
    <canonical names only as needed>

  BAProposition <candidate-id>
    semanticOperatorKey = hasPart
    whole -> <BAReferent>
    part  -> <BAReferent>
    polarity = affirmative | negative
    scopedModifier = <condition|temporalScope>  # omit if absent
```

Se `decision != ACCEPT_HAS_PART`, BA_OUTPUT non contiene una proposition `hasPart` accettabile.

15.14 Discriminazioni minime da congelare

Confine	Regola
`hasPart` vs `realize`	Una realization materializza un significato astratto; una part appartiene alla composizione di un whole.
`hasPart` vs `produce`	Un result e' reso disponibile da un actor; questo non stabilisce che il result diventi parte della sua composizione o di quella di un altro referent.
`hasPart` vs `constrain`	Un constraint qualifica un target; non lo decompone. La composizione non propaga automaticamente i constraint.
`hasPart` vs storage/location/use	Memorizzazione, collocazione o uso possono coinvolgere gli stessi referent, ma non equivalgono a composizione.

Composizione funzionale stabile, realization e comportamento runtime

Quando una parte tecnica svolge una funzione logicamente distinguibile, la BA preserva, quando source-supported, la funzione come identita' stabile e la realizzazione concreta come identita' sostituibile. `hasPart` puo' descrivere la **composizione funzionale stabile** del whole strutturale governato — per esempio il sistema o una funzione composta — mentre `realize` collega la funzione alla realization corrente. Decision, `condition` e relazioni comportamentali descrivono quali parti partecipano e come interagiscono nei diversi scenari runtime: non ridefiniscono automaticamente la composizione. Un percorso, branch o scenario operativo diventa `whole` di `hasPart` soltanto quando la source ne governa realmente la composizione come tale, non per il solo fatto che selezioni, attivi o coordini funzioni gia' parte del sistema. La composizione dell'environment o del deployment costituisce un ulteriore asse distinto e non viene dedotta automaticamente dalla composizione funzionale ne' da `realize`.

15.15 Rappresentazione Mermaid di hasPart

Per `hasPart` la projection canonica usa il **containment grafico**: il `part` viene collocato direttamente dentro il container del `whole`. Non viene disegnata una freccia `hasPart`. La semantica deriva dal View Contract, non dalla comodita' sintattica di Mermaid.

```
flowchart TB
subgraph A["A"]
  direction TB
  subgraph B["B"]
    direction TB
    D["D"]
  end
  C["C"]
end
```

La lettura BA e' esattamente: `A hasPart B`, `A hasPart C`, `B hasPart D`. Il fatto che `D` risulti geometricamente anche dentro `A` non crea `A hasPart D`: solo il containment *immediato* proietta una proposition.

15.16 Occurrence multiple dello stesso BAReferent

Se lo stesso part appartiene direttamente a piu' whole, la BA mantiene una sola identita'. La projection puo' ripetere l'occurrence:

```
A contains X
B contains X<copy>
```

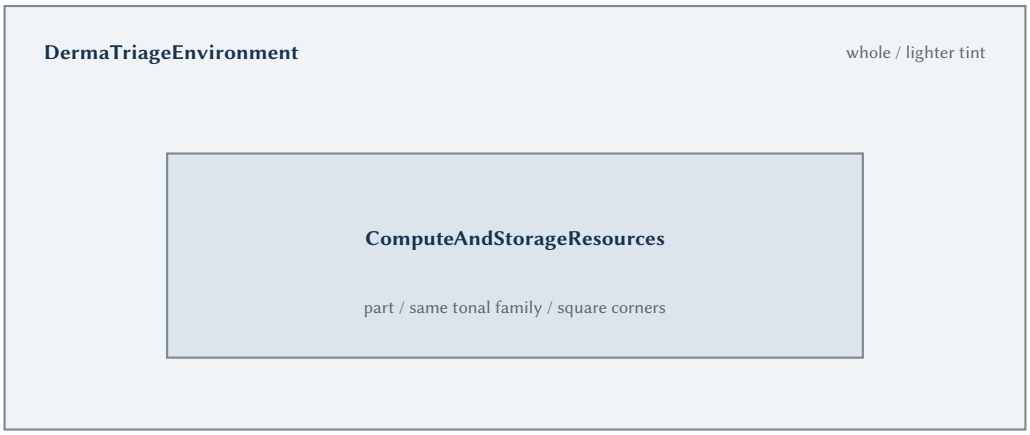
`<copy>` e' metadata di projection: non crea `X-copy`, non crea `copyOf` e non modifica le proposition. La primary occurrence e le copie vengono assegnate con ordinamento canonico dei direct whole; se il referent copiato e' a sua volta whole, il suo subtree visibile puo' essere ripetuto applicando la stessa convenzione.

Intersezione grafica non canonica

Una view specializzata puo' usare intersezioni geometriche quando il renderer e la topologia lo consentono. La projection canonica resta pero' nesting + `<copy>`, perche' non deve dipendere dalla possibilita' di disegnare intersezioni arbitrarie senza ambiguita'.

15.17 Regole grafiche del containment

`hasPart` decide il nesting, non sostituisce l'identita' grafica gia' assegnata ai referent. Per gli elementi generici padre e figlio usano rettangoli ad angoli retti e la stessa famiglia cromatica; il container esterno puo' usare una tonalita' piu' chiara per mantenere leggibile il contenuto. Se una part possiede gia' una regola grafica specifica — per esempio `semanticKind=store`, gia' rappresentato come cilindro nella Sezione 13, un documento o un'altra semantic kind — quella regola prevale anche quando l'elemento e' annidato. Il containment cambia la posizione grafica del referent, non la sua identita' visuale semantica.



Lettura deterministica

Il nesting diretto del box `ComputeAndStorageResources` dentro `DermaTriageEnvironment` e' la projection di una sola candidate proposition `hasPart`. Nessuna freccia aggiuntiva viene creata. Colore, bordo e forma non aggiungono BA semantics: derivano dalle regole di projection. Un'occurrence `<copy>` mantiene esattamente lo stile grafico del BAReferent canonico.

Precedenza delle regole grafiche

Ordine di precedenza: (1) regola grafica semantica gia' assegnata al BAReferent; (2) regola operator-specific di `hasPart`, cioe' containment; (3) fallback generico rettangolare nella stessa famiglia tonale del parent; (4) dettagli di layout del renderer. Nessuna differenza di colore, bordo o forma viene introdotta soltanto dalla profondita' del nesting.

16 Boundary e attraversamento di boundary

REBUILD CONTRACT

16.1 Che cosa rappresenta

Un `Boundary` identifica un confine architetturale o topologico che delimita uno scope governato. Il suo scopo BA e' rendere interrogabile *dove* cambia uno scope rilevante prima che la security analysis stabilisca se quel cambiamento corrisponda a differenze di trust, rischio, controllo amministrativo, rete o altre classificazioni security-specific.

Smallest semantic fact

Esiste un confine identificabile `B` che delimita uno scope identificato `S`. La BA puo' quindi preservare `B` come BAReferent e asserire `B boundaryOf S`. Una relazione gia' identificata puo' inoltre essere qualificata da una proposition `crossesBoundary` quando la source/evidence supporta che la relazione attraversi proprio `B`.

16.2 Neutralita' rispetto alla security analysis

`Boundary` non significa `TrustBoundary`, non classifica automaticamente un lato come trusted o untrusted e non assegna livello di rischio. "Remote", "external", "cloud" o il nome di un provider non sono, da soli, classificazioni di trust. La BA preserva il confine; una successiva analisi di sicurezza puo' qualificarlo senza mutarne l'identita' topologica.

16.3 Quando usarlo

Usare il costrutto quando il confine deve sopravvivere come identita' interrogabile e la documentazione/evidence consente di identificare lo scope delimitato. Non introdurlo soltanto per racchiudere graficamente nodi o migliorare il layout.

Gate minimo

Se lo scope non e' identificabile o l'esistenza del confine deriva soltanto dalla posizione nel diagramma, l'esito e' `INSUFFICIENT EVIDENCE`. La projection non puo' creare un Boundary che la BA non possiede.

16.4 Identita' del Boundary

`Boundary` e' la lettura umana del semantic kind `boundary` applicato a un BAReferent. Il kind descrive la natura del referent; non crea una terza famiglia BA1 e non sostituisce la proposition che collega il confine allo scope.

```
BAReferent <boundary-id>
  semanticKind = [boundary]
```

16.5 Firma di `boundaryOf`

```
boundaryOf
  boundary  -> BAReferent [1]  # semanticKind includes boundary
  scope     -> BAReferent [1]
  polarity  = affirmative | negative [1]
```

Role	Card.	Significato
`boundary`	1	BAReferent che identifica il confine governato.
`scope`	1	BAReferent il cui perimetro/topologia e' delimitato dal boundary.

boundaryOf non e' hasPart

`B boundaryOf S` non significa `S hasPart B`, non rende il boundary un componente runtime e non afferma che gli elementi geometricamente disegnati dentro un frame siano automaticamente part di `S`. Composizione e delimitazione restano assi distinti.

16.6 Identita' e riuso

Lo stesso Boundary puo' qualificare piu' relazioni e sopravvivere a sostituzioni di provider, endpoint o realization finche' resta invariato il significato del confine. Se cambia realmente lo scope delimitato, applicare il normale identity/change test invece di rinominare per convenienza grafica.

16.7 Operatore `crossesBoundary`

`crossesBoundary` preserva il fatto che una relation/proposition gia' identificata attraversi un Boundary specifico. E' una proposition distinta e qualificante: non modifica l'operatore della proposition qualificata.

```
crossesBoundary
  relation  -> BAProposition [1]
  boundary  -> BAReferent [1]  # semanticKind includes boundary
  polarity  = affirmative | negative [1]
```

Qualificazione senza riscrittura

Se `BAP-X` e' una proposition `transfer`, una seconda proposition puo' dichiarare `crossesBoundary relation -> BAP-X`. `BAP-X` resta `transfer`: source, destination, content, behavior, provenance e identity non vengono copiati ne' sostituiti.

16.8 BAProposition come term controllato

Il riferimento proposition-to-proposition e' ammesso soltanto nelle firme che lo dichiarano. Serve quando il nuovo fact ha come oggetto l'asserzione gia' identificata, non uno dei suoi partecipanti preso isolatamente.

Qualification without mutation

Una proposition qualificante aggiunge un asse ortogonale a una proposition owner/referenced. Non eredita automaticamente i suoi role, non ne cambia polarity o scope, non crea una nuova versione della proposition qualificata e non autorizza riferimenti ricorsivi o self-reference.

16.9 Non limitato a transfer

Il regression corrente usa `transfer`, ma `crossesBoundary` non e' definito come role interno di `transfer`. Se un futuro operatore rappresenta una relazione capace di attraversare un boundary e la source lo governa, la stessa qualificazione puo' essere riusata senza alterare la firma dell'operatore originario.

16.10 Evidence richiesta per il crossing

L'attraversamento non viene dedotto dal solo fatto che un partecipante non compaia nella closure di `hasPart` dello scope. Assenza di evidence non equivale a evidence di esteriorita'. Per accettare `crossesBoundary` deve esistere evidence positiva sufficiente per localizzare la relation rispetto al boundary oppure una source che governa direttamente il crossing.

16.11 Invarianti

- ogni `boundaryOf` identifica esattamente un Boundary e uno scope;
- ogni `crossesBoundary` riferisce una BAProposition gia' identificata e un Boundary gia' identificato;
- la proposition qualificata mantiene semanticOperatorKey, role, identity e provenance;
- un Boundary non assegna automaticamente trust, rischio, affidabilita' o controllo ai lati che delimita;
- un crossing puo' essere security-relevant senza essere gia' valutato dalla security analysis;
- lo stesso Boundary puo' essere attraversato da piu' proposition senza duplicarne l'identita';
- una source/endpoint non identificata non viene inventata per completare il crossing.

16.12 Forbidden inference

Dal solo `Boundary` o `crossesBoundary` non inferire: `trusted/untrusted`, threat, vulnerability, exploitability, risk level, encryption, authentication, integrity verification, ownership, network zone, provider trust, secure/insecure transfer, oppure successo/fallimento dei controlli.

External non equivale a untrusted

Un locator remoto o un provider esterno puo' supportare la distinzione topologica necessaria al crossing, ma non autorizza a qualificare il provider come non affidabile. La security analysis puo' successivamente applicare una classificazione di trust/risk al Boundary o al crossing preservando le identity BA originali.

16.13 Domande da porre alla documentazione

1. Quale scope e' realmente delimitato? Esiste gia' come BAReferent?
2. Il confine deve essere interrogato/riusato come identita' autonoma oppure e' soltanto layout?
3. La source identifica direttamente il boundary oppure fornisce evidence sufficiente per il suo significato topologico?
4. Quale BAProposition viene qualificata dal crossing? E' gia' identificata e stabile?
5. Quale evidence mostra che la relation attraversa il boundary? Non usare la sola assenza da `hasPart` come prova di "outside".
6. Source, destination o altri partecipanti rilevanti sono identificati? Se manca un endpoint, il crossing e' determinabile oppure resta unresolved?
7. Si sta introducendo trust/risk nella BA topologica? Se si', separare la successiva security analysis.
8. La projection desiderata sta guidando l'estrazione? Se si', fermarsi: il diagramma non e' source semantica.

Procedura minima

Applicare l'identity test al Boundary; identificare lo scope; emettere `boundaryOf`; individuare la proposition eventualmente attraversante; verificare evidence positiva del crossing; emettere `crossesBoundary` soltanto quando determinabile. In caso contrario conservare il gap documentale senza creare BA fittizia.

16.14 Stato unresolved non e' una proposition inventata

Se una dependency installation e' governata ma la package source/registry e' `NOT SPECIFIED`, non creare `PyPIStore`, un generico external Store o un `crossesBoundary` presunto. L'authoring/review puo' registrare:

```
boundaryCrossingAssessment = UNRESOLVED
reason = dependency source / package registry NOT SPECIFIED
```

Questo assessment resta fuori dalla BA finche' l'evidence non supporta la proposition.

16.15 Regression case - DermaTriage MR-C6

Il caso corrente fornisce un singolo confine topologico utile a piu' crossing:

```
BAReferent candidate:
  DermaTriageEnvironmentBoundary
    semanticKind = [boundary]

BAP-MRC6-08  boundaryOf
  boundary -> DermaTriageEnvironmentBoundary
  scope    -> DermaTriageEnvironment
```

Repository acquisition. La proposition accettata `BAP-FRC6-01-07-03` descrive il transfer dal repository remoto GitHub al source-code Store locale. La qualificazione candidata e':

```
BAP-FRC6-01-07-07  crossesBoundary
  relation -> BAP-FRC6-01-07-03
  boundary -> DermaTriageEnvironmentBoundary
```

Model acquisition. `BAP-FRC6-01-03A1-04` descrive il transfer dal Google Drive model Store all'ActiveClassifierStore. Gli ID A1-10..13 restano inutilizzati; il nuovo candidate usa il successivo ID libero:

```
BAP-FRC6-01-03A1-14  crossesBoundary
  relation -> BAP-FRC6-01-03A1-04
  boundary -> DermaTriageEnvironmentBoundary
```

Dependency installation. Per `pip install -r requirements.txt`, la source/package registry resta `NOT SPECIFIED`. Non viene quindi materializzata alcuna proposition `crossesBoundary`: lo stato resta `UNRESOLVED` nell'assessment documentale.

Un solo Boundary, piu' crossing

GitHub e Google Drive non richiedono due Boundary soltanto perche' sono provider differenti. Nel regression corrente il fatto governato e' l'attraversamento del perimetro di `DermaTriageEnvironment`; eventuali confini provider-specific richiederebbero evidence e identity test propri.

16.16 Output atteso dall'authoring controllato

```
BOUNDARY_ASSESSMENT
  decision = ACCEPT_BOUNDARY | NOT_A_BOUNDARY | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  crossing = TRUE | FALSE | UNRESOLVED
  notSpecified = [<only unresolved material items>]

BA_OUTPUT
  BAReferent <boundary-id>
    semanticKind = [boundary]

  BAProposition <boundary-of-id>
    semanticOperatorKey = boundaryOf
    boundary -> <boundary BAReferent>
    scope    -> <scope BAReferent>

  BAProposition <crossing-id>       # only when supported
    semanticOperatorKey = crossesBoundary
    relation -> <existing BAProposition id>
    boundary -> <boundary BAReferent>
```

16.17 Discriminazioni da congelare

Confine	Regola
Boundary vs TrustBoundary	La BA topologica identifica il confine; trust/risk e' qualificazione successiva della security analysis.
`boundaryOf` vs `hasPart`	Delimitazione e composizione sono assi distinti; nessuna relation implica l'altra.
`crossesBoundary` vs `transfer`	Il crossing qualifica una proposition esistente; non sostituisce ne' modifica `transfer`.
Outside vs absence	Non appartenenza non si deduce dall'assenza di `hasPart`; serve evidence positiva.
UNRESOLVED vs FALSE	Evidence insufficiente non e' una negazione del crossing.

Handoff alla security analysis e alla projection

La BA rende deterministico *quale* Boundary esiste e *quale* proposition lo attraversa. La security analysis puo' poi qualificare rischio, trust e controlli senza riscrivere la topologia. Colori, forme, frame e stati visuali (per esempio un conservative worst-case per crossing non ancora valutati) appartengono alla guida BA-to-Mermaid/projection, non alla semantica del costrutto BA.

A Materiale temporaneo di rebuild

APPENDICE TEMPORANEA DI REBUILD - DA ELIMINARE PRIMA DELLA GUIDA DEFINITIVA

Il contenuto di questa appendice governa *come stiamo ricostruendo la guida*, non il funzionamento della Base Analysis. Deve restare disponibile durante il rebuild per garantire coerenza, genealogia e regression, ma va rimosso dalla guida definitiva dopo il consolidamento. Se una sua regola deve sopravvivere come metodologia BA, deve essere trasferita esplicitamente nella parte metodologica destinata alla guida definitiva prima della rimozione.

A.1 Scopo, lineage e confine della ricostruzione

Questa guida non nasce per dichiarare che la revisione piu' recente sia automaticamente la migliore. Nasce per rendere leggibile e applicabile il patrimonio BA gia' costruito, riesaminandolo in modo uniforme mentre viene applicato alla documentazione DermaTriage corrente.

Regola di costruzione cumulativa

La nuova struttura editoriale puo' essere diversa da R3, Core R42 o Complete R42. Il significato precedente, pero', non puo' scomparire per effetto della riscrittura. Ogni distinzione precedente deve essere preservata, raffinata, rilocalizzata, ritestata, respinta con motivazione oppure lasciata esplicitamente aperta.

A.2 Tre livelli da non confondere

Livello	Esempio	Effetto
Authority corrente	BA0–BA5 + Operational Guide R3	Governa il metodo corrente finche' non esiste promotion esplicita.
Research evidence	R42, operator review, ledger, pressure, candidate, vecchi appunti	Puo' motivare chiarimenti o cambi, ma non modifica da solo il metodo.
Rebuild candidate	questa guida	Superficie cumulativa di review; diventa authority solo con checkpoint successivo.

Nessun target di cardinalita'

Il fatto che la BA corrente abbia quattordici operatori top-level non crea un obiettivo di quattordici operatori. Anche il precedente snapshot a tredici elementi resta genealogia utile, non un target da ripristinare. Il numero finale deve essere un risultato della necessita' semantica e della regression, non una quota.

APPENDICE TEMPORANEA DI REBUILD - DA ELIMINARE PRIMA DELLA GUIDA DEFINITIVA

Il contenuto di questa appendice governa *come stiamo ricostruendo la guida*, non il funzionamento della Base Analysis. Deve restare disponibile durante il rebuild per garantire coerenza, genealogia e regression, ma va rimosso dalla guida definitiva dopo il consolidamento. Se una sua regola deve sopravvivere come metodologia BA, deve essere trasferita esplicitamente nella parte metodologica destinata alla guida definitiva prima della rimozione.

A.3 Corpus di ricostruzione e stati epistemici

Il rebuild usa come input controllato il registro `DDTA_R25_BASE_ANALYSIS_REBUILD_SOURCE_REGISTER_R1.md`. Prima di chiudere un costrutto vanno ricercati non solo il suo nome corrente, ma anche alias storici, pressure collegate, candidate construct, alternative rifiutate e counterexample.

Recency non equivale ad authority

R42 Core/Complete e' una superficie di lettura aggiornata e molto utile per il rebuild. R3 e BA0–BA5 restano tuttavia authority corrente. La guida nuova puo' usare R42 come base esplicativa senza promuovere automaticamente le sue candidate o chiudere le sue open pressure.

A.4 Etichette epistemiche minime

Etichetta	Uso	Disciplina
CURRENT / PRESERVED	Semantica corrente preservata	Deve restare compatibile con l'authority o dichiararne esplicitamente il delta.
CANDIDATE / NOT ADMITTED	Costrutto/evoluzione con evidence positiva ma non ammessa	Non usare come vocabulary corrente senza admission esplicita.
OPEN	Problema reale non chiuso	Mantenere il gap visibile; non completare per convenienza.
REJECTED	Ipotesi testata e respinta nello scope disponibile	Conservare ragione, test e condizioni di eventuale reopen.
REBUILD CONTRACT	Regola editoriale del rebuild	Governa come descriviamo e confrontiamo i costrutti durante questa ricostruzione.

Regola anti-perdita

Una guida piu' corta non e' una guida migliore se per ottenere la sintesi elimina una distinzione ancora utile. Il rebuild puo' comprimere la prosa soltanto dopo aver dimostrato che la distinzione resta ricostruibile e tracciabile.

APPENDICE TEMPORANEA DI REBUILD - DA ELIMINARE PRIMA DELLA GUIDA DEFINITIVA

Il contenuto di questa appendice governa *come stiamo ricostruendo la guida*, non il funzionamento della Base Analysis. Deve restare disponibile durante il rebuild per garantire coerenza, genealogia e regression, ma va rimosso dalla guida definitiva dopo il consolidamento. Se una sua regola deve sopravvivere come metodologia BA, deve essere trasferita esplicitamente nella parte metodologica destinata alla guida definitiva prima della rimozione.

A.5 Construct Description Contract

REBUILD CONTRACT

Prima di riscrivere il primo costrutto BA, fissiamo il modo in cui *tutti* i costrutti verranno descritti. Questo contratto serve a impedire che la guida cambi struttura in funzione del caso del momento e rende confrontabili operatori current, condition form, strutture candidate, pressure e ipotesi rifiutate.

Invariante editoriale

Ogni costrutto usa la stessa sequenza descrittiva. Un campo puo' essere NOT APPLICABLE, NOT FROZEN o OPEN; non puo' essere omesso silenziosamente. Se un costrutto futuro dimostra che il contratto non basta, si modifica prima il contratto e poi si riallineano i costrutti gia' trattati.

Controllo rigoroso, esposizione naturale

Il CDC e' un contratto di *copertura*, non un template rigido di impaginazione. I campi richiesti devono essere tutti trattati e ricostruibili durante la review, ma la guida definitiva puo' combinarli in paragrafi e sezioni piu' naturali quando questo non nasconde informazioni, stato epistemico o confini semantici. Struttura di controllo rigorosa non significa esposizione meccanica.

APPENDICE TEMPORANEA DI REBUILD - DA ELIMINARE PRIMA DELLA GUIDA DEFINITIVA

Il contenuto di questa appendice governa *come stiamo ricostruendo la guida*, non il funzionamento della Base Analysis. Deve restare disponibile durante il rebuild per garantire coerenza, genealogia e regression, ma va rimosso dalla guida definitiva dopo il consolidamento. Se una sua regola deve sopravvivere come metodologia BA, deve essere trasferita esplicitamente nella parte metodologica destinata alla guida definitiva prima della rimozione.

A.6 Direzione obbligatoria della spiegazione

```
human semantic idea
    -> BA term / construct class
    -> formal shape
    -> boundaries and discriminators
    -> examples
    -> evidence and genealogy
    -> open questions
    -> current rebuild disposition
```

Da evitare

- iniziare dalla firma e lasciare al lettore il compito di indovinarne il significato;
- usare l'esempio DermaTriage come definizione implicita del costrutto;
- aggiungere campi solo a un costrutto perche' il suo caso concreto ne ha bisogno;
- completare la firma di una candidate non ammessa per rendere il template piu' ordinato;
- nascondere rejected hypothesis o pressure per ottenere una sezione apparentemente "chiusa".

A.7 Revisione del contratto

Il contratto parte come CDC-R1. Ogni sua modifica futura deve avere un counterexample, una motivazione generalizzabile e una regression sui costrutti gia' descritti.

APPENDICE TEMPORANEA DI REBUILD - DA ELIMINARE PRIMA DELLA GUIDA DEFINITIVA

Il contenuto di questa appendice governa *come stiamo ricostruendo la guida*, non il funzionamento della Base Analysis. Deve restare disponibile durante il rebuild per garantire coerenza, genealogia e regression, ma va rimosso dalla guida definitiva dopo il consolidamento. Se una sua regola deve sopravvivere come metodologia BA, deve essere trasferita esplicitamente nella parte metodologica destinata alla guida definitiva prima della rimozione.

A.8 Construct Description Contract - template, parte 1

N.	Campo	Contenuto richiesto
1	Canonical name	Nome usato dal rebuild. Alias storici vanno nel campo genealogy, non come nomi paralleli current.
2	Construct class	Top-level operator, condition form, local reusable structure, candidate, open pressure, rejected hypothesis o altro meccanismo BA dichiarato.
3	Epistemic status	Stato esplicito: current/preserved, candidate/not admitted, open, rejected oppure altro stato definito.
4	Owning BA layer / contract	Dove il significato appartiene: BA0, BA1, BA2, BA3, BA4, BA5 oppure confine cross-layer esplicito.
5	Human definition	Spiegazione in linguaggio naturale di che cosa rappresenta il costrutto.
6	Smallest semantic fact	Il nucleo minimo che si perde eliminando il costrutto.
7	When to use	Condizioni positive che giustificano l'uso. Devono riferirsi a significato governato, non a parole chiave.
8	When not to use	Casi in cui il costrutto sembra plausibile ma imporrebbe semantica sbagliata o eccessiva.
9	Canonical syntax / shape	Forma corrente se ammessa. Per candidate non chiuse: NOT FROZEN salvo working shape esplicitamente marcata.
10	Roles, types, cardinalities	Ruoli e vincoli formali ammessi. Nessun role viene inventato per l'esempio corrente.

Semantica prima della sintassi

I campi 5–8 devono poter essere compresi anche da chi non conosce ancora la sintassi BA. I campi 9–10 formalizzano il significato; non lo sostituiscono.

APPENDICE TEMPORANEA DI REBUILD - DA ELIMINARE PRIMA DELLA GUIDA DEFINITIVA

Il contenuto di questa appendice governa *come stiamo ricostruendo la guida*, non il funzionamento della Base Analysis. Deve restare disponibile durante il rebuild per garantire coerenza, genealogia e regression, ma va rimosso dalla guida definitiva dopo il consolidamento. Se una sua regola deve sopravvivere come metodologia BA, deve essere trasferita esplicitamente nella parte metodologica destinata alla guida definitiva prima della rimozione.

A.9 Construct Description Contract - template, parte 2

N.	Campo	Contenuto richiesto
11	Semantic invariants	Cio' che deve restare vero in ogni uso valido del costrutto.
12	Forbidden inferences	Fatti che la presenza del costrutto non autorizza a dedurre.
13	Closest constructs / discriminators	Confini pairwise con i costrutti piu' facili da confondere.
14	Minimal generic positive example	Esempio didattico minimo, separato dall'evidence di progetto.
15	DermaTriage application example	Uso reale sulla documentazione corrente, quando disponibile e supportato.
16	Negative / boundary example	Caso in cui il costrutto va rifiutato o usato con limite preciso.
17	Genealogy and research evidence	Authority, candidate guide, operator review, PR/CC/CL/CMD/OBS e precedenti rilevanti.
18	Open pressures / candidate deltas	Questioni ancora vive che non devono essere nascoste dalla descrizione principale.
19	Review / acceptance questions	Domande ripetibili che permettono a un autore di verificare l'uso.
20	Current rebuild disposition	Cosa decide questa revisione: preserve, refine, retest, reject, hold, candidate admission later, ecc.

Change protocol del CDC

Se un costrutto non entra nel template, non si crea una eccezione locale. Si registra il counterexample, si verifica se il problema e' generale, si produce una revisione CDC-R2 e si riesaminano retroattivamente i costrutti gia' descritti. Nessuna modifica implicita del formato e' ammessa.

Candidate con firma non chiusa

Per `provideService` il template resta completo, ma i campi "Canonical syntax" e "Roles, types, cardinalities" possono dichiarare NOT FROZEN. E' preferibile una lacuna esplicita a una firma inventata solo per uniformare l'impaginazione.

APPENDICE TEMPORANEA DI REBUILD - DA ELIMINARE PRIMA DELLA GUIDA DEFINITIVA

Il contenuto di questa appendice governa *come stiamo ricostruendo la guida*, non il funzionamento della Base Analysis. Deve restare disponibile durante il rebuild per garantire coerenza, genealogia e regression, ma va rimosso dalla guida definitiva dopo il consolidamento. Se una sua regola deve sopravvivere come metodologia BA, deve essere trasferita esplicitamente nella parte metodologica destinata alla guida definitiva prima della rimozione.

A.10 Roadmap e registro temporaneo dei costrutti

Questo registro e' la coda esplicita della nuova guida cumulativa. Non promuove alcun costrutto: conserva cio' che e' gia' ricostruito, cio' che deve essere integrato e cio' che resta da studiare, usando il Source Register R25 e i ledger di review come evidence non normativa.

Registro temporaneo - candidate/strutture da integrare nella guida

Costrutto	Classe	Stato corrente	Azione nel rebuild
`decisionRule`	local `operatorStructure` di `produce` (rilocazione candidate)	`CANDIDATE / NOT ADMITTED`; due regression case positivi + C7-P02 predicate pressures	Sez. 12: priority + readiness, `gt/lt` candidate source-driven, endpoint pressure su governed-domain / governed-fact criterion reuse. Facial Access + checkpoint restano necessari prima di promotion.
`store`	controlled `semanticKind` di `BAReferent`	direct-field + optional binding working model sotto regression	Sez. 13: `semanticKind=store` sul BAReferent; cilindro in projection; binding opzionali. `classify` non e' richiesto per il typing ordinario e resta in responsibility review.
`selection`	local `operatorStructure` riusabile di `produce`	`FORMALIZED IN R7 / NON-NORMATIVE`; inventory BA2 top-level invariato	Sez. 14: candidate source, selected result, membership criteria, selection basis e bound; positive controls recent-N/top-K/best + RC-001. Prossimo microstep: retrieval/access.
`hasPart`	candidate top-level `semanticOperatorKey` BA2	`FORMALIZED IN R7 / NON-NORMATIVE`; regression MR-C6 positivo	Sez. 15: whole/part binario, non-transitivita', no constraint propagation; projection canonica a containment con <copy> per occurrence multiple.
`CC-02 provideService`	candidate construct	`TESTED_POSITIVE / NOT_ADMITTED`	Hold: formal gate G1–G8 / evidence review prima di eventuale integrazione.
`CC-03 storedIn`	candidate construct	`DEFERRED / NO CURRENT ADMISSION PRESSURE`	Non integrare finche' un caso governato dimostra perdita reale con `Store + transfer`; non creare una relation at-rest per completezza grafica.
`CC-04 initiate`	candidate construct	`TESTED_POSITIVE / NOT_ADMITTED`	Hold: formal gate G1–G8 e regression cross-corpus prima di eventuale integrazione.

Pressures / delta ancora da studiare

`PR-01` execution identity; `PR-02` pipeline composition/order; `PR-04` boundary/interaction; `PR-05/06` ordered/scalar comparison; `PR-07` structured information contract; `PR-09` acquisition/refresh; `PR-12` negative implication/non-sufficiency; `PR-14` applicability/configuration binding; **decisionRule governed-criterion reuse**: validita' rispetto a un dominio governato e occorrenza di un governed fact senza duplicazione; `CMD-OP04` observe.result removal; `CMD-OP05` transition state/value refinement; `OBS-OT-01` operation-target/effect-scope. Nessuna voce puo' scomparire senza disposition esplicita.

Stop point di questa R7

La Rebuild R7 mantiene completa la ricostruzione di `transfer`, `produce`, `realize`, `constrain` e `condition`; preserva la candidate `decisionRule`; documenta `store` con `semanticKind` diretto sul BAReferent e binding opzionali sotto regression; formalizza `selection` come reusable local `operatorStructure` di `produce`; integra `hasPart`; e aggiunge ora `Boundary`/`boundaryOf` e `crossesBoundary` come candidate verificati sul regression MR-C6, senza promozione automatica dell'authority BA. `classify` resta in responsibility review e `storedIn` deferito. Il Work Plan R7 puo' proseguire mantenendo separati i successivi finding source-first e il microstep retrieval/access C0.3.

Indice di integrita' delle pagine

Ogni pagina possiede un MD5 del proprio contenuto canonico; il valore stampato e' escluso dal payload e l'hash della pagina indice usa <SELF> per evitare dipendenza circolare.

Pag.	Contenuto	MD5 contenuto
1	Frontespizio della Base Analysis Guide Rebuild R7	b72d1ef67f3ac0b3e3238e688b5d7024
2	1 - Che cos'e' la Base Analysis	fa01b7db301f77cd9988907daf2b3397
3	1 - Perche' esiste e principi di base	3b620cf21e74f64e397656ec9f3038da
4	1 - Direzione dell'authority e feedback	b12d8f94dc04c833c0d916453255f9be
5	1 - Minimum sufficient BA, NOT SPECIFIED e gap	44c236d6809ab29df535fe29ff00352d
6	2 - BA View / Projection Contract e source types	b9514c3f03bdb13c46c35ba6ae647130
7	2 - Mermaid e campi minimi della view	db02dca9f3877577832e39efb2e6b7c1
8	2 - Coverage mode e famiglie di viste	b4b3bc661001ccb16d9acdef681806bd
9	2 - View gap e mapping discipline	dcd3d2d1eb4eb28971b488dcd1c7cf01
10	3 - Dal testo documentale al significato governato	9fdf9a0139ea36eaa577008c81fb109f
11	3 - Governed semantic fact	f0c4184f4f1ebccb7ce1ab9394c5c5209
12	3 - Segmentazione, ambiguita e livello documentale	e3200807fcbf2aa290f0748b9c099b69
13	4 - Elementi fondamentali della Base Analysis	b9028ef062953f4b7ab73bfb7ac7975c
14	4 - BAReferent: identita analitica riusabile	23934a19c0820cfc4e2d1aaaaac74123
15	4 - BAReferent: esempi DermaTriage e confini	17d66f19d9937e4c823ee9c212fb6ad8
16	4 - BAProposition: asserzione analitica identificabile	df015ccb75d48910ced0a5f1ddcdd881
17	4 - BAReferent e BAProposition insieme	8063825cd44fdd393ef91bf9a5957e86
18	5 - Granularita, minimalita e costruzione della BA	f23c4cd3c7776e49c0d794a9ecfef9a9
19	6 - Indipendenza tra struttura documentale e struttura BA	bc3e4197b5ba6fc299035dd3ee55f0fa
20	6 - Structural feedback e no authority inversion	90bad45a1ddf4546a84ffcaafe19514f
21	6 - DermaTriage DEC02 DEC04 come pressure case	a4a5a63a56b235dc9685ef8ade259ac3
22	6 - Downstream Utility Test e riallineamento BA	1a84bfcdad71b08085d90a414d4a8a46e
23	7 - transfer: definizione, smallest fact e criteri d'uso	e33af8b0be91d76075d6f8d588152778
24	7 - transfer: firma canonica, ruoli e cardinalita	e3dfd0ae1ea4b8fc206e94586671531d
25	7 - transfer: identity continuity, invarianti e forbidden inference	7fe9e1357a2287afc0bcd8cdc5d05cad
26	7 - transfer: domande di identificazione e procedura di estrazione	ca23e6876cbd5a9485a716d5b3e2a2a7
27	7 - transfer: esempio generico e worked example DermaTriage corrente	5b77098bebe4aee19463db63e451b2de
28	7 - transfer: prompt LLM per l'identificazione controllata	f59be85e2e26e94598d971d526ab5349
29	7 - transfer: output BA atteso e discriminazioni differite	daa47270de924161b6041ddc1f28526b
30	7 - transfer: rappresentazione Mermaid	fbf9ff38b0f615ee2066e351a264262d
31	7 - transfer: esempio grafico Mermaid	124230f8d8f7900c16fc39a88a25fa5c
32	8 - produce: definizione, smallest fact e criteri d'uso	a04b76a31302078a1ca5e5411d643f1e
33	8 - produce: firma canonica, ruoli e cardinalita	a33497f18dc46cda6fa6fc5ca79ce25b3
34	8 - produce: identity continuity, invarianti e forbidden inference	90363cec59b9cd817ff464809b0071e5
35	8 - produce: domande di identificazione e procedura di estrazione	33bf51c48bbe73a21ea78aaa81bce926
36	8 - produce: esempio generico e worked example DermaTriage corrente	cc6deb21f9c4c0bc45bf043617c58d0c
37	8 - produce: prompt LLM per l'identificazione controllata	929ad404323716bdb65630c172f8b864
38	8 - produce: output BA atteso e discriminazioni differite	29011b2ce81135be54d4b0002b9c69c8
39	8 - produce: rappresentazione Mermaid	7abd09f421a17323e61078e606c76cc5
40	8 - produce: esempio grafico Mermaid	b0e191c6045d2c9e9e1f7237f8a64722
41	9 - realize: definizione, smallest fact e criteri d'uso	fd686d1106dfb09f995cbd206d72f204
42	9 - realize: firma canonica, ruoli e cardinalita	7bf58985b2b7e03d4fa8cedf3a820e17
43	9 - realize: identity continuity, invarianti e forbidden inference	9b9c396fdc4904f4fc2aed0db151c640
44	9 - realize: domande di identificazione e procedura di estrazione	787e4c5ce9931b03135f6fe32337ab0c
45	9 - realize: esempio generico e worked example DermaTriage corrente	878f6fa9ebb37d3e1c92b53232d28d37
46	9 - realize: prompt LLM per l'identificazione controllata	1d9565d3ea82d81047cb85703c2209f8
47	9 - realize: output BA atteso e discriminazioni differite	541905031f0138d136150e7ae4a8c356
48	9 - realize: rappresentazione Mermaid	452922a9ba3eda4dd4fa6a64e3e8f262
49	9 - realize: esempio grafico Mermaid	ea668a6b4e75d6ae2af23bddd09f313e
50	10 - constrain: definizione, smallest fact e criteri d'uso	8eaecceab767d383792c89556a2de0a5
51	10 - constrain: firma canonica e forme di constraintValue	af4db5949e0aab4804e89cd6ac9e92e0
52	10 - constrain: identity continuity, invarianti e forbidden inference	fc50de66cce38880e9166f51bfb29b58
53	10 - constrain: domande di identificazione e procedura di estrazione	ed20ede9432d24c5fee8c50f3d80f862
54	10 - constrain: esempio generico e worked example DermaTriage corrente	9866d0d6f214e0b181ad9015f5137ad9
55	10 - constrain: prompt LLM per l'identificazione controllata	3505ea11275edcfd95f5eea19a71a5c4
56	10 - constrain: output BA atteso e discriminazioni differite	2e1a2c1b73e09d122c8929481c96b89f
57	10 - constrain: rappresentazione Mermaid	355deffe798b78d1fd962f60d355d796
58	10 - constrain: esempio grafico Mermaid composto con transfer	9ac0774d88929da648825995c1076378
59	11 - condition: definizione, smallest fact e criteri d'uso	ff13530c402da52d442bd04aa8b443d7
60	11 - condition: forma canonica, scope locale e cardinalita	d5d06eb1db7d0bce9416648ab93e2c02
61	11 - condition: identity boundary, invarianti e forbidden inference	a6de99c516eda39f3ac29f1a6246bb90
62	11 - condition: domande di identificazione e procedura di estrazione	ec89bcdef246536ca6936608071ae78d
63	11 - condition: esempio generico e worked example DermaTriage corrente	5693f684c29fcc1a655605d77b85877d
64	11 - condition: prompt LLM per l'identificazione controllata	3cef93fdcd6726629e1dd667c5c0f775
65	11 - condition: output BA atteso e discriminazioni	7956aba2ac06ebcc3c7973d91b70469a
66	11 - condition: rappresentazione Mermaid e semantic determinism	e1c453cf3020816d9d9622231b489029
67	11 - condition: esempio grafico Mermaid composto con produce	ecda2667c8851cd4a8a88c0b5ab7a817
68	12 - decisionRule: definizione, smallest fact e criteri d'uso	0887305061dcc7dcf31ac52378909413
69	12 - decisionRule: forma canonica candidate e cardinalita	7940dcfe542defa1459ba46f4a56d57e
70	12 - decisionRule: identity boundary, invarianti e forbidden inference	b80eca2250f56d38b713aa1fe754a217
71	12 - decisionRule: domande di identificazione e procedura di estrazione	be8445e138ded46fedbb8c8239746469
72	12 - decisionRule: esempio generico e regressioni case A DermaTriage	94fe00fb66dfa7336e7273ec19e84522
73	12 - decisionRule: regression case B DermaTriage readiness C7	392dd1e9fdb0a1c0b7c9f4c4d0e6fd82
74	12 - decisionRule: pressure case C DermaTriage endpoint verification C7-P02	093249db9ac81d86e2947658411 6e1bf
75	12 - decisionRule: prompt LLM per l'identificazione controllata	1e416d6e8015f620685a9c6e48ae5527
76	12 - decisionRule: output candidate e discriminazioni	7f53974df56531f1056455e6992f5a1b
77	12 - decisionRule: rappresentazione Mermaid e visibility profile	ed7403904c7be9c068edc9e51c0db442
78	12 - decisionRule: esempio grafico DermaTriage candidate	686a906a0aa8c36f6d6d564c7ccbc9d7
79	13 - Store: definizione, smallest fact e criteri d'uso	7f04a38cba2726ed2c38669a57491ef1
80	13 - Store: forma canonica, profilo binding e cardinalita	7e09a4a32edab694e1b96da249779080
81	13 - Store: identity continuity, invarianti e forbidden inference	f4f2018409271760df58e2cbc0e349539
82	13 - Store: domande di identificazione, enrichment MR Decision FR e procedura	aabbe964a5f1ee9f87864f035745131e
83	13 - Store: esempio generico e pressure case DermaTriage RC001	1231009e9f6363f0ff1a91e379e66e2f
84	13 - Store: prompt LLM per identificazione ed enrichment controllati	2a701c771ac5753385b56105bab3c258
85	13 - Store: output atteso e discriminazioni con costrutti vicini	12d72ba8edfed0badf4a8d1bf10b958
86	13 - Store: rappresentazione Mermaid, direzioni e binding opzionali	f971ae4163101485d17a2525eed12930
87	13 - Store: esempio grafico con transfer IN e OUT	fca15cd7f890667a60f46932685dc9b9
88	14 - selection: definizione, smallest fact e criteri d'uso	7ca23dbdea726062717d92e8edcb2a90
89	14 - selection: forma canonica locale, ruoli e cardinalita	6d65eca46280537a5a2e0e5ec9bbe8d1
90	14 - selection: invarianti, confini semantici e forbidden inference	78d5aeca1bed00446ac85e4a81ea80fe
91	14 - selection: domande di identificazione e procedura di estrazione	b43df1bdbbe817b4927efb93a65ef290
92	14 - selection: esempio generico e positive control DermaTriage FR06	8ea61ece3848e2e6eae7458fe86118f1
93	14 - selection: positive control top-K e best senza full ranking	adf1abd8f97e3e52920d492b96aaf418
94	14 - selection: composizione con Store, contract e pressure case RC001	a0e7014efbf6db25c56ce330e60be766
95	14 - selection: prompt LLM per identificazione controllata	1c30ae85bc1f5ba814e03fec5da97dcfd
96	14 - selection: output atteso e discriminazioni con costrutti vicini	31418116d99a43c7196b8fcb428b7d62
97	14 - selection: rappresentazione Mermaid e esempio grafico con Store	0093149bd269138f15c2db83fea56324
98	15 - hasPart: definizione, smallest fact e criteri d'uso	e56c699b0e3814b7ee17c572f7ff19fc
99	15 - hasPart: firma canonica, ruoli e cardinalita	6b110c42481bf99d3b7042c4bd373159
100	15 - hasPart: identity continuity, invarianti e forbidden inference	f41a65968ca547a5a345f6c4cbd81495
101	15 - hasPart: domande di identificazione e procedura di estrazione	ae79b3fcdad6bce7bfb669880ea61643
102	15 - hasPart: esempio generico e regressioni case DermaTriage MR-C6	14152ea348ddde960ecaed4168736ee
103	15 - hasPart: prompt LLM per l'identificazione controllata	3a3739c47e7fe4959e76a26cff8f53549
104	15 - hasPart: output BA atteso e discriminazioni	2fbba520a32ed9a5a9543ad9d107d027
105	15 - hasPart: projection Mermaid a containment e occurrence multiple	74f217286e414a180c1737d8f6ec1e7e
106	15 - hasPart: regole grafiche e worked example containment	93b0ca8880dc4da9887259c4a75ed437a
107	16 - Boundary: definizione, neutralita semantica e criteri d'uso	5b69b4d552e3bf2b95a77b166a1f2952
108	16 - Boundary: semanticKind boundary e operatore boundaryOf	21ede95f599985ac1d63fb65f0fdc203
109	16 - crossesBoundary: proposition qualificante e riferimento a BAProposition	09003cdd9882afc94a6557fe42761f5a
110	16 - Boundary: evidence, invarianti e forbidden inference	47c0fe1cfe547096a372443c16325924
111	16 - Boundary: domande di identificazione, procedura e unresolved crossing	516087520bed492a8f301f8e0772dcba
112	16 - Boundary: regression DermaTriage GitHub Google Drive e Stage 2	2647f3048fb2ba53d2f26a44bdb1c5ca
113	16 - Boundary: output authoring discriminazioni e handoff security projection	73cec746d17ac4afb6e69a65ffc5312c
114	Appendice temporanea - scopo, lineage e confine del rebuild	3915eaeee02f714ac47e0dfa764b236f
115	Appendice temporanea - corpus e stati epistemici	5475a02c3223c3fac62a7eebe62f6be2
116	Appendice temporanea - Construct Description Contract e principio espositivo	e1b1becfc138e20474529fae47de6a05
117	Appendice temporanea - CDC direzione della spiegazione e revisione	435e9ecddb23d230df9cc32766e10610
118	Appendice temporanea - CDC campi 1-10	9d02d27cc2ad8bb4e1b30d441667e854
119	Appendice temporanea - CDC campi 11-20	90b6659e5a0105e0cd7268d3f76137f6
120	Appendice temporanea - roadmap e registro dei costrutti da integrare/studiare	84cfbc76debaca8141cc88702aa0179f
121	Indice di integrita' delle pagine	691d59fa456478a17e6ee36975680287
122	Appendice temporanea - laboratorio prove grafiche Mermaid da cancellare	13cdd31df41e1d75e405e6ba020261fa

PROVA GRAFICA TEMPORANEA - DA CANCELLARE

Scopo della pagina

Questa pagina e' un laboratorio grafico temporaneo e non stabilisce da sola le projection definitive. Deve essere eliminata quando le convenzioni saranno stabilizzate. La prova corrente conserva la vista composta `produce + transfer` nella quale `ClinicalDescription` resta un dato passivo: e' result della produzione e content del transfer, ma non genera frecce verso la destination. Le prove di operatori successivi possono essere aggiunte qui senza trasformare questa appendice in metodologia normativa.

Prova A - content visivamente tra source e transfer

La disposizione cercata mette `ClinicalDescription` fisicamente fra source e `transfer`. Il percorso attivo resta `source -> transfer -> destination`; la linea tratteggiata `content` e' una associazione di role e non un flusso.

```
flowchart LR
  S["ClinicalDescriptionGeneration<br/>actor / source"]
  C@{ shape: doc, label: "ClinicalDescription<br/>result / content" }
  T(("transfer"))
  D["HistoricalCaseRetrieval<br/>destination"]
  B["ClinicalDescriptionTransfer<br/>behavior"]

  S -->|"produce"| C
  S ==> T ==> D
  C -.-|"content"| T
  B -.-|"behavior"| T

  classDef endpoint fill:#EEF4F9,stroke:#275D86,color:#17324D
  classDef content fill:#FFFFFF,stroke:#275D86,color:#17324D
  classDef transfer fill:#DCEAF5,stroke:#275D86,color:#17324D
  classDef behavior fill:#FFF4E3,stroke:#A96A16,color:#17324D
  class S,D endpoint
  class C content
  class T transfer
  class B behavior
```

